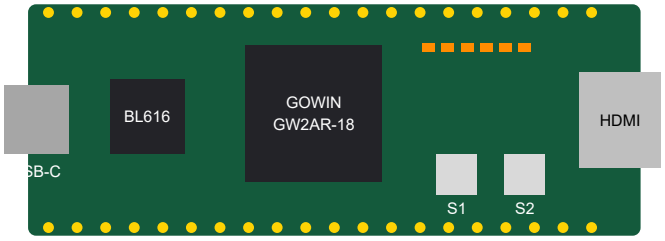
ПЛИС на практике

Sipeed Tang Nano 20K

Учебное пособие для начинающих
разработчиков цифровой аппаратуры

GW2AR-18 · 20 736 LUT4 · 64 Мбит SDRAM · Verilog



Корягин Сергей Андреевич

асп. СПбГЭТУ «ЛЭТИ», педагог ЦО «Кудрово»

г. Кудрово, 2026

Оглавление

Как читать это пособие 3

I. Основы цифровой схемотехники 5

1. Сигналы 6
- 1.1. Что такое сигнал . . . 6
  - 1.1.1. Почему цифровой . . . 6
- 1.2. Логические уровни . . . 7
- 1.3. Двоичная система счисления . . . 8
- 1.4. Временные диаграммы . . . 9
- 1.5. Тактовый сигнал . . . 9

2. Логические элементы и булева алгебра 11
- 2.1. Логические функции . . . 11
- 2.2. Основные элементы . . . 11
- 2.3. Как устроен элемент внутри . . . 12
- 2.4. Булева алгебра . . . 13
  - 2.4.1. Законы де Моргана . . . 13
- 2.5. Универсальные элементы . . . 14
- 2.6. Многовходовые элементы . . . 14

3. Комбинационные схемы 16
- 3.1. Комбинационная и последовательностная логика . . . 16
- 3.2. От словесного условия к схеме . . . 16
  - 3.2.1. Пример: голосование трёх судей . . . 17
- 3.3. Карты Карно . . . 18
- 3.4. Задержка распространения . . . 19
  - 3.4.1. Состязания и «иголки» . . . 19

4. Дешифраторы и шифраторы 21
- 4.1. Дешифратор . . . 21
  - 4.1.1. Где применяются дешифраторы . . . 22
  - 4.1.2. Дешифратор для семисегментного индикатора . . . 23

4.2. Шифратор . . . 24
4.2.1. Приоритетный шифратор . . . 24
4.3. Преобразователи кодов . . . 25

5. Мультиплексоры и демультиплексоры 27
5.1. Цифровой переключатель . . . 27
5.2. Мультиплексор 2:1 . . . 27
5.3. Мультиплексор 4:1 . . . 28
5.4. Дерево мультиплексоров . . . 29
5.5. Мультиплексор как универсальный логический элемент . . . 29
5.6. Демультиплексор . . . 30
5.7. Передача по одной линии . . . 30
5.8. Многоразрядные мультиплексоры . . . 31

6. Арифметические узлы 32
6.1. Сложение двоичных чисел . . . 32
6.2. Полусумматор . . . 32
6.3. Полный сумматор . . . 33
6.4. Многоразрядный сумматор . . . 33
6.5. Отрицательные числа и вычитание . . . 34
6.6. Компараторы . . . 35
6.7. Умножение и АЛУ . . . 36

7. Триггеры 38
7.1. Зачем схеме память . . . 38
7.2. Бистабильная ячейка . . . 38
7.3. RS-триггер . . . 38
7.4. D-защёлка . . . 39
7.5. D-триггер . . . 40
7.6. Временные параметры триггера . . . 41
7.7. Вход разрешения . . . 42
7.8. T-триггер и JK-триггер . . . 42

8. Регистры и счётчики 44
8.1. Регистр хранения . . . 44
8.2. Сдвиговый регистр . . . 45
8.2.1. Зачем сдвигать . . . 45
8.2.2. Кольцевой регистр и генератор псевдослучайных чисел . . . 46
8.3. Счётчики . . . 46
8.3.1. Асинхронный счётчик . . . 46
8.3.2. Синхронный счётчик . . . 47
8.3.3. Делитель частоты и счётчик по модулю . . . 47

9. Синхронные схемы, автоматы и память 49
9.1. Синхронное проектирование . . . 49
9.1.1. Максимальная тактовая частота . . . 49
9.1.2. Конвейер . . . 50
9.2. Конечные автоматы . . . 50
9.2.1. Пример: детектор последовательности . . . 51
9.2.2. Кодирование состояний . . . 52
9.3. Память . . . 52
9.3.1. ПЗУ: память только для чтения . . . 52
9.3.2. ОЗУ: оперативная память . . . 53
9.4. Что дальше . . . 53

II. ПЛИС на практике: Tang Nano 20K 55

10. Что такое ПЛИС 56
10.1. Схема, которую можно перепаять без паяльника . . . 56
10.2. Немного истории . . . 57
10.3. ПЛИС, микроконтроллер и заказная микросхема . . . 57
10.4. Где применяются ПЛИС . . . 58

11. Что внутри ПЛИС 60
11.1. Главная идея: таблица истинности как память . . . 60
11.2. Триггер: у схемы появляется память . . . 62
11.3. Логическая ячейка CFU . . . 62
11.4. Трассировочные ресурсы . . . 63
11.5. Специализированные блоки . . . 63
11.6. Где хранится конфигурация . . . 64

12. Плата Sipeed Tang Nano 20K 65
12.1. Общий вид . . . 65
12.2. Характеристики . . . 66
12.3. Структурная схема . . . 67
12.4. Выводы, которые понадобятся в примерах . . . 67

13. От текста до работающей схемы 69
13.1. Маршрут проектирования . . . 69
13.2. Что делает синтезатор . . . 70
13.3. Инструменты . . . 70
13.3.1. Gowin EDA (официальная среда) . . . 70
13.3.2. Открытые инструменты . . . 71
13.4. Файл ограничений .cst . . . 71
13.4.1. Файлы .cst в примерах . . . 72

13.4.2. Тактовая частота . . . 73

14. Основы языка Verilog 74
14.1. Код описывает схему, а не последовательность действий . . . 74
14.2. Модуль . . . 74
14.3. Сигналы: wire и reg . . . 75
14.3.1. Шины (многоразрядные сигналы) . . . 75
14.3.2. Запись чисел . . . 75
14.4. Операторы . . . 76
14.5. Блоки always . . . 76
14.5.1. Комбинационный: always @(*) . . . 76
14.5.2. Последовательностный: always @(posedge clk) . . . 76
14.5.3. Блокирующее (=) и неблокирующее (<=) присваивание . . . 77
14.6. Условия: if и case . . . 77
14.7. Параметры и константы . . . 78
14.8. Иерархия: модуль внутри модуля . . . 78

15. Практика: семь проектов 79
15.1. Проект 1. Логические элементы на кнопках . . . 79
15.2. Проект 2. Мигающий светодиод . . . 81
15.3. Проект 3. Бегущий огонь . . . 82
15.4. Проект 4. Кнопка: синхронизация, дребезг, фронт . . . 83
15.4.1. Препятствие 1: дребезг контактов . . . 83
15.4.2. Препятствие 2: метастабильность . . . 83
15.4.3. Препятствие 3: нажатие длится миллионы тактов . . . 84
15.5. Проект 5. Светофор: конечный автомат . . . 85
15.6. Проект 6. «Дышащий» светодиод: ШИМ . . . 86
15.7. Проект 7. Дешифратор, мультиплексор и демультиплексор . . . 88

16. Симуляция: проверяем схему до загрузки 90
16.1. Зачем нужна симуляция . . . 90
16.2. Тестбенч . . . 90
16.3. Пример 1: таблица истинности проекта 1 . . . 91
16.4. Пример 2: мигалка с уменьшенной частотой . . . 91
16.5. Временные диаграммы в GTKWave . . . 91

17. Память SDRAM на 64 Мбит 93
17.1. Зачем ПЛИС внешняя память . . . 93
17.2. Как устроена DRAM . . . 93
17.3. Организация памяти . . . 94
17.4. Команды SDRAM . . . 94

17.5. Контроллер SDRAM 95
17.5.1. Порты встроенной памяти 96
17.6. Пропускная способность 96

18. Задания для самостоятельной работы **98**

A. Шпаргалка **100**
A.1. Verilog на одной странице 100
A.2. Типичные ошибки 100
A.3. Выводы Tang Nano 20K 100
A.4. Полезные ссылки 101

Как читать это пособие

Это пособие рассчитано на тех, кто только начинает знакомство с цифровой электроникой. Предварительных знаний почти не требуется: достаточно школьной математики и желания разобраться, как «думают» компьютеры, контроллеры дронов и любые другие цифровые устройства.

Книга состоит из двух частей.

Часть I. Основы цифровой схемотехники. Что такое сигнал и почему техника цифровая; логические элементы и булева алгебра; комбинационные схемы; типовые узлы — дешифраторы, шифраторы, мультиплексоры, демультиплексоры, сумматоры, компараторы; триггеры, регистры, счётчики; синхронные схемы, конечные автоматы и память. Эта часть не привязана к конкретной микросхеме и служит общей базой для всех наших курсов.

Часть II. ПЛИС на практике: Tang Nano 20K. Что такое ПЛИС и что у неё внутри; плата Sipeed Tang Nano 20K; путь от текста до работающей схемы; язык Verilog; семь законченных проектов; симуляция; встроенная память SDRAM; задания для самостоятельной работы.

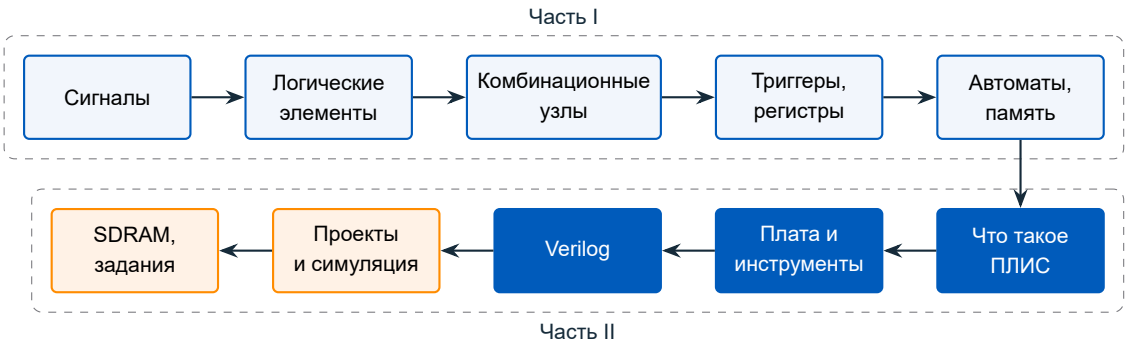


Рис. 1.: Путь, который мы пройдём

Если вы уже изучили основы цифровой схемотехники в другом нашем курсе, можете сразу переходить ко второй части и возвращаться к первой как к справочнику: во второй части есть ссылки на нужные главы первой.

Все исходные тексты примеров лежат рядом с книгой, в папке `personal/fpga/tang_nano_20k/code`. Их можно открыть, скопировать и сразу загрузить в плату.

Обозначения

- Синие блоки «Справка» содержат полезные подробности.
- Оранжевые блоки «Внимание!» предупреждают о типичных ошибках.
- Фиолетовые блоки «Главная мысль» подводят итог раздела.
- Зелёные блоки «Задание» предлагают что-то сделать самостоятельно.

Часть I.

Основы цифровой схемотехники

1 Сигналы

1.1. Что такое сигнал

Сигнал — это физическая величина, которая меняется во времени и несёт информацию. В электронике чаще всего это *напряжение* на проводе. Микрофон превращает звук в изменяющееся напряжение, датчик температуры — температуру в напряжение, кнопка — нажатие в напряжение.

Сигналы бывают двух видов.

Аналоговый сигнал может принимать *любое* значение в некотором диапазоне: 1,2 В, 1,21 В, 1,215 В… Так выглядит звук, сигнал с микрофона или датчика.

Цифровой сигнал принимает только несколько разрешённых значений. В двоичной цифровой технике их два: **логический 0** (низкое напряжение) и **логическая 1** (высокое напряжение).

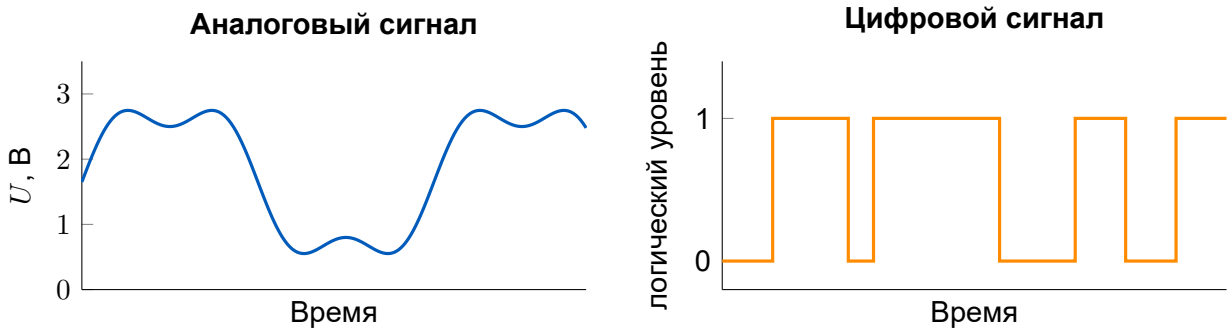


Рис. 1.1.: Аналоговый сигнал меняется плавно, цифровой — скачками между двумя уровнями

1.1.1. Почему цифровой

Почему вся современная вычислительная техника цифровая? Главная причина — **устойчивость к помехам**. Любой провод ловит помехи: от соседних проводов, от моторов, от радиоволн. В аналоговом сигнале помеха искажает саму информацию, и отделить её от полезного сигнала уже нельзя. В цифровом сигнале нас интересует только одно: *выше или ниже порога* напряжение. Небольшая помеха ничего не меняет.

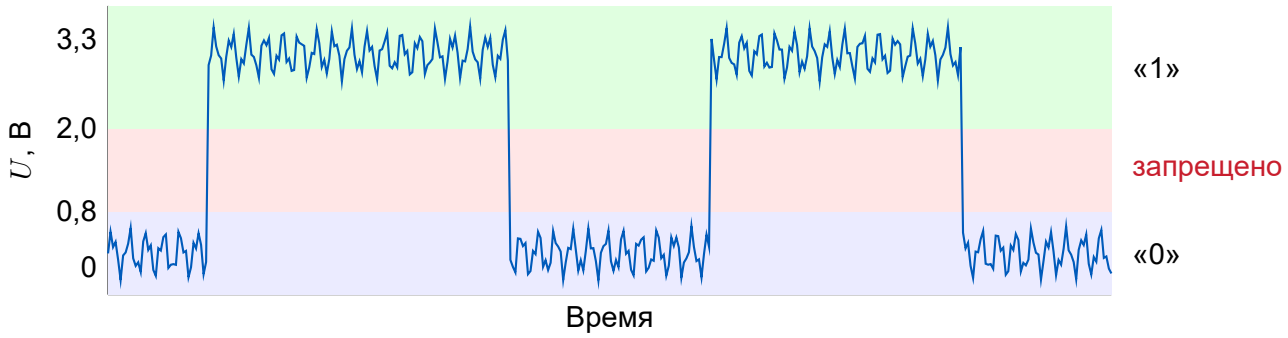


Рис. 1.2.: Сигнал с помехами. Пока напряжение не заходит в запрещённую зону, приёмник безошибочно отличает 0 от 1 (пороги — для стандарта LVCMOS 3,3 В)

1.2. Логические уровни

Какое напряжение считать нулём, а какое — единицей, определяет **стандарт логических уровней**. ПЛИС на плате Tang Nano 20K использует стандарт **LVCMOS33** (питание 3,3 В):

Таблица 1.1.: Уровни LVCMOS 3,3 В

Параметр	Значение	Смысл
$U_{IL,max}$	0,8 В	всё, что ниже, вход считает нулём
$U_{IH,min}$	2,0 В	всё, что выше, вход считает единицей
$U_{OL,max}$	0,4 В	выход выдаёт «0» не выше этого напряжения
$U_{OH,min}$	2,4 В	выход выдаёт «1» не ниже этого напряжения

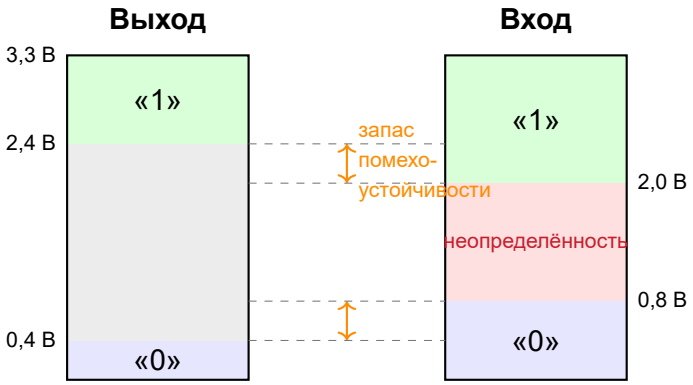


Рис. 1.3.: Выход гарантирует уровни «с запасом», поэтому помеха в 0,4 В не приводит к ошибке

Активный высокий и активный низкий уровень

Обычно «действие» происходит при единице: 1 — светодиод горит, 1 — мотор включён. Это **активный высокий уровень**. Но часто бывает наоборот: сигнал сброса *срабатывает при нуле*. Такие сигналы называют сигналами с **активным низким уровнем** и помечают чертой сверху ($\overline{\text{RESET}}$), решёткой (RESET#) или окончанием _n

(rst_n). На плате Tang Nano 20K светодиоды именно такие — они горят от нуля.

1.3. Двоичная система счисления

Один цифровой провод передаёт один **бит** — 0 или 1. Чтобы передать число больше единицы, берут несколько проводов (битов) и используют **двоичную систему счисления**. Каждый разряд имеет вес — степень двойки:

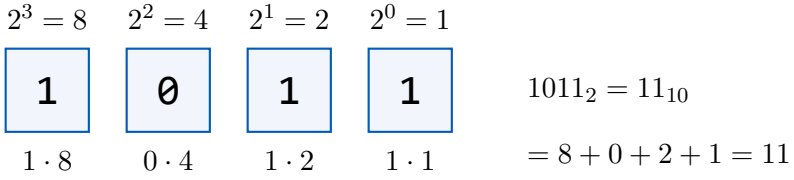


Рис. 1.4.: Перевод двоичного числа в десятичное

С помощью n битов можно записать 2^n разных чисел: от 0 до $2^n - 1$. Четыре бита дают 16 комбинаций, восемь (**байт**) — 256. Длинные двоичные числа неудобно читать, поэтому их записывают в **шестнадцатеричной** системе: каждые 4 бита заменяются одной цифрой от 0 до F.

Таблица 1.2.: Числа от 0 до 15 в трёх системах счисления

Дес.	Двоич.	Шестн.	Дес.	Двоич.	Шестн.
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

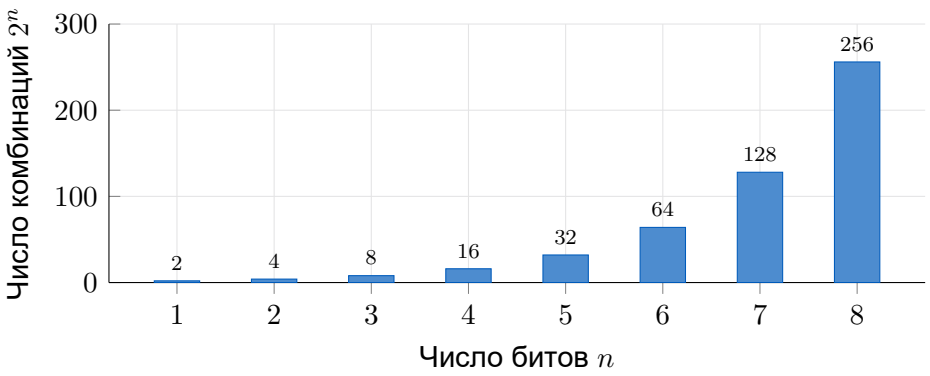


Рис. 1.5.: Каждый новый бит удваивает число возможных значений

1.4. Временные диаграммы

Чтобы показать, как сигналы меняются во времени, рисуют **временные диаграммы**: по горизонтали — время, по вертикали — уровень. Переход из 0 в 1 называют **передним фронтом** (или просто фронтом), переход из 1 в 0 — **задним фронтом** (или спадом).

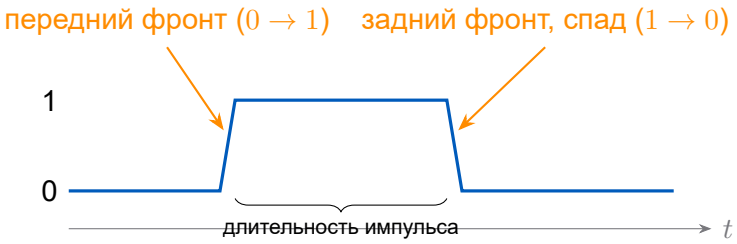


Рис. 1.6.: Импульс на временной диаграмме

Если на диаграмме несколько проводов объединены в число, их рисуют «шиной»: две линии, между которыми написано значение, а крестик обозначает момент смены значения.

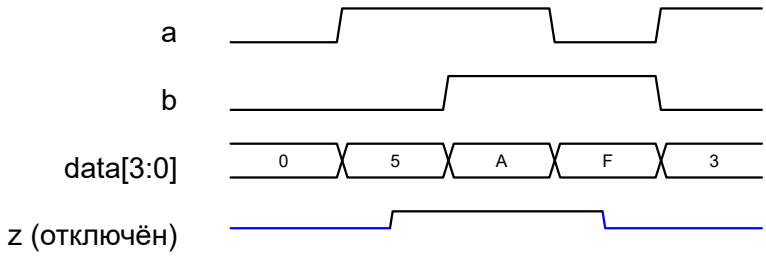


Рис. 1.7.: Одиночные сигналы, шина и сигнал в третьем (отключённом) состоянии Z

1.5. Тактовый сигнал

Особую роль в цифровой технике играет **тактовый сигнал** (clock, `clk`) — последовательность импульсов, которые идут строго через равные промежутки времени. Он работает как дирижёр: по его фронтам все элементы памяти схемы одновременно обновляют свои значения.

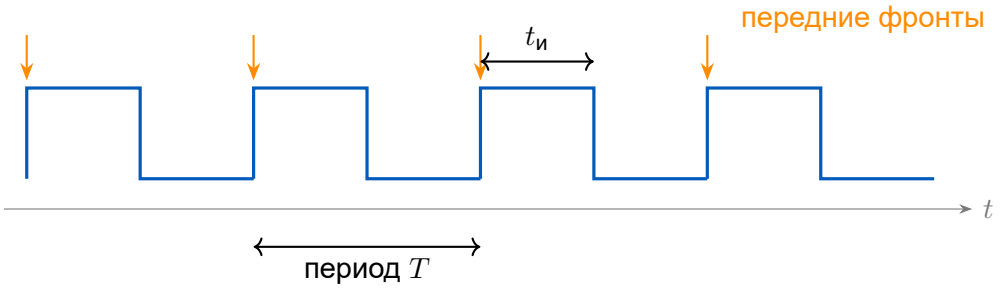


Рис. 1.8.: Тактовый сигнал: период T, длительность импульса $t_и$

Основные характеристики тактового сигнала:

$$f = \frac{1}{T}, \qquad D = \frac{t_{\text{и}}}{T} \cdot 100\,\%.$$

Здесь f — **частота** (сколько периодов в секунду, измеряется в герцах), D — **коэффициент заполнения** (обычно 50 %). На плате Tang Nano 20K стоит генератор на 27 МГц:

$$T = \frac{1}{27\,000\,000\ \text{Гц}} \approx 37 \cdot 10^{-9}\ \text{с} = 37\ \text{нс}.$$

За одну секунду происходит 27 миллионов тактов. За время одного моргания глаза (около 0,1 с) — почти три миллиона.

Таблица 1.3.: Приставки для частоты и времени

Частота	Период	Пример
1 Гц	1 с	мигание светодиода
1 кГц = 10^3 Гц	1 мс = 10^{-3} с	звуковой сигнал «писк»
1 МГц = 10^6 Гц	1 мкс = 10^{-6} с	простые микроконтроллеры
27 МГц	37 нс	генератор Tang Nano 20K
1 ГГц = 10^9 Гц	1 нс = 10^{-9} с	процессоры компьютеров

Главная мысль

Цифровой сигнал имеет два уровня: 0 и 1. Несколько битов образуют двоичное число. Изменения сигналов во времени изображают временными диаграммами, а тактовый сигнал задаёт ритм, в котором работает вся схема.

2 Логические элементы и булева алгебра

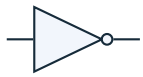
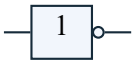
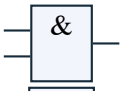
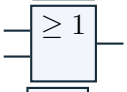
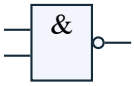
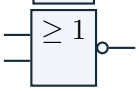
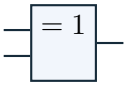
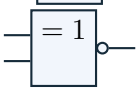
2.1. Логические функции

Логическая функция ставит в соответствие каждой комбинации входных битов значение выхода: 0 или 1. Функцию удобно задавать **таблицей истинности** — перечнем всех комбинаций входов и значений выхода. У функции от n входов в таблице 2^n строк.

Схема, которая вычисляет логическую функцию, называется **логическим элементом** (вентилем). Логические элементы изображают условными знаками. В нашей стране принят стандарт ГОСТ (прямоугольник с символом внутри), в зарубежной литературе и в программах часто используют американские (ANSI) знаки. Полезно узнавать оба.

2.2. Основные элементы

Таблица 2.1.: Основные логические элементы. Столбец «таблица» — значения выхода для входов $ab = 00, 01, 10, 11$

ANSI	ГОСТ / IEC	Название	Формула	Таблица
		НЕ (инверсия)	$y = \overline{a}$	`10` (для $a = 0, 1$)
		И (конъюнкция)	$y = a \cdot b$	`0001`
		ИЛИ (дизъюнкция)	$y = a + b$	`0111`
		И-НЕ (штрих Шеффера)	$y = \overline{a \cdot b}$	`1110`
		ИЛИ-НЕ (стрелка Пирса)	$y = \overline{a + b}$	`1000`
		Исключающее ИЛИ	$y = a \oplus b$	`0110`
		Исключающее ИЛИ-НЕ	$y = \overline{a \oplus b}$	`1001`

Кружок на выходе элемента обозначает инверсию. Поэтому И-НЕ рисуют как элемент И с кружком, ИЛИ-НЕ — как ИЛИ с кружком.

Словами:

- **НЕ**: выход противоположен входу;
- **И**: выход равен 1, только если *все* входы равны 1;
- **ИЛИ**: выход равен 1, если *хотя бы один* вход равен 1;
- **Исключающее ИЛИ**: выход равен 1, если входы *различны* (для многих входов — если число единиц нечётно).

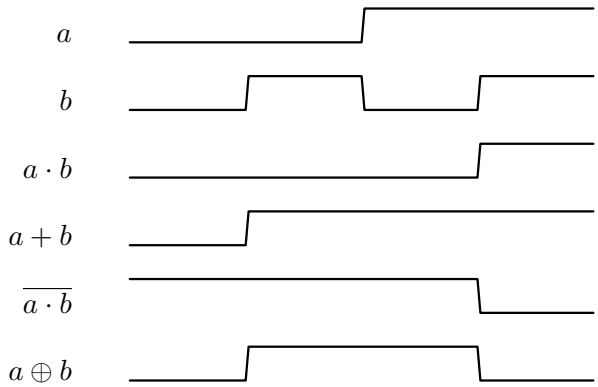


Рис. 2.1.: Работа элементов на временной диаграмме: перебираются все четыре комбинации входов

2.3. Как устроен элемент внутри

Внутри микросхем логические элементы строятся из транзисторов. В современной технологии КМОП (CMOS) используются пары транзисторов двух типов: p-канальный открыт при нуле на затворе, n-канальный — при единице. Самый простой элемент — инвертор — состоит всего из двух транзисторов.

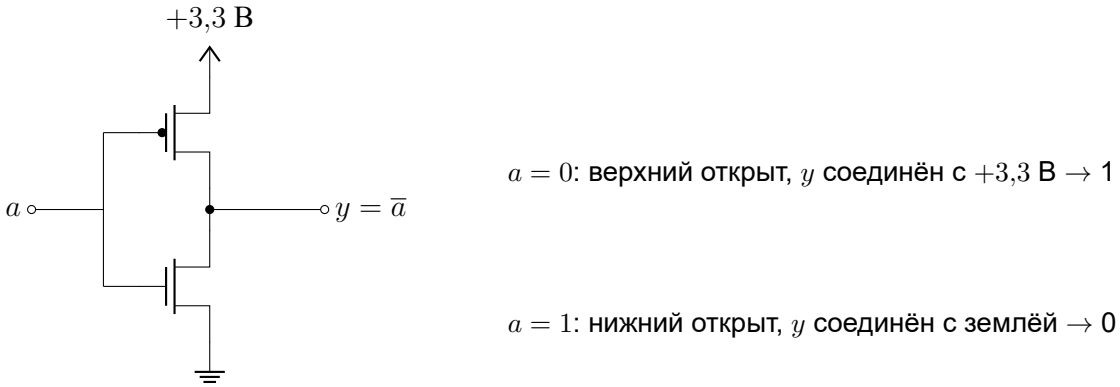


Рис. 2.2.: КМОП-инвертор: два транзистора работают как переключатели

Элементы И-НЕ и ИЛИ-НЕ получаются из четырёх транзисторов, а И и ИЛИ — это И-НЕ и ИЛИ-НЕ с инвертором на выходе (шесть транзисторов). Поэтому в микросхемах самыми «дешёвыми» оказываются именно инвертирующие элементы.

2.4. Булева алгебра

С логическими выражениями можно обращаться почти как с обычными алгебраическими: раскрывать скобки, выносить общий множитель, сокращать. Этими правилами занимается **булева алгебра**, названная в честь английского математика Джорджа Буля. Операцию И записывают как умножение ($a \cdot b$ или ab), ИЛИ — как сложение ($a + b$), НЕ — чертой сверху ($\overline{a}$).

Таблица 2.2.: Основные законы булевой алгебры

Закон	**Для И**	**Для ИЛИ**
Действия с константами	$a \cdot 0 = 0,\ a \cdot 1 = a$	$a + 1 = 1,\ a + 0 = a$
Повторение	$a \cdot a = a$	$a + a = a$
Дополнение	$a \cdot \overline{a} = 0$	$a + \overline{a} = 1$
Двойное отрицание		$\overline{\overline{a}} = a$
Переместительный	$ab = ba$	$a + b = b + a$
Сочетательный	$(ab)c = a(bc)$	$(a + b) + c = a + (b + c)$
Распределительный	$a(b + c) = ab + ac$	$a + bc = (a + b)(a + c)$
Поглощение	$a(a + b) = a$	$a + ab = a$
Законы де Моргана	$\overline{a \cdot b} = \overline{a} + \overline{b}$	$\overline{a + b} = \overline{a} \cdot \overline{b}$

Внимание: необычный закон

Второй распределительный закон $a + bc = (a + b)(a + c)$ не выполняется в обычной арифметике, но верен в булевой алгебре. Проверьте его таблицей истинности!

2.4.1. Законы де Моргана

Законы де Моргана — самые полезные на практике. Они говорят: *инверсию над выражением можно «протащить» внутрь, заменив И на ИЛИ и наоборот*. Графически это значит, что элемент И-НЕ можно нарисовать как ИЛИ с инверсиями на входах:

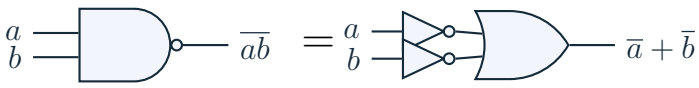


Рис. 2.3.: Закон де Моргана в виде схемы

Таблица 2.3.: Проверка закона де Моргана таблицей истинности

a	b	$\overline{a \cdot b}$	$\overline{a}$	$\overline{b}$	$\overline{a} + \overline{b}$
0	0	**1**	1	1	**1**
0	1	**1**	1	0	**1**
1	0	**1**	0	1	**1**
1	1	**0**	0	0	**0**

2.5. Универсальные элементы

Удивительный факт: **из одних только элементов И-НЕ можно собрать любую логическую схему**. То же верно для ИЛИ-НЕ. Такие элементы называют **универсальными**. Достаточно показать, как из И-НЕ получить НЕ, И и ИЛИ — а из них уже строится всё остальное.

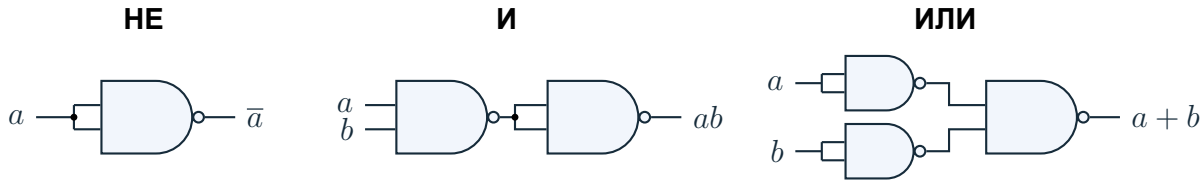


Рис. 2.4.: НЕ, И и ИЛИ, собранные только из элементов И-НЕ

Проверим схему ИЛИ по законам де Моргана: $\overline{\overline{a} \cdot \overline{b}} = \overline{\overline{a}} + \overline{\overline{b}} = a + b$.

Зачем это нужно

В эпоху микросхем серии К155 (ТТЛ) разработчики часто собирали целые устройства из микросхем К155ЛА3, в каждой из которых четыре элемента И-НЕ. Так схема получалась дешевле и требовала меньше разных деталей. Внутри ПЛИС вопрос решён иначе — там используются таблицы истинности (глава 11).

2.6. Многовходовые элементы

Элементы И, ИЛИ и Исключающее ИЛИ могут иметь больше двух входов. Их можно собрать из двухвходовых, соединив «цепочкой» или «деревом»:

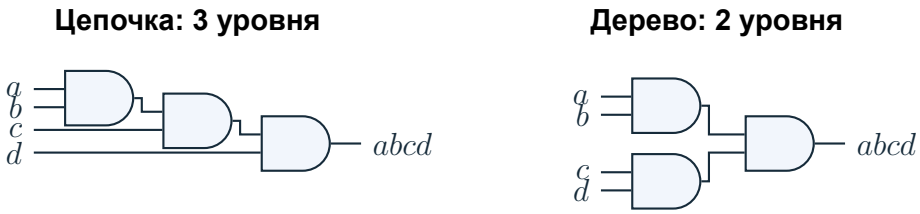


Рис. 2.5.: Четырёхвходовый элемент И из двухвходовых. Дерево работает быстрее: сигнал проходит через два элемента, а не через три

Главная мысль

Семь основных логических элементов (НЕ, И, ИЛИ, И-НЕ, ИЛИ-НЕ, Исключающее ИЛИ и Исключающее ИЛИ-НЕ) задаются таблицами истинности. Выражения с ними упрощаются по законам булевой алгебры, а законы де Моргана позволяют менять И на ИЛИ. Элементы И-НЕ и ИЛИ-НЕ универсальны: из любого из них собирается любая схема.

3 Комбинационные схемы

3.1. Комбинационная и последовательностная логика

Все цифровые схемы делятся на два больших класса.

Комбинационные схемы — выходы зависят *только от текущих значений входов*. У них нет памяти: подайте те же входы — получите те же выходы, неважно, что было раньше. Примеры: логические элементы, дешифраторы, мультиплексоры, сумматоры.

Последовательностные схемы — выходы зависят не только от текущих входов, но и от *предыстории*, то есть от того, что было раньше. У них есть память. Примеры: триггеры, регистры, счётчики. О них — в главах 7–9.

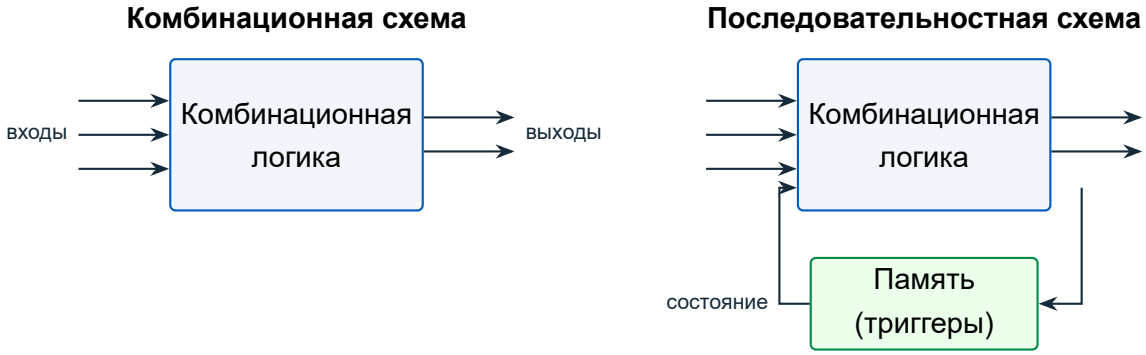


Рис. 3.1.: Главное отличие: у последовательностной схемы есть память и обратная связь через неё

3.2. От словесного условия к схеме

Разработка комбинационной схемы всегда идёт по одному плану:

1. понять, какие входы и выходы нужны, и дать им имена;
2. составить **таблицу истинности**;
3. записать логическое выражение;
4. **упростить** его;
5. нарисовать схему.

3.2.1. Пример: голосование трёх судей

На соревнованиях работают три судьи: a, b, c. Каждый нажимает кнопку, если засчитывает попытку. Лампа y должна загораться, если попытку засчитали *хотя бы двое*. Такая функция называется **мажоритарной** (от лат. *major* — большинство).

Шаг 1–2. Таблица истинности. Три входа — $2^3 = 8$ строк:

Таблица 3.1.: Таблица истинности мажоритарной функции

№	a	b	c	y	Слагаемое
0	0	0	0	0	
1	0	0	1	0	
2	0	1	0	0	
3	0	1	1	**1**	$\overline{a}bc$
4	1	0	0	0	
5	1	0	1	**1**	$a\overline{b}c$
6	1	1	0	**1**	$ab\overline{c}$
7	1	1	1	**1**	abc

Шаг 3. Выражение. Для каждой строки, где $y = 1$, запишем произведение всех входов: вход берётся как есть, если он равен 1, и с инверсией, если равен 0. Такое произведение равно единице *только* в этой строке. Сложив все такие произведения, получим функцию. Эта форма записи называется **совершенной дизъюнктивной нормальной формой** (СДНФ):

$$y = \overline{a}bc + a\overline{b}c + ab\overline{c} + abc.$$

Шаг 4. Упрощение. Слагаемое abc можно повторить сколько угодно раз ($x + x = x$). Добавим его ещё дважды и сгруппируем:

$$y = (\overline{a}bc + abc) + (a\overline{b}c + abc) + (ab\overline{c} + abc) = bc(\overline{a} + a) + ac(\overline{b} + b) + ab(\overline{c} + c) = bc + ac + ab.$$

Вместо четырёх трёхвходовых элементов И и трёх инверторов нужно всего три двухвходовых элемента И и один элемент ИЛИ.

Шаг 5. Схема.

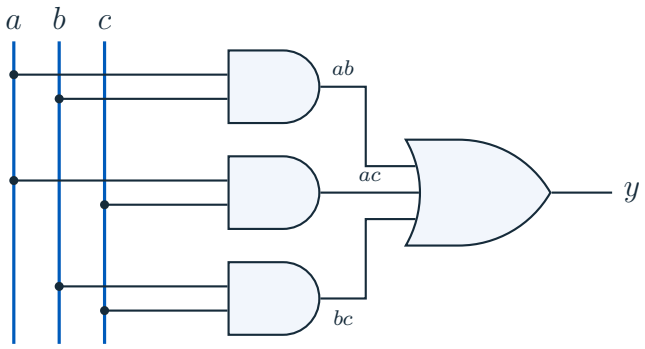


Рис. 3.2.: Мажоритарный элемент «2 из 3»: $y = ab + ac + bc$

3.3. Карты Карно

Упрощать выражения алгебраически можно, но легко ошибиться. Для функций от 2–4 переменных есть наглядный способ — **карта Карно**. Это та же таблица истинности, но нарисованная в виде прямоугольника так, что *соседние клетки отличаются значением только одной переменной*. Поэтому столбцы подписаны в порядке 00, 01, 11, 10 (код Грея), а не 00, 01, 10, 11.

Правило упрощения: единицы объединяют в прямоугольные группы по 2, 4, 8 клеток (степень двойки), стараясь сделать группы как можно больше. Каждой группе соответствует одно произведение, в котором остаются только те переменные, что *не меняются* внутри группы.

$a\backslash bc$	00	01	11	10
0	0	0	1	0
1	0	1	1	1

bc — столбец $bc = 11$ (a меняется)

ac — клетки $abc = 101, 111$

ab — клетки $abc = 111, 110$

$y = ab + ac + bc$

Рис. 3.3.: Карта Карно мажоритарной функции: три группы по две клетки дают три слагаемых

Кто будет упрощать схемы в ПЛИС?

Когда вы будете работать с ПЛИС, упрощение выполняет программа-синтезатор, причём гораздо лучше человека. Но понимать, *как* это делается, всё равно полезно: это помогает оценить, сколько ресурсов займёт ваша схема и насколько быстро она будет работать.

3.4. Задержка распространения

До сих пор мы считали, что элемент срабатывает мгновенно. На самом деле транзисторам нужно время, чтобы переключиться, а проводам — чтобы зарядиться. Выход элемента меняется через время $t_{зд}$ после входа. Это **задержка распространения**. У современных микросхем она составляет от долей наносекунды до нескольких наносекунд.

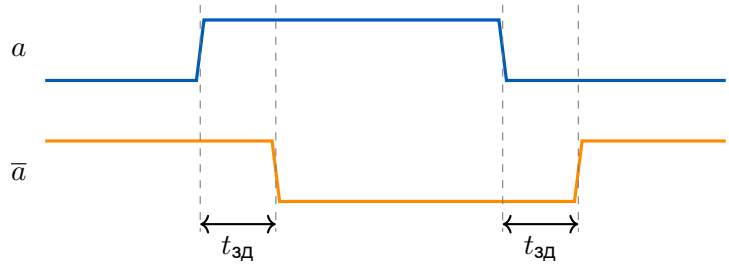


Рис. 3.4.: Выход инвертора меняется с задержкой $t_{зд}$

Если сигнал проходит через несколько элементов подряд, задержки складываются. Самый длинный путь от входа до выхода называется **критическим путём**. Он определяет, насколько быстро может работать схема.

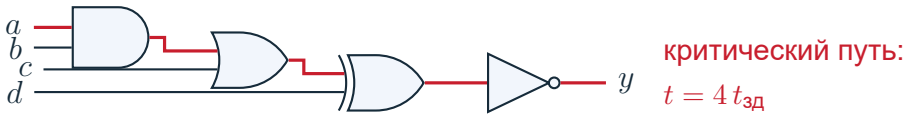


Рис. 3.5.: Критический путь проходит через четыре элемента

3.4.1. Состязания и «иголки»

Из-за задержек на выходе комбинационной схемы могут появляться короткие ложные импульсы — **иголки** (glitches). Классический пример: $y = a \cdot \overline{a}$. По законам алгебры y всегда равен 0. Но инвертор срабатывает с задержкой, и в течение короткого времени после фронта a *оба* входа элемента И равны 1.

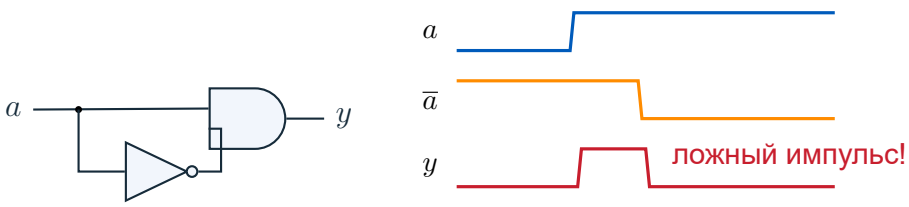


Рис. 3.6.: Возникновение «иголки» из-за задержки инвертора

Как с этим бороться

Иголки — главная причина, по которой в серьёзных схемах выходы комбинационной логики «читают» не когда попало, а только по фронту тактового сигнала, когда все переходные процессы уже закончились. Это называется **синхронным проектированием**

(глава 9), и все схемы для ПЛИС строятся именно так.

Главная мысль

Комбинационная схема не имеет памяти. Её проектируют по таблице истинности, упрощают алгебраически или по карте Карно. Реальные элементы имеют задержку, из-за которой возможны кратковременные ложные импульсы.

4 Дешифраторы и шифраторы

Из логических элементов строят более крупные «кирпичи» — типовые комбинационные узлы. Они встречаются почти в любом цифровом устройстве, и их полезно узнавать с первого взгляда. Начнём с узлов, которые преобразуют один способ записи числа (код) в другой.

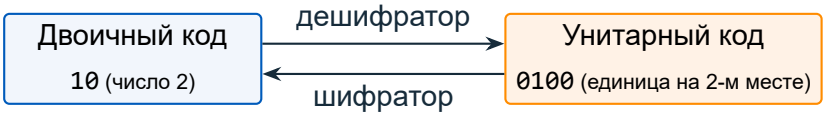
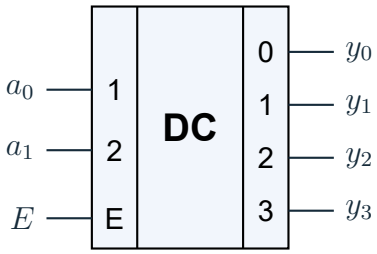


Рис. 4.1.: Дешифратор и шифратор выполняют взаимно обратные преобразования

Унитарный код (one-hot, «одна горячая») — код, в котором из N проводов единица стоит ровно на одном, а номер этого провода и есть передаваемое значение. Он удобен, чтобы что-то *выбрать*: включить одну из нескольких ламп, одну из нескольких микросхем памяти.

4.1. Дешифратор

Дешифратор (decoder) имеет n входов и 2^n выходов. Входы задают двоичный номер, и на выходе с этим номером появляется 1, а на всех остальных — 0. На схемах дешифратор обозначают буквами DC.



E	a_1	a_0	y_0	y_1	y_2	y_3
0	×	×	0	0	0	0
1	0	0	**1**	0	0	0
1	0	1	0	**1**	0	0
1	1	0	0	0	**1**	0
1	1	1	0	0	0	**1**

× — «неважно, 0 или 1»

Рис. 4.2.: Дешифратор 2→4 с входом разрешения E: обозначение и таблица истинности

Вход E (Enable, разрешение) позволяет «выключить» дешифратор: при $E = 0$ все выходы равны нулю, какой бы номер ни был подан.

Как построить дешифратор из элементов? Каждый выход — это произведение, равное единице ровно для одной комбинации входов:

$$y_0 = E\,\overline{a_1}\,\overline{a_0}, \quad y_1 = E\,\overline{a_1}\,a_0, \quad y_2 = E\,a_1\,\overline{a_0}, \quad y_3 = E\,a_1\,a_0.$$

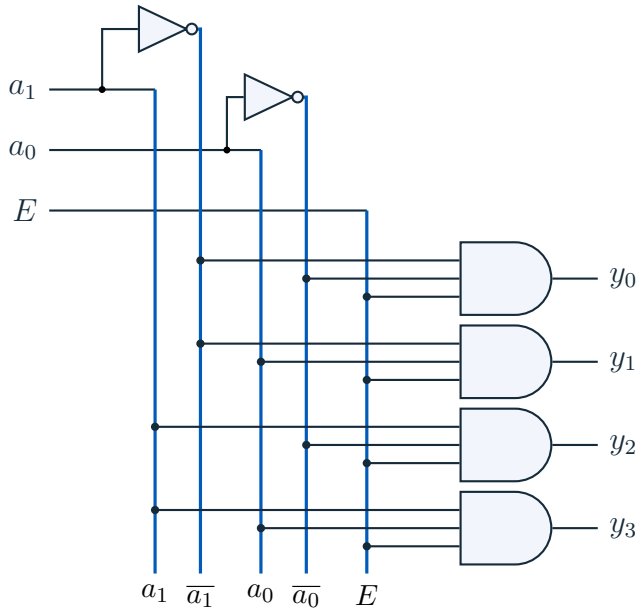


Рис. 4.3.: Дешифратор 2→4 на элементах И и НЕ

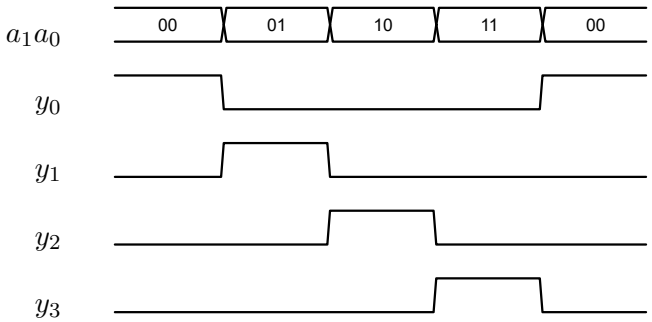


Рис. 4.4.: При переборе номеров единица «переходит» с одного выхода на другой

4.1.1. Где применяются дешифраторы

- **Выбор устройства по адресу.** В компьютере к одной шине подключено много микросхем памяти и периферии. Дешифратор по старшим битам адреса выбирает, какая из них сейчас должна ответить.
- **Управление индикаторами**: включение одного из нескольких светодиодов, разрядов индикатора, строк матрицы.
- **Выбор команды в процессоре**: код операции дешифрируется в управляющие сигналы «сложить», «прочитать из памяти» и т. д.

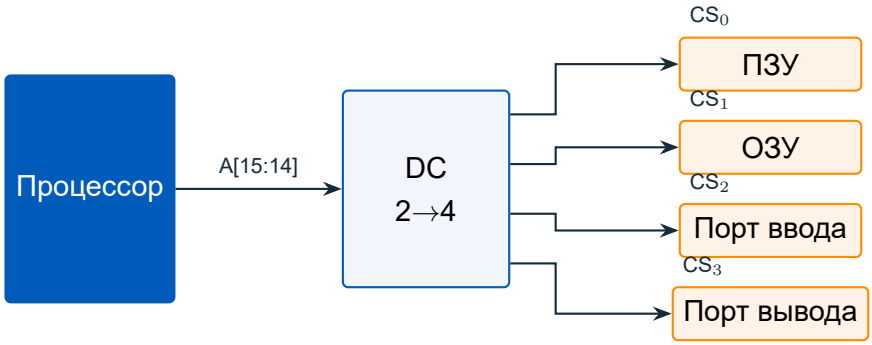


Рис. 4.5.: Дешифратор адреса: два старших бита адреса выбирают одну из четырёх микросхем

4.1.2. Дешифратор для семисегментного индикатора

Семисегментный индикатор состоит из семи светодиодных полосок, обозначаемых буквами от a до g. Зажигая разные сегменты, можно показать любую цифру. Узел, который переводит двоичный код цифры в сигналы для сегментов, тоже называют дешифратором (хотя он выдаёт не унитарный код).

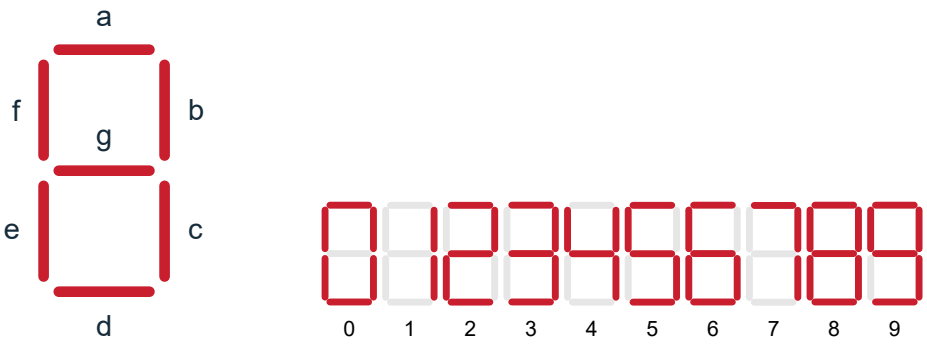


Рис. 4.6.: Семисегментный индикатор: обозначение сегментов и изображение цифр

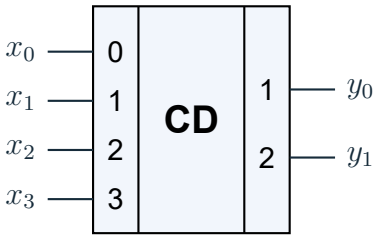
Таблица 4.1.: Таблица истинности дешифратора для семисегментного индикатора (1 — сегмент горит)

Цифра	x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

Каждый столбец a–g — это отдельная логическая функция от четырёх переменных. Её можно упростить по карте Карно и собрать из элементов, как в главе 3.

4.2. Шифратор

Шифратор (encoder, на схемах обозначается CD) выполняет обратное преобразование: из унитарного кода на 2^n входах делает n-разрядный двоичный номер той линии, на которой стоит единица.



x_0	x_1	x_2	x_3	y_1	y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

Рис. 4.7.: Шифратор 4→2: обозначение и таблица истинности

По таблице видно: y_0 равен 1, когда активна линия 1 или 3; y_1 — когда активна линия 2 или 3. Значит, шифратор собирается из одних элементов ИЛИ:

$$y_0 = x_1 + x_3, \qquad y_1 = x_2 + x_3.$$

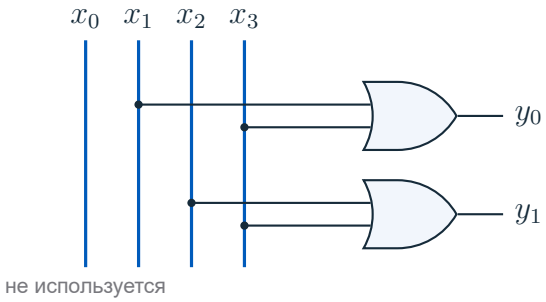


Рис. 4.8.: Шифратор 4→2 из двух элементов ИЛИ

4.2.1. Приоритетный шифратор

Что будет, если единица окажется сразу на двух входах, например, если одновременно нажаты кнопки 1 и 2? Простой шифратор выдаст $y_1 y_0 = 11$, то есть номер 3, — явную ошибку. Поэтому на практике применяют **приоритетный шифратор**: он выдаёт номер *старшего* из активных входов, а остальные игнорирует. Дополнительный выход V (valid) показывает, что хотя бы один вход активен.

Таблица 4.2.: Приоритетный шифратор 4→2 (× — значение не важно)

x_3	x_2	x_1	x_0	y_1	y_0	V
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	×	0	1	1
0	1	×	×	1	0	1
1	×	×	×	1	1	1

Выражения: $y_1 = x_3 + x_2$, $y_0 = x_3 + \overline{x_2}\,x_1$, $V = x_3 + x_2 + x_1 + x_0$.

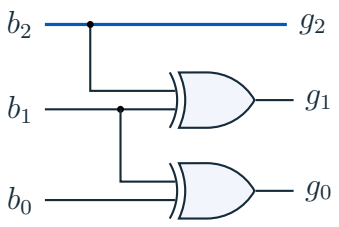
Где применяются шифраторы

- **Клавиатуры**: номер нажатой клавиши превращается в код.
- **Контроллеры прерываний**: из нескольких одновременных запросов процессору передаётся номер самого важного.
- **Датчики положения**: из показаний линейки фотодатчиков получают число — координату.

4.3. Преобразователи кодов

Дешифраторы и шифраторы — частные случаи **преобразователей кодов**. Ещё один полезный пример — перевод двоичного кода в **код Грея**, в котором соседние числа отличаются ровно одним битом. Это важно для датчиков угла поворота: при переходе от 3 (`011`) к 4 (`100`) в двоичном коде меняются сразу три бита, и в промежуточный момент датчик может выдать что угодно. В коде Грея меняется только один бит.

Число	Двоичный	Грея
0	`000`	`000`
1	`001`	`001`
2	`010`	`011`
3	`011`	`010`
4	`100`	`110`
5	`101`	`111`
6	`110`	`101`
7	`111`	`100`



$g_i = b_{i+1} \oplus b_i$

Рис. 4.9.: Двоичный код и код Грея; преобразователь на элементах «Исключающее ИЛИ»

Главная мысль

Дешифратор: номер $\rightarrow$ единица на одном из 2^n выходов. **Шифратор**: единица на одном из входов $\rightarrow$ её номер. **Приоритетный шифратор** при нескольких активных входах выбирает старший.

5 Мультиплексоры и демультиплексоры

5.1. Цифровой переключатель

Представьте старый галетный переключатель: ручка поворачивается и соединяет общий контакт с одним из нескольких. **Мультиплексор** (MUX) делает то же самое с цифровыми сигналами: у него несколько информационных входов, один выход и **адресные** (управляющие) входы, которые выбирают, какой из информационных входов соединить с выходом.

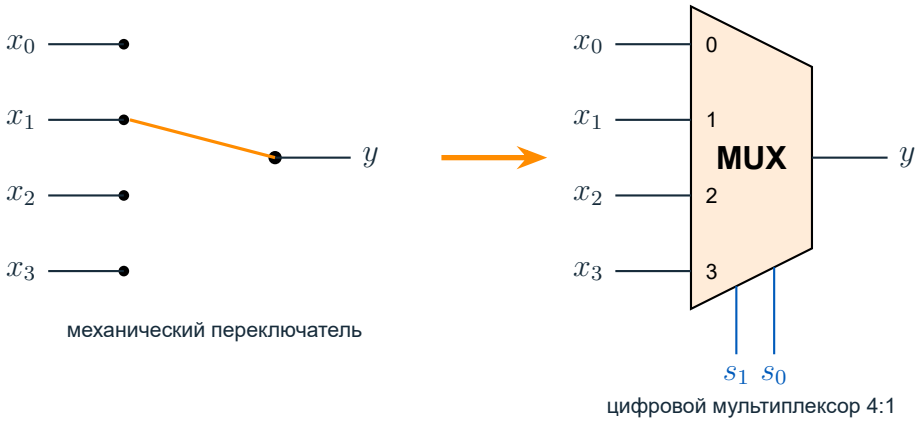


Рис. 5.1.: Мультиплексор — это управляемый переключатель: при $s_1 s_0 = 01$ выход соединён со входом x_1

Мультиплексор с n адресными входами может выбирать из 2^n информационных входов. Его обозначают дробью: **2:1**, **4:1**, **8:1** (сколько входов : сколько выходов).

5.2. Мультиплексор 2:1

Самый простой мультиплексор имеет два входа a, b и один адресный вход s: если $s = 0$, на выход проходит a; если $s = 1$ — проходит b.

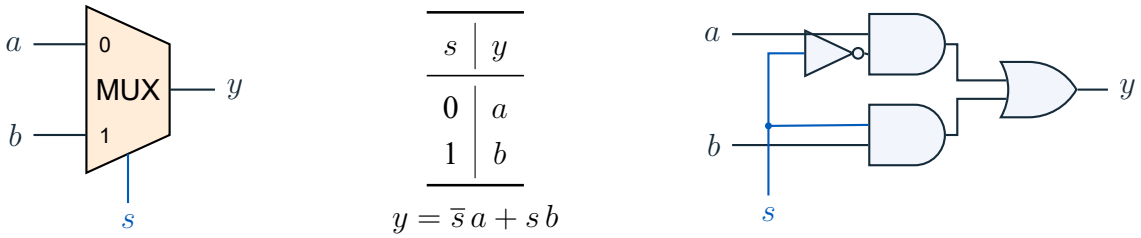


Рис. 5.2.: Мультиплексор 2:1: обозначение, таблица, формула и схема на элементах

Как работает схема: при $s = 0$ верхний элемент И «пропускает» a, а нижний «закрыт» (на его входе 0). При $s = 1$ — наоборот. Элемент ИЛИ собирает результат.

5.3. Мультиплексор 4:1

У мультиплексора 4:1 два адресных входа $s_1 s_0$ задают номер входа от 0 до 3.

$$y = \overline{s_1}\,\overline{s_0}\,x_0 + \overline{s_1}\,s_0\,x_1 + s_1\,\overline{s_0}\,x_2 + s_1\,s_0\,x_3.$$

Присмотритесь к формуле: множители при x_i — это ровно выходы *дешифратора* 2→4 из главы 4! Значит, мультиплексор состоит из дешифратора адреса, четырёх элементов И и одного ИЛИ.

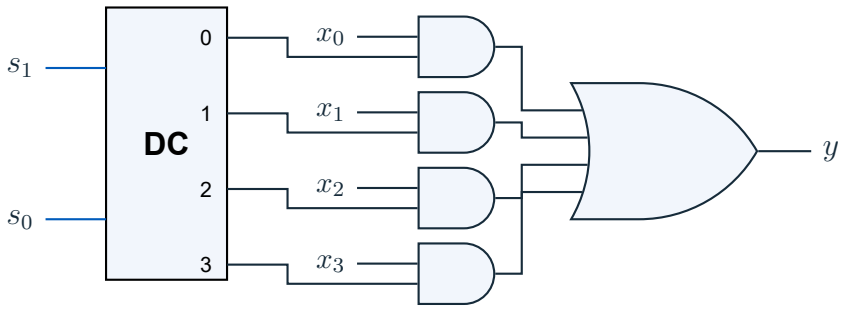


Рис. 5.3.: Мультиплексор 4:1 = дешифратор адреса + элементы И + элемент ИЛИ

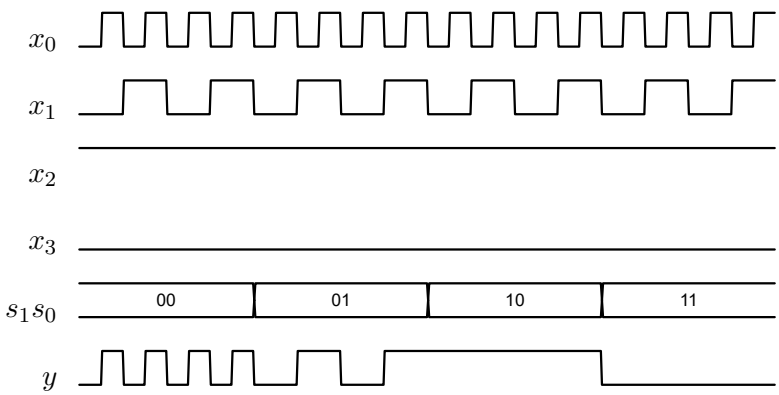


Рис. 5.4.: Мультиплексор 4:1 по очереди передаёт на выход разные входные сигналы

5.4. Дерево мультиплексоров

Большой мультиплексор можно собрать из маленьких. Мультиплексор 4:1 — это три мультиплексора 2:1: первый уровень выбирает по младшему биту адреса, второй — по старшему. Мультиплексор 8:1 — дерево из семи мультиплексоров 2:1 в три уровня, и так далее.

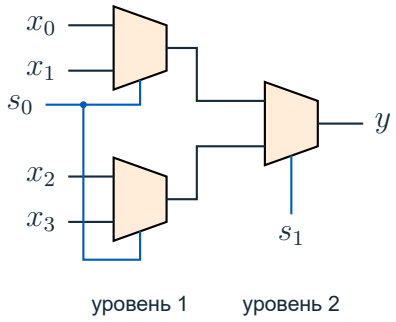
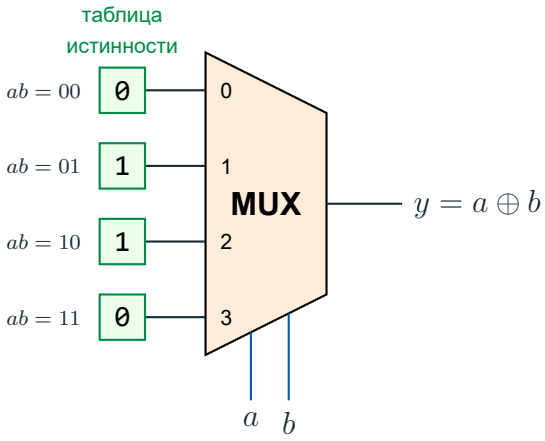


Рис. 5.5.: Мультиплексор 4:1 из трёх мультиплексоров 2:1

5.5. Мультиплексор как универсальный логический элемент

Вот удивительное свойство мультиплексора. Подадим на адресные входы *переменные* a и b, а на информационные входы — *константы* 0 и 1, взятые из правого столбца таблицы истинности какой-либо функции. Тогда мультиплексор будет выбирать нужную строку таблицы и выдавать значение функции! Любую функцию от n переменных можно реализовать одним мультиплексором 2^n:1.



Подставьте `0001` — получится И, `0111` — ИЛИ, `1110` — И-НЕ… Схема та же — меняются только четыре бита слева.

Рис. 5.6.: Мультиплексор 4:1 с константами на входах вычисляет любую функцию двух переменных (здесь — Исключающее ИЛИ)

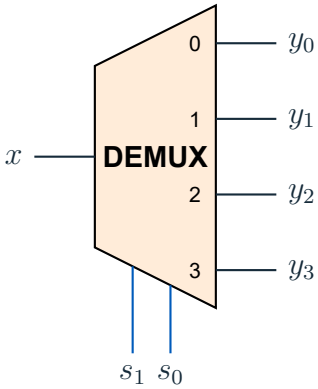
Запомните этот рисунок

Мультиплексор, на входы которого поданы биты из маленькой памяти, — это и есть **LUT**, основной строительный блок ПЛИС. Во второй части книги (глава 11) мы уви-

дим, что ПЛИС состоит из тысяч таких «программируемых мультиплексоров».

5.6. Демультиплексор

Демультиплексор (DEMUX) выполняет обратное действие: у него один информационный вход x и 2^n выходов. Адрес s определяет, на какой из выходов будет передан входной сигнал; на остальных выходах — 0.



s_1	s_0	y_0	y_1	y_2	y_3
0	0	x	0	0	0
0	1	0	x	0	0
1	0	0	0	x	0
1	1	0	0	0	x

$y_0 = x\,\overline{s_1}\,\overline{s_0},\quad y_1 = x\,\overline{s_1}\,s_0,\quad \ldots$

Рис. 5.7.: Демультиплексор 1:4

Сравните формулы с дешифратором со входом разрешения (рис. 4.2): они совпадают, если вход E назвать x. Поэтому **демультиплексор — это дешифратор, у которого на вход разрешения подаётся информационный сигнал**. В справочниках эти две микросхемы часто объединяют под одним названием.

5.7. Передача по одной линии

Вместе мультиплексор и демультиплексор позволяют передать несколько сигналов по *одному* проводу. Адрес быстро перебирает значения $0, 1, 2, 3, 0, 1 \ldots$, и в каждый момент по линии идёт сигнал одного из источников. На приёмной стороне демультиплексор с таким же адресом раскладывает их по своим выходам. Это **временное разделение каналов**: так работают телефонные линии, шины компьютеров, а в дронах — приёмник пульта, передающий по одному проводу сигналы всех каналов (протоколы PPM, SBUS).

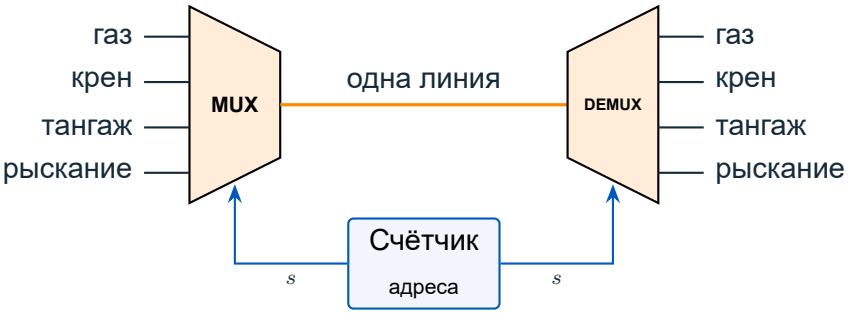
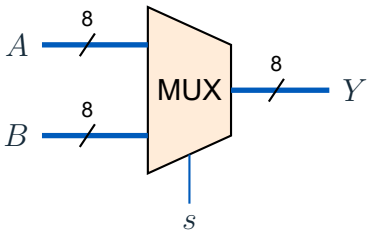


Рис. 5.8.: Четыре канала пульта по одному проводу

5.8. Многоразрядные мультиплексоры

Часто нужно выбирать не между битами, а между *числами*. Для этого несколько мультиплексоров ставят параллельно с общим адресом. Например, 8-разрядный мультиплексор 2:1 — это восемь мультиплексоров 2:1, у которых соединены адресные входы. На схемах такой узел рисуют одним символом с шинами.



$s = 0$: $Y = A$
$s = 1$: $Y = B$

Рис. 5.9.: Мультиплексор 8-разрядных чисел. Косая черта с цифрой показывает число проводов в шине

Главная мысль

Мультиплексор выбирает один из 2^n входов по адресу. Он состоит из дешифратора адреса и элементов И-ИЛИ и может реализовать любую функцию, если подать на входы константы. **Демультиплексор** передаёт один вход на выбранный выход — это дешифратор с информационным входом разрешения.

6 Арифметические узлы

Компьютер — по-английски «вычислитель», и главное, что он умеет, — считать. В этой главе мы увидим, что сложение, вычитание и сравнение чисел выполняются всё теми же логическими элементами.

6.1. Сложение двоичных чисел

Двоичные числа складываются «столбиком», как десятичные, только перенос в следующий разряд возникает, когда сумма достигает двух:

$$0+0=0,\quad 0+1=1,\quad 1+0=1,\quad 1+1=10_2 \text{ (ноль и перенос единицы)}.$$

	1	1	1		переносы
+	0	1	1	1	$0111_2 = 7$
	0	0	1	1	$0011_2 = 3$
	1	0	1	0	$1010_2 = 10$

Рис. 6.1.: Сложение столбиком в двоичной системе

6.2. Полусумматор

Сложение двух одноразрядных чисел a и b даёт двухразрядный результат: бит суммы s и бит переноса c. Таблица истинности:

a	b	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$s = a \oplus b, \qquad c = a \cdot b$$

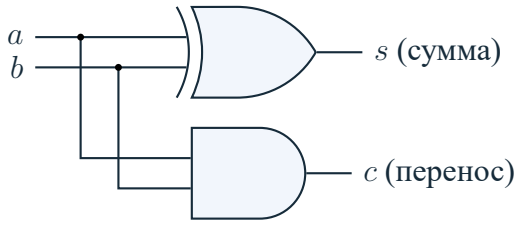


Рис. 6.2.: Полусумматор (half adder)

6.3. Полный сумматор

Полусумматор годится только для младшего разряда. В остальных разрядах складываются *три* бита: a_i, b_i и перенос c_{in} из предыдущего разряда. Такой узел называется **полным сумматором**.

a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$s = a \oplus b \oplus c_{in}$$

$$c_{out} = ab + ac_{in} + bc_{in}$$

Заметьте: c_{out} — это мажоритарная функция «2 из 3» из главы 3! Перенос возникает, когда единиц хотя бы две.

Рис. 6.3.: Полный сумматор (full adder): таблица истинности и формулы

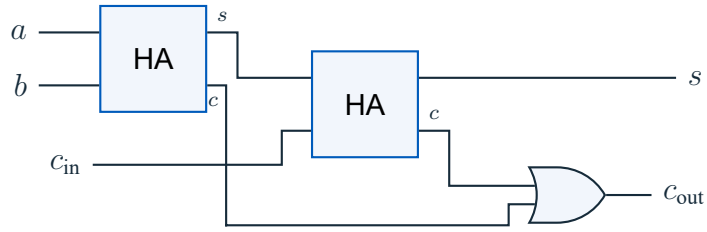


Рис. 6.4.: Полный сумматор из двух полусумматоров и элемента ИЛИ

6.4. Многоразрядный сумматор

Чтобы сложить n-разрядные числа, n полных сумматоров соединяют в цепочку: выход переноса каждого разряда идёт на вход переноса следующего. Такой сумматор называется **сумматором с последовательным переносом** (ripple carry adder).

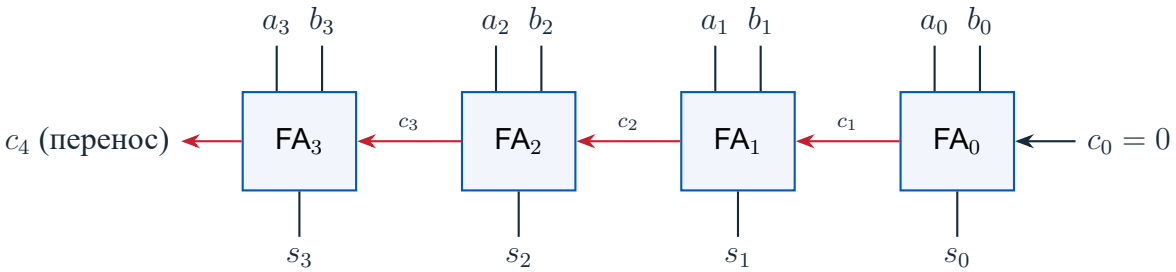


Рис. 6.5.: Четырёхразрядный сумматор с последовательным переносом. Красным — цепочка переноса

Недостаток такого сумматора: старший разряд не может посчитать результат, пока не получит перенос от младшего, а тот — от ещё более младшего. Задержка растёт пропорционально числу разрядов. Поэтому в быстрых процессорах используют сумматоры с **ускоренным переносом**, в которых перенос вычисляется параллельно специальной схемой. А в ПЛИС для этой цели рядом с каждым LUT проложена отдельная быстрая *цепочка переноса*.

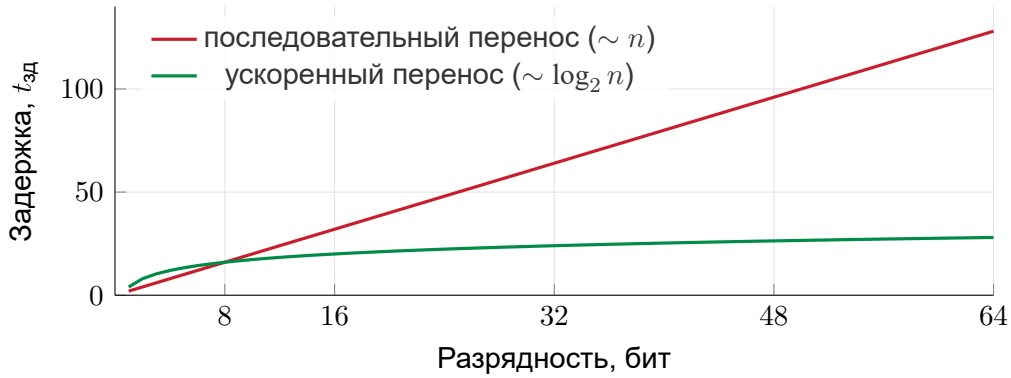


Рис. 6.6.: Задержка сумматора в зависимости от разрядности (оценка)

6.5. Отрицательные числа и вычитание

Как записать отрицательное число, если у нас есть только нули и единицы? Используется **дополнительный код**: старший бит считается имеющим *отрицательный* вес. Для 4 битов веса разрядов равны $-8, 4, 2, 1$.

Таблица 6.1.: Четырёхбитные числа в дополнительном коде

Код	Число	Код	Число
`0000`	0	`1000`	-8
`0001`	1	`1001`	-7
`0010`	2	`1010`	-6
`0011`	3	`1011`	-5
`0100`	4	`1100`	-4
`0101`	5	`1101`	-3
`0110`	6	`1110`	-2
`0111`	7	`1111`	-1

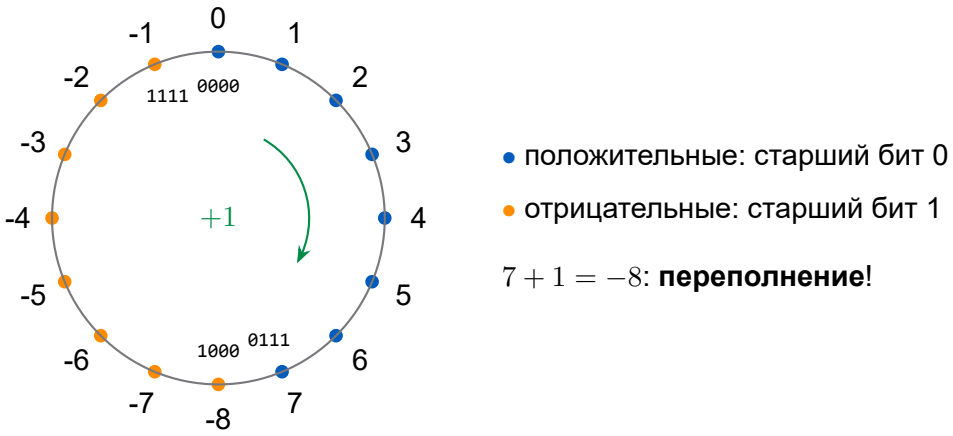


Рис. 6.7.: Числа в дополнительном коде удобно представлять на круге: прибавление единицы — шаг по часовой стрелке

Главное достоинство дополнительного кода: **складывать и вычитать можно одним и тем же сумматором**. Чтобы получить $-b$, нужно инвертировать все биты b и прибавить единицу:

$$a - b = a + \overline{b} + 1.$$

Инверсию сделают элементы «Исключающее ИЛИ» (при управляющем сигнале 1 элемент $x \oplus 1 = \overline{x}$ инвертирует), а единицу — вход переноса младшего разряда.

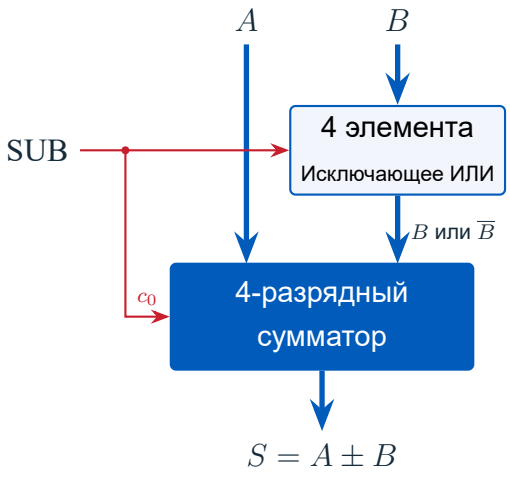


Рис. 6.8.: Сумматор-вычитатель: при SUB $= 0$ считает $A + B$, при SUB $= 1$ — $A - B$

6.6. Компараторы

Компаратор сравнивает два числа. Проще всего проверить *равенство*: числа равны, если равны все их разряды. Равенство двух битов проверяет элемент «Исключающее ИЛИ-НЕ» ($\overline{a \oplus b} = 1$, когда $a = b$), а элемент И объединяет результаты всех разрядов.

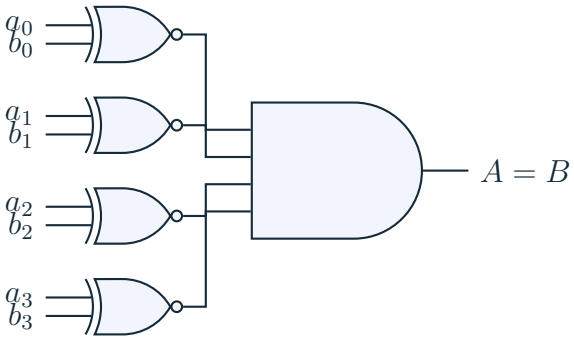


Рис. 6.9.: Компаратор равенства 4-разрядных чисел

Сравнение «больше/меньше» устроено сложнее: числа сравниваются, начиная со старшего разряда. Первый разряд, в котором они различаются, определяет результат. Для одного разряда:

$$(a > b) = a\,\overline{b}, \qquad (a < b) = \overline{a}\,b, \qquad (a = b) = \overline{a \oplus b}.$$

Кстати, $A < B$ можно проверить и вычитателем: если $A - B$ отрицательно (старший бит результата равен 1, при отсутствии переполнения), то $A < B$.

6.7. Умножение и АЛУ

Двоичное умножение ещё проще десятичного: каждая цифра множителя — это 0 или 1, поэтому частичные произведения — это либо ноль, либо сдвинутый множимый. Их остаётся сложить.

			1	0	1	1		$1011_2 = 11$
		×	0	1	1	0		$0110_2 = 6$
			0	0	0	0		
		1	0	1	1			сдвинутые копии
	1	0	1	1				
0	0	0	0					
1	0	0	0	0	1	0		$1000010_2 = 66$

Рис. 6.10.: Умножение столбиком: сдвиги и сложения

Умножитель $n \times n$ бит требует примерно n^2 полных сумматоров, поэтому он получается большим. В ПЛИС для этого есть готовые аппаратные умножители (блоки DSP).

Сумматор, вычитатель, логические операции и сдвиги, собранные вместе и выбираемые мультиплексором по коду операции, образуют **арифметико-логическое устройство** (АЛУ) — сердце любого процессора.

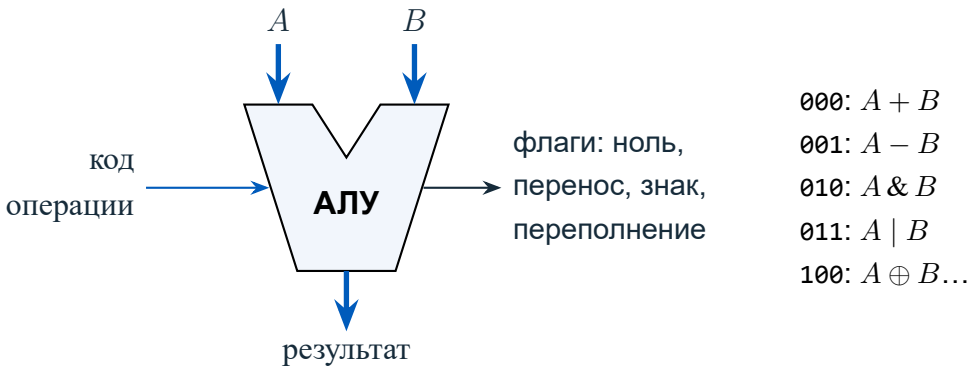


Рис. 6.11.: Арифметико-логическое устройство

Главная мысль

Сложение выполняется полными сумматорами, соединёнными цепочкой переноса. Отрицательные числа записываются в дополнительном коде, и тогда вычитание сводится к сложению. Компаратор равенства строится из элементов «Исключающее ИЛИ-НЕ» и И.

7 Триггеры

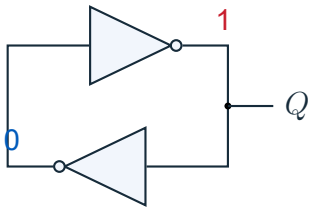
7.1. Зачем схеме память

Представьте кнопку «включить/выключить» на пульте: нажали — свет загорелся, отпустили — продолжает гореть, нажали ещё раз — погас. В момент, когда кнопка отпущена, входы схемы одинаковы, а выход может быть и 0, и 1 — в зависимости от того, *что было раньше*. Комбинационная схема так не умеет. Нужна **память**.

Простейший элемент памяти хранит один бит и называется **триггером** (от англ. *trigger* — «спусковой крючок»; в английской литературе — latch и flip-flop).

7.2. Бистабильная ячейка

Соединим два инвертора в кольцо. Если на выходе первого 1, то на выходе второго 0, и этот 0 поддерживает 1 на выходе первого. Состояние *само себя поддерживает*. Возможно и противоположное устойчивое состояние. У схемы два устойчивых состояния — она **бистабильна** и может хранить один бит.



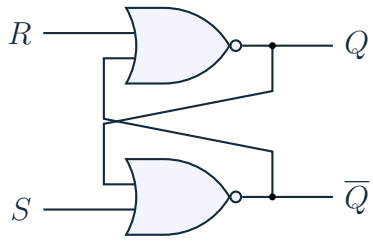
Состояние поддерживает само себя,
но изменить его пока нечем.

Рис. 7.1.: Два инвертора в кольце — бистабильная ячейка

7.3. RS-триггер

Чтобы записывать в ячейку новое значение, заменим инверторы на элементы ИЛИ-НЕ и используем их вторые входы. Получится **RS-триггер** (асинхронная RS-защёлка):

- вход S (Set — «установить») записывает 1;
- вход R (Reset — «сбросить») записывает 0;
- если $S = R = 0$, триггер **хранит** прежнее значение.



S	R	Q	**Режим**
0	0	Q	хранение
1	0	1	установка
0	1	0	сброс
1	1	?	запрещено

Рис. 7.2.: RS-триггер на двух элементах ИЛИ-НЕ и его таблица переходов

Разберём, почему он работает. Помним: выход ИЛИ-НЕ равен 0, если хотя бы один вход равен 1.

- $S = 1$: нижний элемент выдаёт $\overline{Q} = 0$. У верхнего оба входа равны нулю ($R = 0$, $\overline{Q} = 0$), поэтому $Q = 1$.
- Теперь S вернули в 0. На нижнем элементе $Q = 1$, поэтому $\overline{Q}$ по-прежнему 0, а Q по-прежнему 1: триггер *запомнил* единицу.
- $S = R = 1$ — оба выхода становятся нулями (хотя должны быть противоположны), а при одновременном снятии сигналов неизвестно, в какое состояние триггер «упадёт». Поэтому это сочетание запрещено.

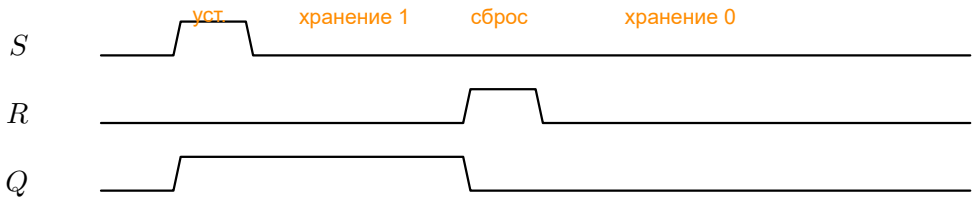


Рис. 7.3.: Работа RS-триггера: короткий импульс на S или R переключает триггер, а без импульсов он хранит значение

RS-триггер на элементах И-НЕ

Точно так же триггер собирают из двух элементов И-НЕ. Но тогда входы срабатывают от *нуля*: $\overline{S} = 0$ устанавливает, $\overline{R} = 0$ сбрасывает, а запрещено сочетание $\overline{S} = \overline{R} = 0$. Такой триггер — классическое средство подавления дребезга механического переключателя: при первом же касании контакта он переключается и дальнейших «прыжков» не замечает.

7.4. D-защёлка

У RS-триггера два неудобства: есть запрещённое сочетание входов, и он реагирует на входы в любой момент. Добавим перед ним два элемента И, вход данных D и вход разрешения C (clock, «синхронизация»). Получится **D-защёлка** (D latch):

- при $C = 1$ защёлка **прозрачна**: выход повторяет вход, $Q = D$;
- при $C = 0$ защёлка **хранит** значение, которое было на D в момент перехода C из 1 в 0.

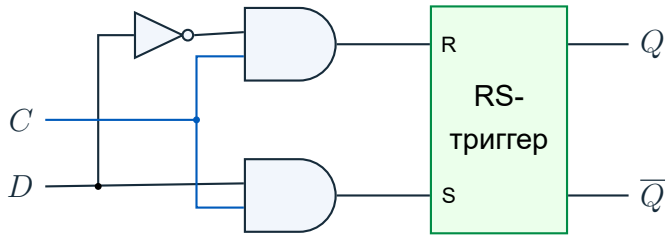


Рис. 7.4.: D-защёлка: запрещённое сочетание $S = R = 1$ теперь невозможно, так как на S и R подаются D и $\overline{D}$

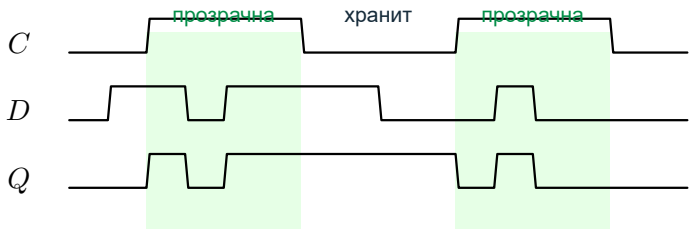
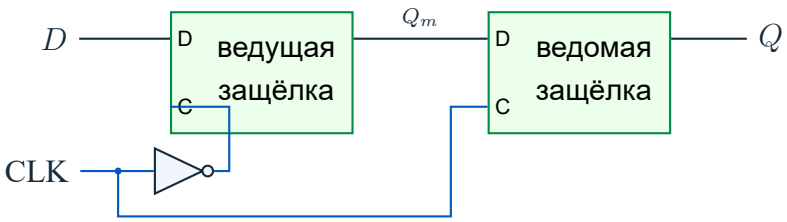


Рис. 7.5.: D-защёлка при $C = 1$ передаёт все изменения D на выход, при $C = 0$ «замораживает» выход

7.5. D-триггер

Прозрачность защёлки — это и достоинство, и недостаток. Если выход защёлки через комбинационную логику идёт обратно на её же вход (как в счётчике), то пока $C = 1$, сигнал будет бегать по кругу, и результат станет непредсказуемым. Нужен элемент, который запоминает значение в *один момент времени* — по *фронту* тактового сигнала. Это **D-триггер** (D flip-flop), главный элемент памяти всей современной цифровой техники и ПЛИС.

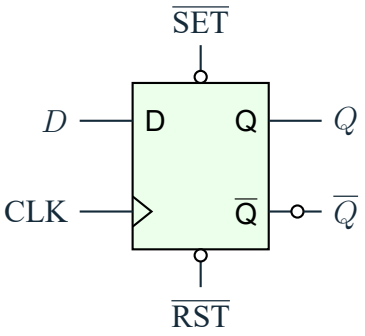
Его можно собрать из двух D-защёлок, включённых последовательно, с противоположными сигналами разрешения. Такая схема называется **«ведущий – ведомый»** (master–slave):



CLK $= 0$: ведущая прозрачна, ведомая хранит;
CLK $= 1$: ведущая хранит, ведомая прозрачна;
$\Rightarrow Q$ меняется только в момент перехода CLK из 0 в 1.

Рис. 7.6.: D-триггер, построенный по схеме «ведущий – ведомый»

На схемах D-триггер рисуют прямоугольником, где вход тактового сигнала отмечен треугольником (знак «срабатывает по фронту»):



треугольник — срабатывает по фронту

кружок — активный уровень 0

SET, RST — асинхронные установка и сброс: срабатывают сразу, не дожидаясь фронта

Рис. 7.7.: Условное обозначение D-триггера

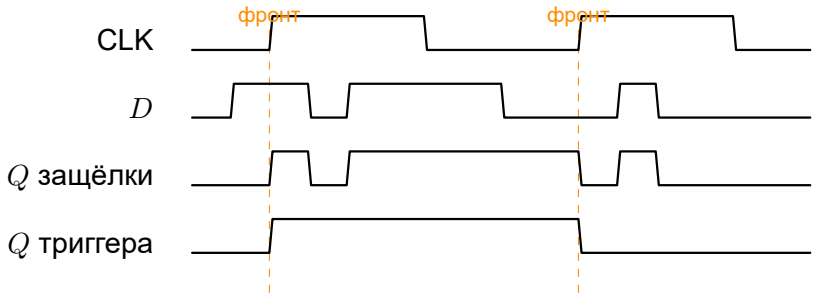


Рис. 7.8.: Сравнение: защёлка (рис. 7.5) пропускает все изменения D, пока CLK $= 1$, а триггер берёт значение D *только в момент фронта*

7.6. Временные параметры триггера

Чтобы триггер надёжно запомнил значение, вход D должен быть неизменным в течение некоторого времени до фронта и после него:

- t_{su} (setup, **время предустановки**) — сколько D должен быть стабилен *до* фронта;
- t_h (hold, **время удержания**) — сколько *после* фронта;
- t_{co} (clock-to-output) — через сколько после фронта меняется Q.

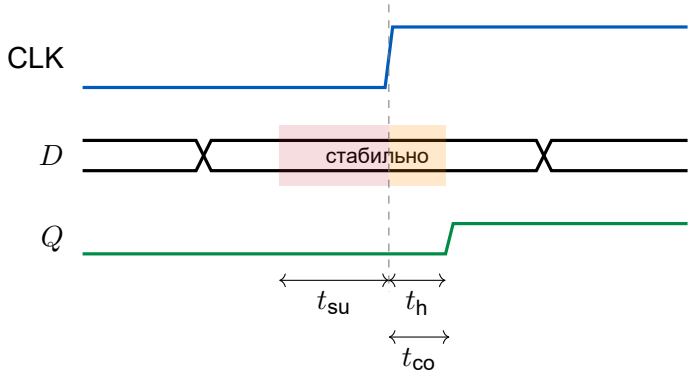


Рис. 7.9.: Время предустановки, удержания и задержка выхода триггера

Метастабильность

Если D меняется внутри окна t_{su}–t_h, триггер может на некоторое время «зависнуть» в промежуточном состоянии — ни 0, ни 1 — и лишь потом случайно упасть в одно из них. Это **метастабильность**. Она неизбежно возникает, когда на триггер приходит сигнал, не связанный с тактом (например, от кнопки). Защита — **синхронизатор** из двух триггеров подряд: к моменту следующего фронта первый триггер почти наверняка успокоится.

7.7. Вход разрешения

Часто нужно, чтобы триггер обновлялся не на каждом фронте, а только по команде. Для этого добавляют вход разрешения CE (clock enable). Устроен он просто: перед входом D ставится мультиплексор, который при $\mathrm{CE} = 0$ подаёт на вход триггера его же выход — триггер переписывает сам себя и не меняется.

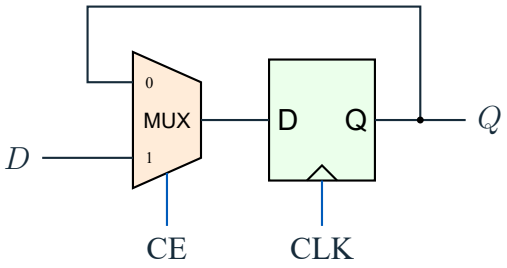
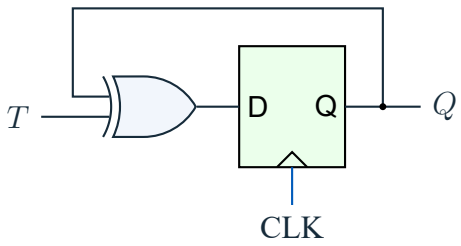


Рис. 7.10.: D-триггер с входом разрешения: мультиплексор + обратная связь

7.8. T-триггер и JK-триггер

T-триггер (toggle, «переключатель») меняет своё состояние на противоположное по каждому фронту, если на входе $T = 1$, и хранит его при $T = 0$. Его легко сделать из D-триггера и элемента «Исключающее ИЛИ»: $D = T \oplus Q$.



T	Q после фронта	
0	Q	хранение
1	$\overline{Q}$	переключение

Рис. 7.11.: T-триггер из D-триггера и элемента «Исключающее ИЛИ»

Если на T-триггер постоянно подавать $T = 1$, он переключается на каждом фронте, и частота на его выходе будет *вдвое меньше* тактовой. Это простейший **делитель частоты на 2**.

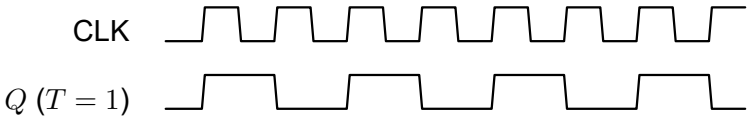


Рис. 7.12.: T-триггер с $T = 1$ делит частоту на 2

JK-триггер объединяет возможности RS- и T-триггеров: J — установка, K — сброс, $J = K = 1$ — переключение (запрещённого сочетания нет). В старых схемах на микросхемах серии 155 JK-триггеры были очень популярны. В ПЛИС же используются только D-триггеры, а остальные виды при необходимости получаются из них добавлением логики на входе.

Таблица 7.1.: Сводка: виды триггеров

Тип	Когда срабатывает	Что делает
RS-защёлка	сразу	S — установить 1, R — сбросить в 0, $S = R = 1$ запрещено
D-защёлка	пока $C = 1$	повторяет D, при $C = 0$ хранит
D-триггер	**по фронту** CLK	запоминает D в момент фронта
T-триггер	по фронту CLK	при $T = 1$ меняет состояние на противоположное
JK-триггер	по фронту CLK	J — установка, K — сброс, $J = K = 1$ — переключение

Главная мысль

Триггер хранит один бит. Главный элемент памяти в цифровой технике — **D-триггер**: он запоминает значение входа D в момент *переднего фронта* тактового сигнала и хранит его до следующего фронта. Чтобы триггер работал надёжно, вход не должен меняться в окне t_{su}–t_h вокруг фронта.

8 Регистры и счётчики

Один триггер хранит один бит. Соединив несколько триггеров, получают узлы, которые хранят числа, сдвигают их и считают.

8.1. Регистр хранения

Регистр — это группа D-триггеров с общим тактовым сигналом. По фронту все триггеры одновременно запоминают свои входы, и регистр хранит n-разрядное число до следующего фронта. На схемах регистр обозначают буквами RG.

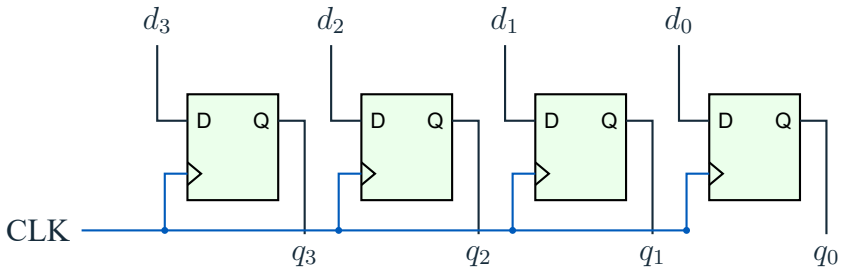


Рис. 8.1.: Четырёхразрядный регистр: четыре D-триггера с общим тактом

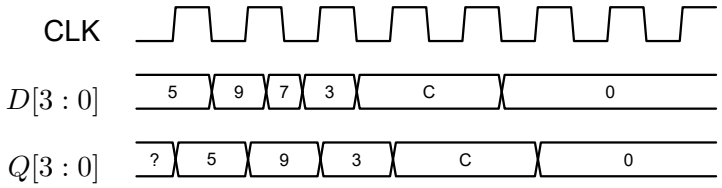


Рис. 8.2.: Регистр «фотографирует» значение шины D в моменты передних фронтов CLK. Значение 7 продержалось на входе меньше такта и пришлось между фронтами — регистр его просто не заметил

Регистры — это «переменные» аппаратуры. В процессоре в регистрах хранятся операнды и результаты вычислений; в любом устройстве на ПЛИС — текущее состояние, счётчики, данные между этапами обработки. Чтобы регистр менял значение не на каждом фронте, а только по команде, используют триггеры с входом разрешения CE (рис. 7.10).

8.2. Сдвиговый регистр

Соединим триггеры цепочкой: выход каждого подаётся на вход следующего. По каждому фронту содержимое **сдвигается** на один разряд, а в первый триггер записывается новый бит с входа D_{in}.

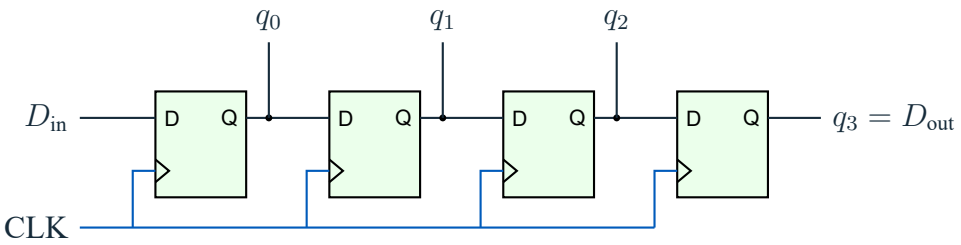


Рис. 8.3.: Четырёхразрядный сдвиговый регистр

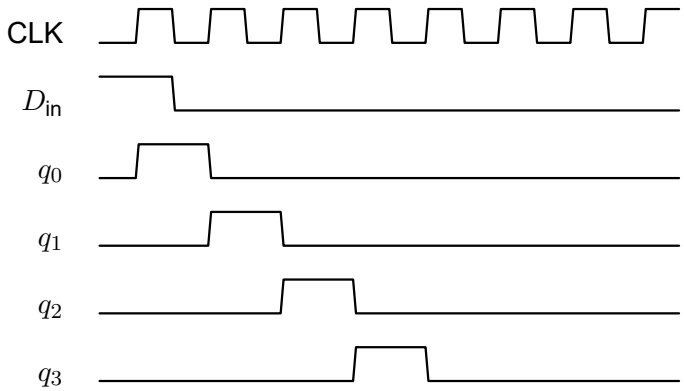


Рис. 8.4.: Единица, поданная на вход, продвигается по регистру на один разряд за такт

8.2.1. Зачем сдвигать

Последовательный → параллельный код. Биты приходят по одному проводу друг за другом (так работают интерфейсы UART, SPI, I²C). После n тактов все n битов оказываются в регистре одновременно и могут быть прочитаны как число.

Параллельный → последовательный код. Обратная задача при передаче: число загружается в регистр целиком, а затем «выталкивается» по одному биту за такт. Для загрузки перед каждым триггером ставят мультиплексор «сдвиг/загрузка».

Задержка сигнала на заданное число тактов.

Умножение и деление на 2 : сдвиг двоичного числа влево на один разряд умножает его на 2, вправо — делит.

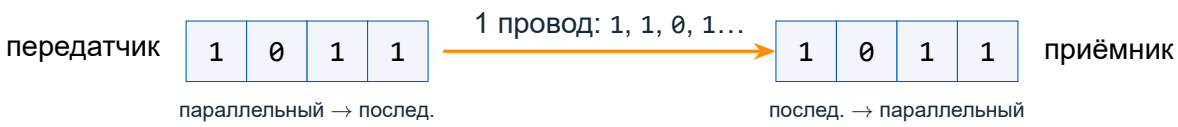
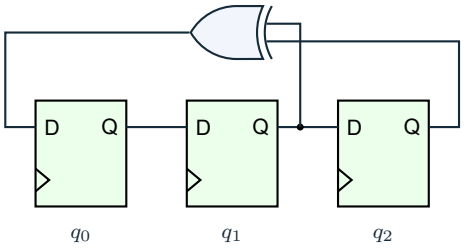


Рис. 8.5.: Передача числа по одному проводу с помощью двух сдвиговых регистров

8.2.2. Кольцевой регистр и генератор псевдослучайных чисел

Если выход последнего триггера соединить со входом первого, получится **кольцевой регистр**: записанная в него единица будет бесконечно бегать по кругу (эффект «бегущего огня» в главе 15).

А если на вход подать «Исключающее ИЛИ» от нескольких разрядов, получится **регистр сдвига с линейной обратной связью** (LFSR). При правильном выборе отводов он перебирает все $2^n - 1$ ненулевых состояний в «перемешанном» порядке. Такие регистры используются как простые генераторы псевдослучайных чисел, в помехоустойчивом кодировании и для шифрования.



такт	q_0	q_1	q_2
0	1	0	0
1	0	1	0
2	1	0	1
3	1	1	0
4	1	1	1
5	0	1	1
6	0	0	1
7	1	0	0

Рис. 8.6.: Трёхразрядный LFSR ($q_0 \leftarrow q_1 \oplus q_2$) перебирает все семь ненулевых состояний и повторяется

8.3. Счётчики

Счётчик — регистр, значение которого по каждому такту увеличивается (или уменьшается) на единицу. Счётчики измеряют время, делят частоту, перебирают адреса памяти.

8.3.1. Асинхронный счётчик

Посмотрите, как меняются разряды при счёте $0, 1, 2, \ldots, 7$: младший бит переключается каждый шаг, следующий — в два раза реже, следующий — ещё в два раза реже. Каждый разряд переключается, когда предыдущий переходит из 1 в 0. Значит, можно соединить T-триггеры цепочкой: тактовый вход каждого следующего триггера подключить к *инверсному* выходу предыдущего.

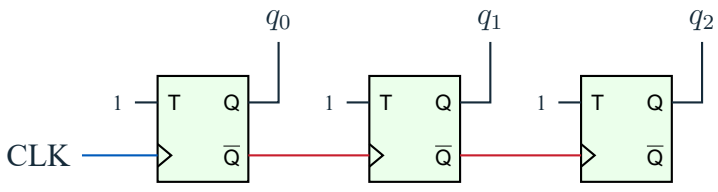


Рис. 8.7.: Асинхронный (последовательный) трёхразрядный счётчик на T-триггерах

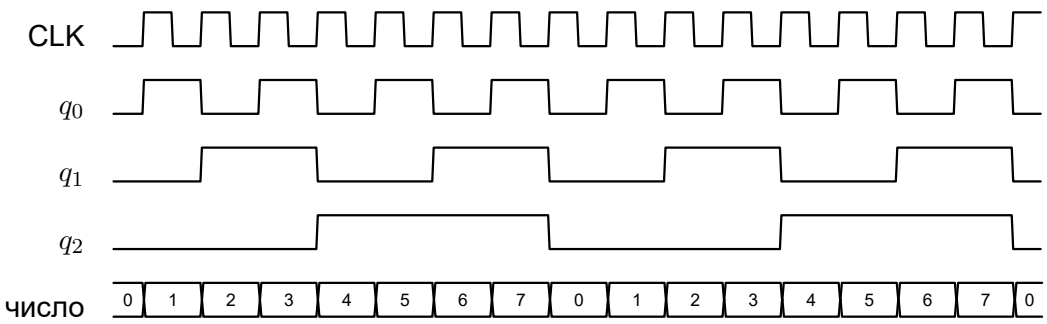


Рис. 8.8.: Каждый следующий разряд меняется вдвое реже: счётчик перебирает числа $0 \ldots 7$, а разряд q_k имеет частоту $f_{\mathrm{CLK}}/2^{k+1}$

Недостаток асинхронного счётчика — триггеры переключаются не одновременно, а «волной», каждый с задержкой относительно предыдущего. В момент перехода $011 \to 100$ на выходе на короткое время появляются ложные значения ($011 \to 010 \to 000 \to 100$). В длинном счётчике эта «волна» идёт долго.

8.3.2. Синхронный счётчик

В синхронном счётчике все триггеры тактируются *одним* сигналом, а новое значение вычисляет комбинационная схема — сумматор, прибавляющий единицу. Все разряды меняются одновременно по фронту. Именно так строятся все счётчики в ПЛИС.

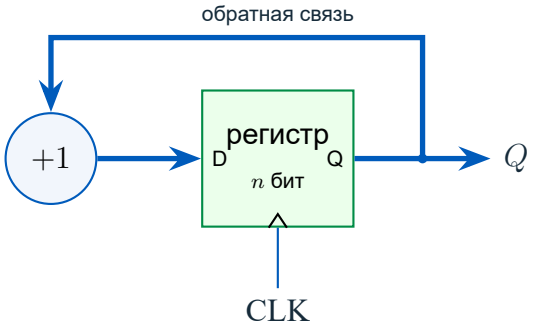


Рис. 8.9.: Синхронный счётчик: регистр + сумматор. Каждый такт регистр записывает $Q + 1$

8.3.3. Делитель частоты и счётчик по модулю

Как видно на рис. 8.8, каждый разряд счётчика — это делитель частоты на степень двойки. А как поделить на число, не являющееся степенью двойки, например на 10? Нужен **счётчик по модулю** N: он считает $0, 1, \ldots, N-1$ и затем сбрасывается в 0. Для этого к схеме добавляют компаратор, который сравнивает значение с $N-1$, и мультиплексор, который вместо $Q+1$ подаёт на регистр 0.

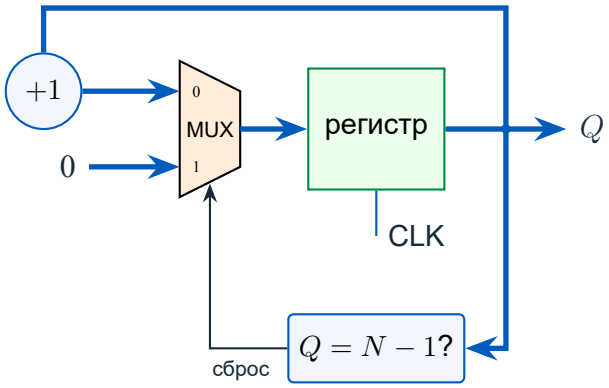


Рис. 8.10.: Счётчик по модулю N

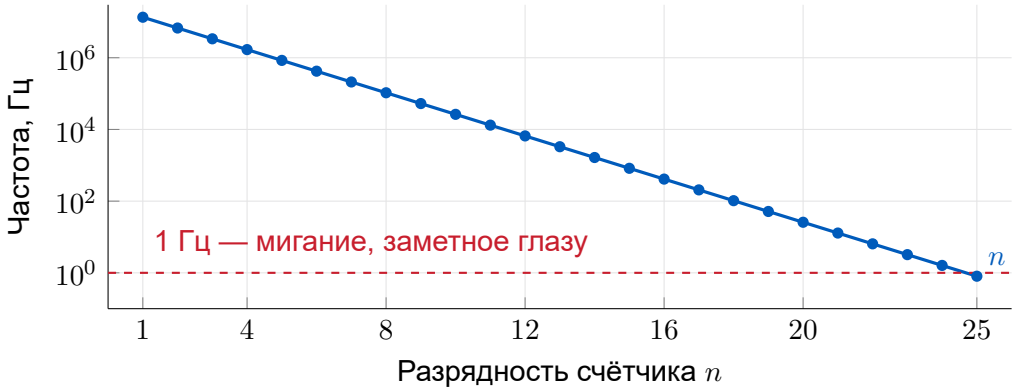


Рис. 8.11.: Частота на выходе старшего разряда n-битного счётчика при такте 27 МГц: $f = 27\,\text{МГц}/2^n$. Чтобы получить мигание около 1 Гц, нужен счётчик примерно на 25 бит

Главная мысль

Регистр — группа триггеров с общим тактом, хранит число. **Сдвиговый регистр** перемещает биты по цепочке и преобразует последовательный код в параллельный и обратно. **Синхронный счётчик** — регистр с сумматором в обратной связи; его разряды делят частоту такта на степени двойки.

9 Синхронные схемы, автоматы и память

9.1. Синхронное проектирование

Мы уже знаем два «материала»: комбинационную логику (она вычисляет) и триггеры (они хранят). Как собрать из них большое надёжное устройство? Почти все современные цифровые схемы строятся по одному принципу — **синхронному**:

- все триггеры тактируются **одним** тактовым сигналом;
- между регистрами находится только комбинационная логика;
- результат вычислений записывается в регистры по фронту, когда все переходные процессы и «иголки» уже закончились.

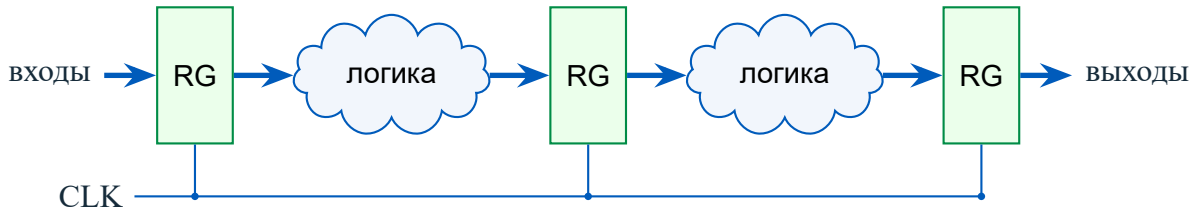


Рис. 9.1.: Синхронная схема: регистры, разделённые комбинационной логикой, и один общий такт

9.1.1. Максимальная тактовая частота

За один такт сигнал должен пройти от выхода одного регистра через логику до входа следующего и успеть «установиться» до фронта. Отсюда главное уравнение синхронной схемы:

$$T_{\text{CLK}} \;\geq\; t_{\text{co}} + t_{\text{логики}} + t_{\text{su}}, \qquad f_{\max} = \frac{1}{t_{\text{co}} + t_{\text{логики}} + t_{\text{su}}}.$$



Рис. 9.2.: Бюджет времени одного такта. Если сумма задержек больше периода, в регистр запишется неверное значение

Пример: $t_{co} = 0{,}5$ нс, $t_{su} = 0{,}5$ нс, а сигнал проходит через 10 элементов по 0,5 нс. Тогда $T \geq 0{,}5 + 5 + 0{,}5 = 6$ нс и $f_{max} \approx 166$ МГц. Чем длиннее цепочка логики между регистрами, тем ниже допустимая частота.

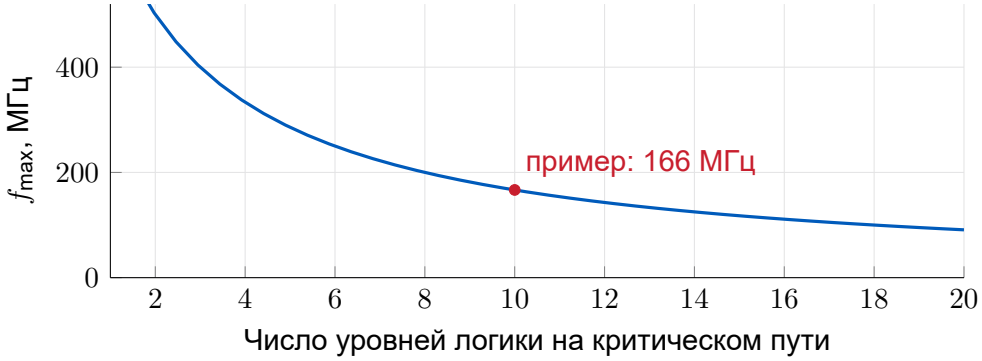


Рис. 9.3.: Максимальная частота падает с ростом числа элементов между регистрами ($t_{co} + t_{su} = 1$ нс, задержка элемента 0,5 нс)

9.1.2. Конвейер

Что делать, если логика слишком длинная? Её можно разрезать на части и поставить между ними дополнительные регистры. Это **конвейер** (pipeline): каждая «станция» делает свою часть работы за один короткий такт, и хотя результат для одних данных появляется через несколько тактов, новые данные можно подавать *каждый* такт. Так работает сборочный конвейер на заводе — и все современные процессоры.

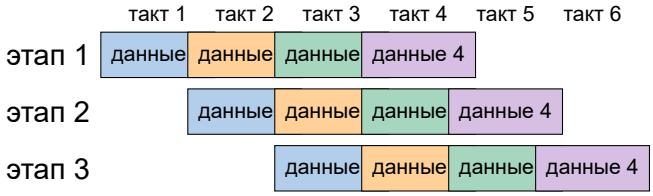


Рис. 9.4.: Трёхэтапный конвейер: на такте 3 одновременно обрабатываются три порции данных, каждая на своём этапе

9.2. Конечные автоматы

Многие устройства работают по сценарию: «ждать нажатия, потом сделать то, потом это, при ошибке вернуться в начало». Такое поведение описывают **конечным автоматом** (FSM, finite state machine). Автомат находится в одном из конечного числа **состояний**; по каждому такту, в зависимости от текущего состояния и входов, он **переходит** в следующее состояние.

Схема любого автомата одинакова: регистр хранит номер текущего состояния, комбинационная логика вычисляет следующее состояние и выходы.

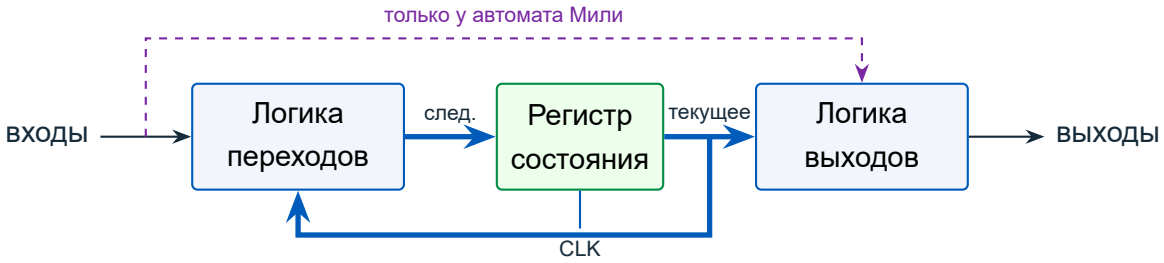


Рис. 9.5.: Структура конечного автомата. У автомата **Мура** выходы зависят только от состояния, у автомата **Мили** — ещё и от входов (пунктир)

9.2.1. Пример: детектор последовательности

Построим автомат, который следит за входом x (по одному биту за такт) и выдаёт $y = 1$, когда последние три принятых бита равны `101`. Такие детекторы ищут стартовые метки в потоке данных.

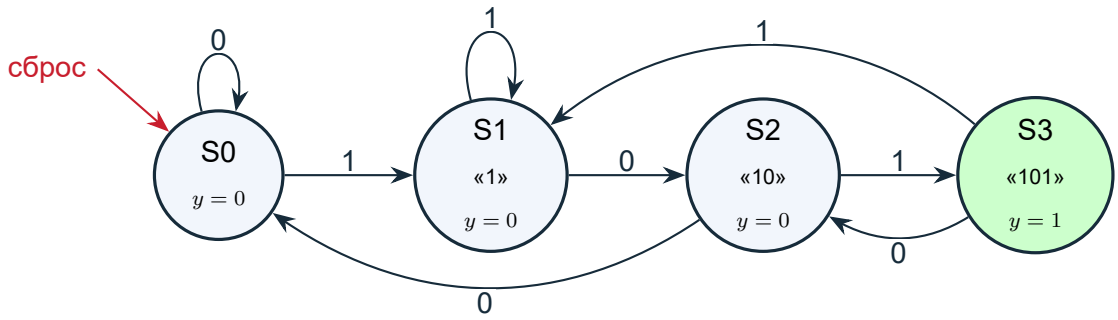


Рис. 9.6.: Граф переходов детектора `101` (автомат Мура). На стрелках — значение входа x

Таблица 9.1.: Таблица переходов детектора

Текущее состояние	**Следующее состояние**		**Выход** y
	$x = 0$	$x = 1$	
S0 (ничего не найдено)	S0	S1	0
S1 (принято «1»)	S2	S1	0
S2 (принято «10»)	S0	S3	0
S3 (принято «101»)	S2	S1	1

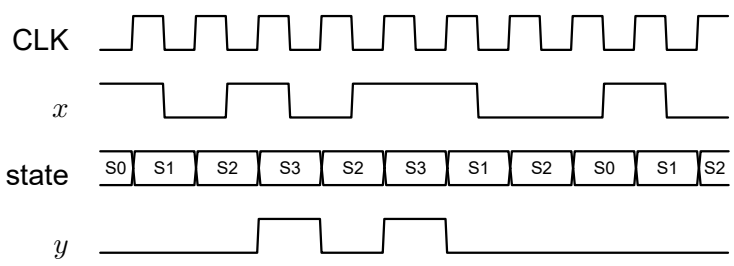


Рис. 9.7.: Работа детектора: вход `1010110010`. Выход равен 1 после каждой найденной тройки `101`, в том числе перекрывающейся

Проверим по таблице: S0 –(1)→ S1 –(0)→ S2 –(1)→ S3 ($y = 1$) –(0)→ S2 –(1)→ S3 ($y = 1$) –(1)→ S1 –(0)→ S2 –(0)→ S0 –(1)→ S1.

9.2.2. Кодирование состояний

Состояния хранятся в регистре в виде двоичных кодов. Четыре состояния можно закодировать двумя битами (`00`, `01`, `10`, `11`) — это **двоичное кодирование**. Можно взять четыре триггера и держать единицу только в одном из них (`0001`, `0010`, …) — это **унитарное кодирование** (one-hot). Оно требует больше триггеров, зато логика переходов проще и быстрее. В ПЛИС, где триггеров много, синтезатор часто выбирает именно его.

9.3. Память

Регистр из n триггеров хранит одно число. Чтобы хранить много чисел, используют **память**: массив ячеек, каждая из которых имеет свой номер — **адрес**. Память характеризуется **глубиной** (числом ячеек) и **шириной** (числом битов в ячейке), например 1024×8 — 1024 ячейки по 8 бит.

9.3.1. ПЗУ: память только для чтения

ПЗУ (постоянное запоминающее устройство, ROM) хранит данные, записанные заранее. По сути это комбинационная схема: адрес — вход, данные — выход, а содержимое — большая таблица истинности. ПЗУ можно построить из дешифратора адреса и элементов ИЛИ: каждая ячейка — это выход дешифратора, подключённый к тем элементам ИЛИ, где в данных стоит 1.

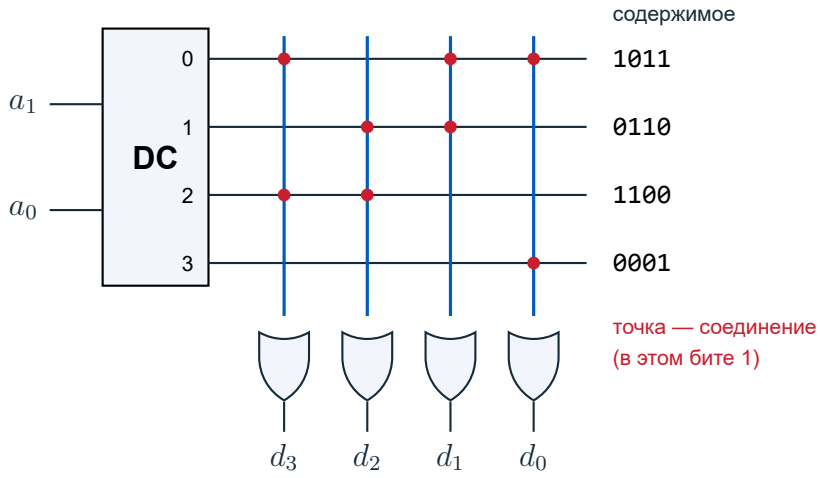


Рис. 9.8.: ПЗУ 4×4: дешифратор выбирает строку, элементы ИЛИ собирают биты. Тот же принцип — «память как таблица истинности» — лежит в основе LUT

9.3.2. ОЗУ: оперативная память

ОЗУ (оперативное запоминающее устройство, RAM) позволяет и читать, и записывать. Концептуально это набор регистров, дешифратор адреса, который при записи выбирает, в какой регистр записать данные, и мультиплексор, который при чтении выбирает, какой регистр подать на выход.

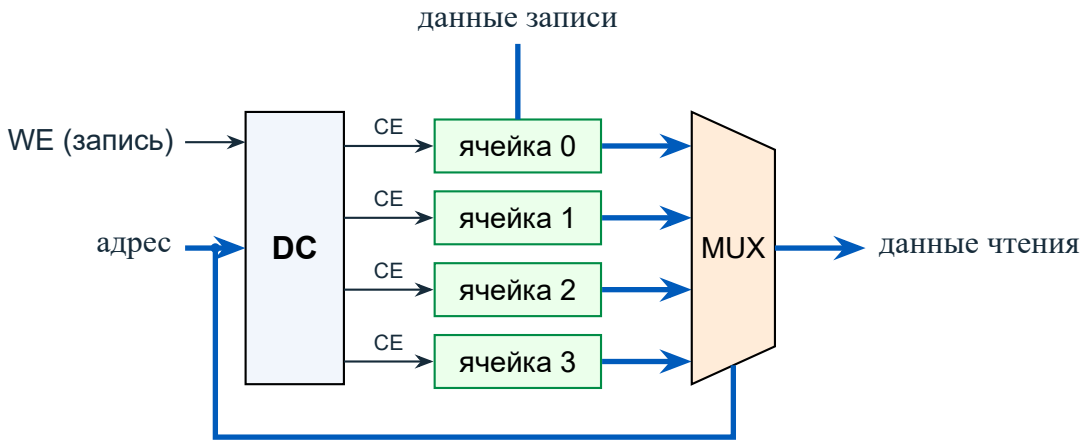


Рис. 9.9.: Устройство ОЗУ «на пальцах»: дешифратор адреса при записи, мультиплексор при чтении

На практике ячейки памяти делают не из D-триггеров, а из более компактных ячеек:

- **статическая память SRAM** — 6 транзисторов на бит, быстрая, не требует обслуживания; из неё сделаны блоки памяти внутри ПЛИС;
- **динамическая память DRAM** — 1 транзистор и 1 конденсатор на бит, очень плотная, но требует периодической регенерации; такая память — SDRAM — встроена в корпус ПЛИС на Tang Nano 20K (глава 17).

9.4. Что дальше

Мы прошли путь от сигнала до памяти и конечных автоматов. Этого достаточно, чтобы понять, как устроено практически любое цифровое устройство: процессор, контроллер дрона, видеокарта. Все эти узлы можно собрать и внутри ПЛИС — причём не паяльником, а описав их текстом.

Таблица 9.2.: Узлы из первой части и то, как они реализуются в ПЛИС

Узел	Ресурс ПЛИС (часть II)
Логические элементы, дешифраторы, шифраторы, мультиплексоры	таблицы LUT (рис. 5.6)
Сумматоры, счётчики	LUT + быстрые цепочки переноса (ALU)
Умножители	аппаратные блоки DSP
Триггеры, регистры, сдвиговые регистры	триггеры в каждой логической ячейке
Конечные автоматы	LUT + триггеры
ПЗУ и ОЗУ	блочная память BSRAM, распределённая память SSRAM
Большая память	встроенная SDRAM 64 Мбит
Тактовый сигнал	генератор 27 МГц, PLL, глобальная тактовая сеть

Главная мысль

Синхронная схема — это регистры и комбинационная логика между ними с одним общим тактом. Её быстродействие определяется критическим путём. Конечный автомат — регистр состояния плюс логика переходов. Память — массив ячеек с дешифратором адреса. Во второй части мы соберём всё это в ПЛИС.

Часть II.

ПЛИС на практике: Tang Nano 20K

10 Что такое ПЛИС

10.1. Схема, которую можно перепаять без паяльника

Представьте, что вам нужно собрать устройство, которое включает лампу, когда нажаты *обе* кнопки. Вы берёте микросхему с элементами И, паяете провода, и всё работает. А завтра заказчик просит: «Пусть лампа горит, когда нажата *ровно одна* кнопка». Придётся выпаивать элемент И, ставить элемент «Исключающее ИЛИ» и перекладывать провода.

А теперь представьте микросхему, внутри которой лежат тысячи «заготовок» логических элементов и триггеров и огромная сеть проводов с управляемыми переключателями на каждом перекрёстке. Какие заготовки чем станут и какие переключатели будут замкнуты, задаётся специальным файлом конфигурации, который загружается в микросхему по кабелю за доли секунды. Это и есть **ПЛИС**.

Главная мысль

ПЛИС (программируемая логическая интегральная схема, англ. **FPGA**, Field-Programmable Gate Array: «вентильная матрица, программируемая в полевых условиях») — это микросхема, внутреннюю цифровую схему которой можно многократно изменять после её производства.

Важно сразу понять разницу: ПЛИС *не выполняет программу*, как процессор. Она *становится схемой*. После загрузки конфигурации внутри кристалла существуют настоящие логические элементы и настоящие провода, по которым сигналы бегут одновременно, как в схеме на макетной плате.

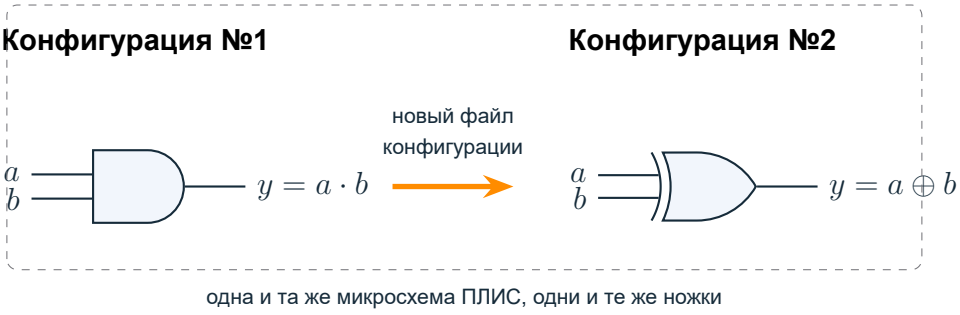


Рис. 10.1.: Одна и та же ПЛИС после загрузки разных конфигураций

10.2. Немного истории

Идея программируемой логики появилась в 1970-х. Сначала это были небольшие матрицы И–ИЛИ (PAL, PLA), в которых пережигались перемычки. Затем появились CPLD (сложные программируемые логические устройства), а в 1985 году компания Xilinx выпустила первую FPGA XC2064, в которой было всего 64 логических блока. Современные ПЛИС содержат миллионы логических ячеек.

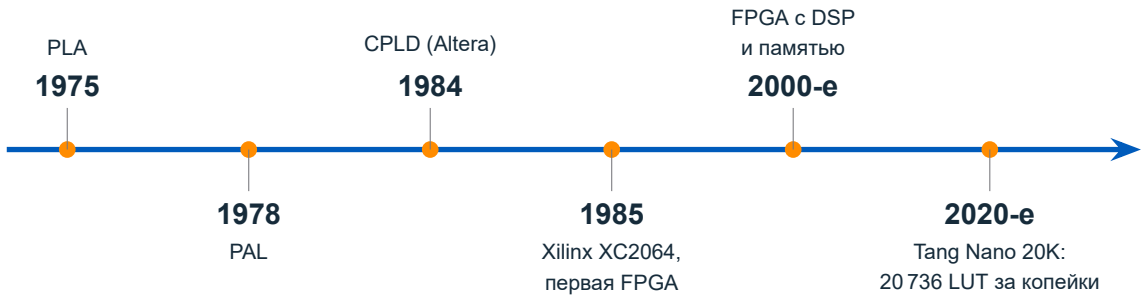


Рис. 10.2.: Эволюция программируемой логики

10.3. ПЛИС, микроконтроллер и заказная микросхема

Чтобы понять, зачем нужны ПЛИС, сравним три способа сделать «умное» цифровое устройство.

Микроконтроллер (МК) — готовый процессор, который *последовательно*, команда за командой, выполняет программу. Например, на МК STM32 работает автопилот квадрокоптера «Геоскан Пионер».

Заказная микросхема (ASIC) — схема, «выжженная» в кремнии на заводе. Она работает быстрее и экономичнее всех, но её разработка стоит миллионы долларов, и изменить её уже нельзя.

ПЛИС (FPGA) — золотая середина: аппаратная схема, как у ASIC, которую можно менять, как программу МК.

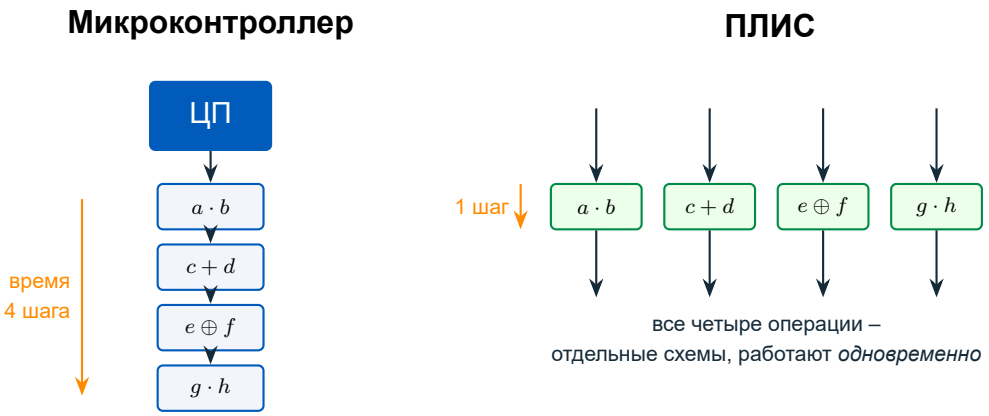


Рис. 10.3.: Процессор выполняет действия по очереди, ПЛИС — параллельно

На рис. 10.3 видна главная суперсила ПЛИС — **параллелизм**. Если нужно выполнить 1000 независимых операций, процессору нужно 1000 шагов, а в ПЛИС можно построить 1000 схем, которые справятся за один такт.

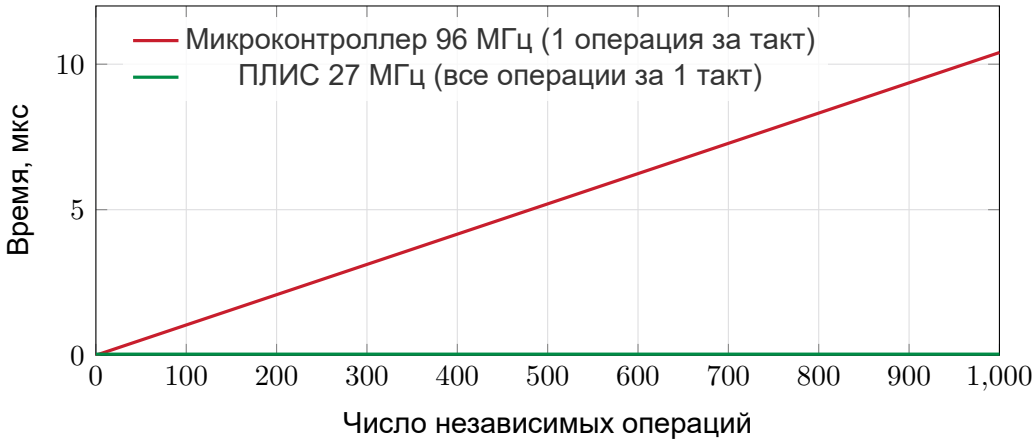


Рис. 10.4.: Время обработки при росте числа операций (упрощённая оценка)

Таблица 10.1.: Сравнение трёх подходов

	Микроконтроллер	**ПЛИС**	**ASIC**
Как работает	программа, по шагам	схема, параллельно	схема, параллельно
Скорость реакции	микросекунды	наносекунды	наносекунды
Можно изменить?	да	да	нет
Цена одного образца	низкая	средняя	огромная
Сложность разработки	низкая	средняя	очень высокая
Энергопотребление	низкое	среднее	самое низкое

10.4. Где применяются ПЛИС

ПЛИС выбирают там, где нужно одно из трёх: *очень быстро*, *очень много параллельно* или *точно по времени*.



Рис. 10.5.: Основные области применения ПЛИС

- **Цифровая обработка сигналов**: фильтры, преобразование Фурье, обработка звука и радиосигналов в реальном времени.
- **Видео**: генерация изображения для HDMI, обработка кадров с камер, поиск объектов прямо «на лету».
- **Связь и интерфейсы**: ПЛИС может «говорить» на любом протоколе (UART, SPI, I^2C, Ethernet, USB), даже на придуманном вами.
- **Робототехника и БПЛА**: точная генерация десятков ШИМ-сигналов, одновременный опрос многих датчиков, стабилизация видео.
- **Прототипирование**: прежде чем заказывать ASIC за миллионы долларов, инженеры проверяют его на ПЛИС.
- **Ретро-компьютеры**: на ПЛИС воссоздают старые игровые приставки и компьютеры (проект MiSTer), причём с точностью до такта.

Главная мысль

ПЛИС — это «пластилин» для цифровой схемотехники. Вы описываете схему, программа-синтезатор раскладывает её по ресурсам кристалла, и через минуту ваша схема уже работает в железе.

11 Что внутри ПЛИС

Если снять крышку с ПЛИС и посмотреть на кристалл под микроскопом, мы увидим аккуратную «шахматную доску»: одинаковые логические блоки, окружённые сетью проводников, а по краям — блоки ввода-вывода. Разберём все эти части по порядку.

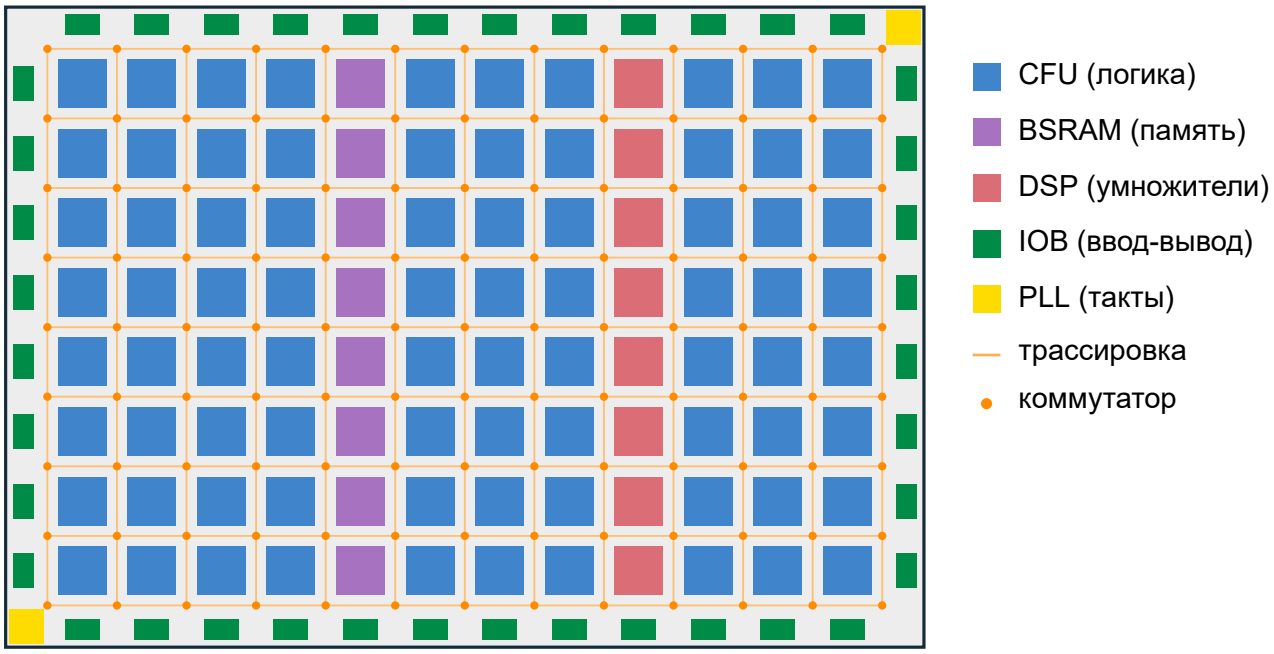


Рис. 11.1.: Упрощённое устройство кристалла ПЛИС

11.1. Главная идея: таблица истинности как память

Как сделать универсальный логический элемент, который по желанию становится И, ИЛИ, «Исключающим ИЛИ» или чем угодно ещё? Инженеры придумали изящный трюк.

Любая логическая функция от n входов полностью задаётся своей таблицей истинности, в которой 2^n строк. Значит, достаточно *запомнить* правый столбец таблицы в маленькой памяти, а входы использовать как *адрес* ячейки. Такое устройство называется **LUT** (Look-Up Table, таблица поиска).

Таблица 11.1.: Одна «заготовка» LUT2 — четыре разных функции

a	b	адрес	И	ИЛИ	И-НЕ	Искл. ИЛИ
0	0	0	0	0	1	0
0	1	1	0	1	1	1
1	0	2	0	1	1	1
1	1	3	1	1	0	0
содержимое памяти:			`1000`	`1110`	`0111`	`0110`

Чтобы LUT «стал» элементом И, в его память записывают числа $0, 0, 0, 1$; чтобы он стал ИЛИ — $0, 1, 1, 1$. Никакой перепайки: меняются лишь четыре бита конфигурации.

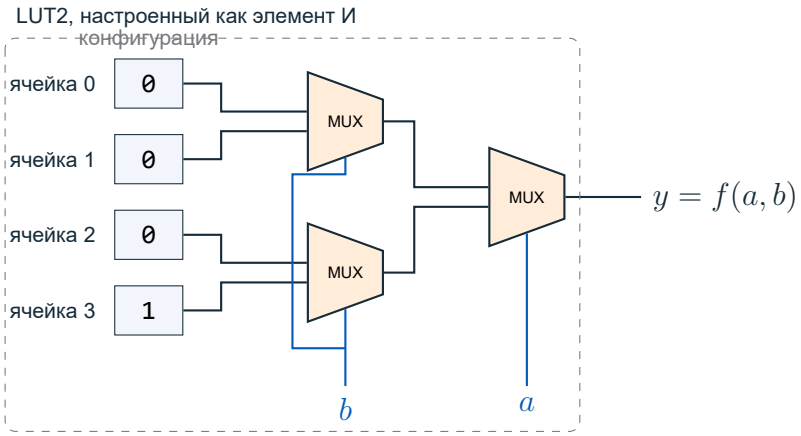


Рис. 11.2.: Устройство LUT2: четыре бита памяти и дерево мультиплексоров. Входы a и b выбирают одну из ячеек

В ПЛИС Gowin, установленной на Tang Nano 20K, используются **LUT4** — таблицы на 4 входа. Сколько различных функций может реализовать один LUT4? В памяти 16 бит, каждый может быть 0 или 1:

$$N = 2^{2^4} = 2^{16} = 65\,536 \text{ функций.}$$

Среди них и И, и ИЛИ-НЕ, и мажоритарный элемент, и полусумматор, и вообще всё, что вы можете придумать для четырёх входов.

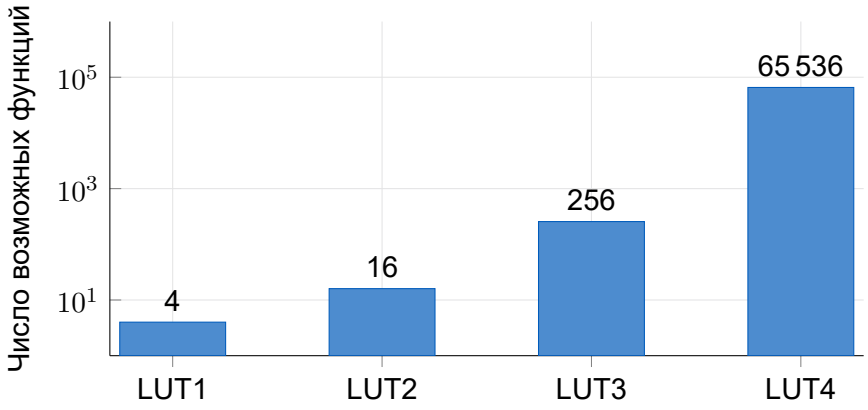


Рис. 11.3.: Число функций, которые умеет реализовывать LUT с n входами (2^{2^n})

А если входов больше четырёх?

Функция от 5, 6 и более переменных раскладывается на несколько LUT4, соединённых между собой. Синтезатор делает это автоматически, а специальные мультиплексоры MUX2 рядом с LUT помогают собрать из двух LUT4 один LUT5, из двух LUT5 — LUT6 и т. д. Вы увидите их в отчёте синтеза под именами `MUX2_LUT5`, `MUX2_LUT6`.

11.2. Триггер: у схемы появляется память

Одних LUT мало. Из главы 3 вы помните, что есть *комбинационные* схемы (выход зависит только от текущих входов) и *последовательностные*, у которых есть память. Для памяти рядом с каждым LUT стоит **D-триггер**.

Вспомним (глава 7): D-триггер запоминает значение входа D в момент *переднего фронта* тактового сигнала (перехода $0 \to 1$) и держит его на выходе Q до следующего фронта.

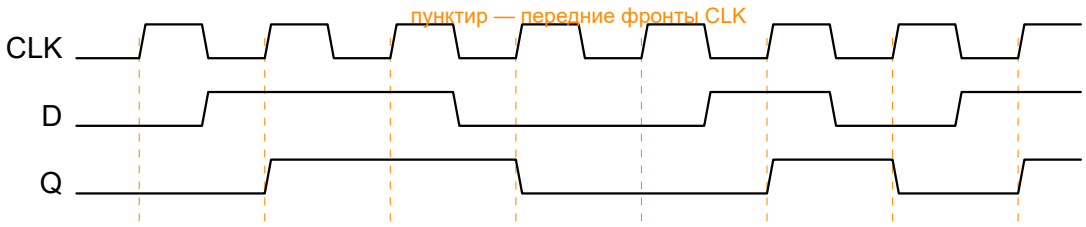


Рис. 11.4.: Временная диаграмма D-триггера: выход Q меняется только по переднему фронту CLK и принимает значение, которое было на D в этот момент

11.3. Логическая ячейка CFU

LUT и триггеры объединяются в **логические ячейки**. У Gowin такая ячейка называется **CFU** (Configurable Function Unit). Внутри CFU есть четыре «ломтика» CLS (Configurable

Logic Slice); в каждом по два LUT4, а в трёх из них ещё и по два триггера. Итого в одной CFU: 8 LUT4 и 6 триггеров.

Проверим: в ПЛИС GW2AR-18 содержится 2592 ячеек CFU, т. е. $2592 \cdot 8 = 20\,736$ LUT4 и $2592 \cdot 6 = 15\,552$ триггеров. Эти числа и указаны в документации на микросхему.

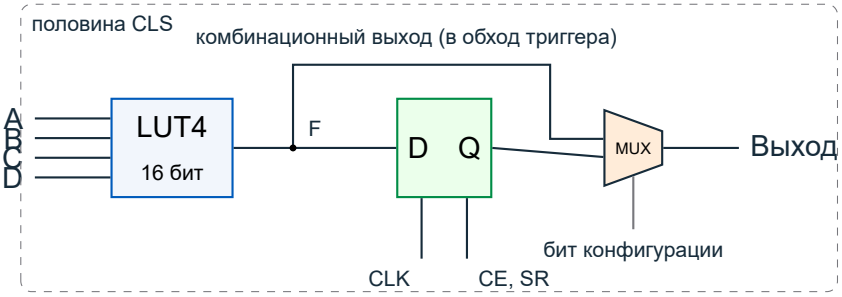


Рис. 11.5.: Базовый элемент логической ячейки: LUT4, D-триггер и мультиплексор, который выбирает, пойдёт ли на выход сигнал с триггера или прямо с LUT

11.4. Трассировочные ресурсы

Тысячи ячеек нужно соединять друг с другом. Для этого между ними проложены **трассировочные каналы** — пучки проводников разной длины (короткие для соседей, длинные для дальних блоков). На пересечениях стоят **коммутационные матрицы**: транзисторные ключи, каждым из которых управляет свой бит конфигурации.



Рис. 11.6.: Коммутационная матрица: замкнутые ключи образуют нужные соединения

Любопытный факт

Большая часть площади кристалла ПЛИС занята вовсе не логикой, а трассировкой и битами конфигурации, которые ею управляют. Поэтому ПЛИС медленнее и дороже ASIC той же сложности: за гибкость приходится платить.

11.5. Специализированные блоки

Кроме универсальной логики, в ПЛИС есть готовые «кирпичи» для частых задач. Сделать то же самое на LUT можно, но получится медленнее и займёт много места.

IOB (блоки ввода-вывода) связывают внутреннюю логику с ножками микросхемы. В IOB можно выбрать стандарт напряжения (LVCMOS 3,3 В, 1,8 В, LVDS…), силу тока, подтяжку к питанию или земле.

BSRAM (блочная память) — готовые блоки статической памяти по 18 Кбит. В GW2AR-18 их 46, всего 828 Кбит. Используются для буферов, FIFO, таблиц.

DSP (цифровая обработка сигналов) — аппаратные умножители 18×18 бит с сумматорами. Их 48 штук; они умножают за один такт.

PLL (фазовая автоподстройка частоты) — умеют из входной частоты 27 МГц получить почти любую другую: 50, 100, 126, 371 МГц… В GW2AR-18 два PLL.

Глобальная тактовая сеть — специальные провода, которые разносят тактовый сигнал по всему кристаллу с минимальным разбросом по времени.

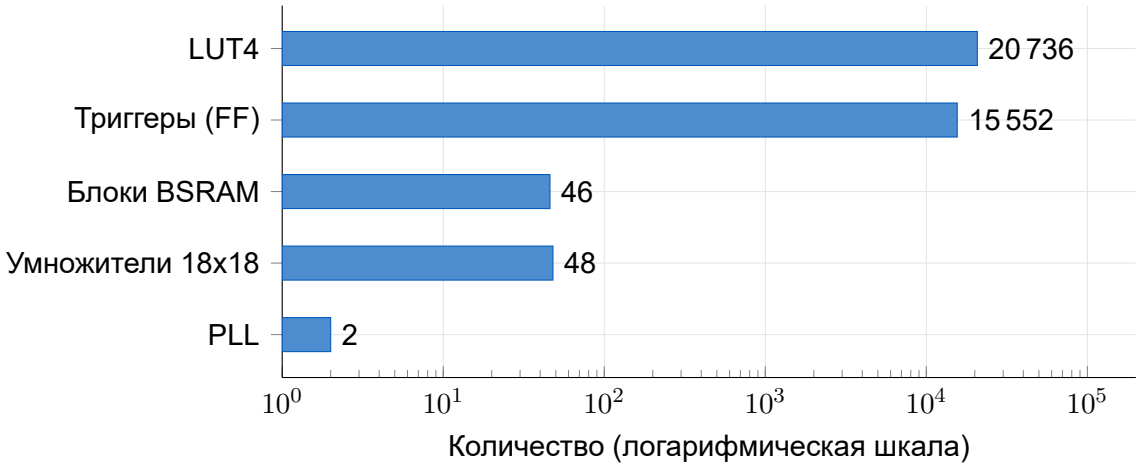


Рис. 11.7.: Ресурсы ПЛИС GW2AR-18 на плате Tang Nano 20K

11.6. Где хранится конфигурация

Память конфигурации ПЛИС Gowin сделана по технологии SRAM, как оперативная память компьютера: при выключении питания она стирается. Поэтому у ПЛИС два режима загрузки:

- **в SRAM** — быстро, удобно для отладки, но до выключения питания;
- **во Flash** — файл конфигурации записывается во флеш-память, и при каждом включении ПЛИС сама загружает из неё свою схему за десятки миллисекунд.

Главная мысль

Внутри ПЛИС: **LUT** (любая логическая функция), **триггеры** (память на 1 бит), **трассировка** (провода и ключи), **IOB** (ножки), а также готовые блоки **памяти**, **умножителей** и **PLL**. Конфигурация — это миллионы битов, которые задают содержимое каждого LUT и состояние каждого ключа.

12 Плата Sipeed Tang Nano 20K

12.1. Общий вид

Tang Nano 20K — маленькая (примерно 22×54 мм) и недорогая отладочная плата компании Sipeed. На ней установлена ПЛИС **Gowin GW2AR-LV18QN88C8/I7**, а также всё необходимое, чтобы сразу начать эксперименты: кнопки, светодиоды, разъём HDMI, слот для карты памяти и встроенный программатор с разъёмом USB-C.

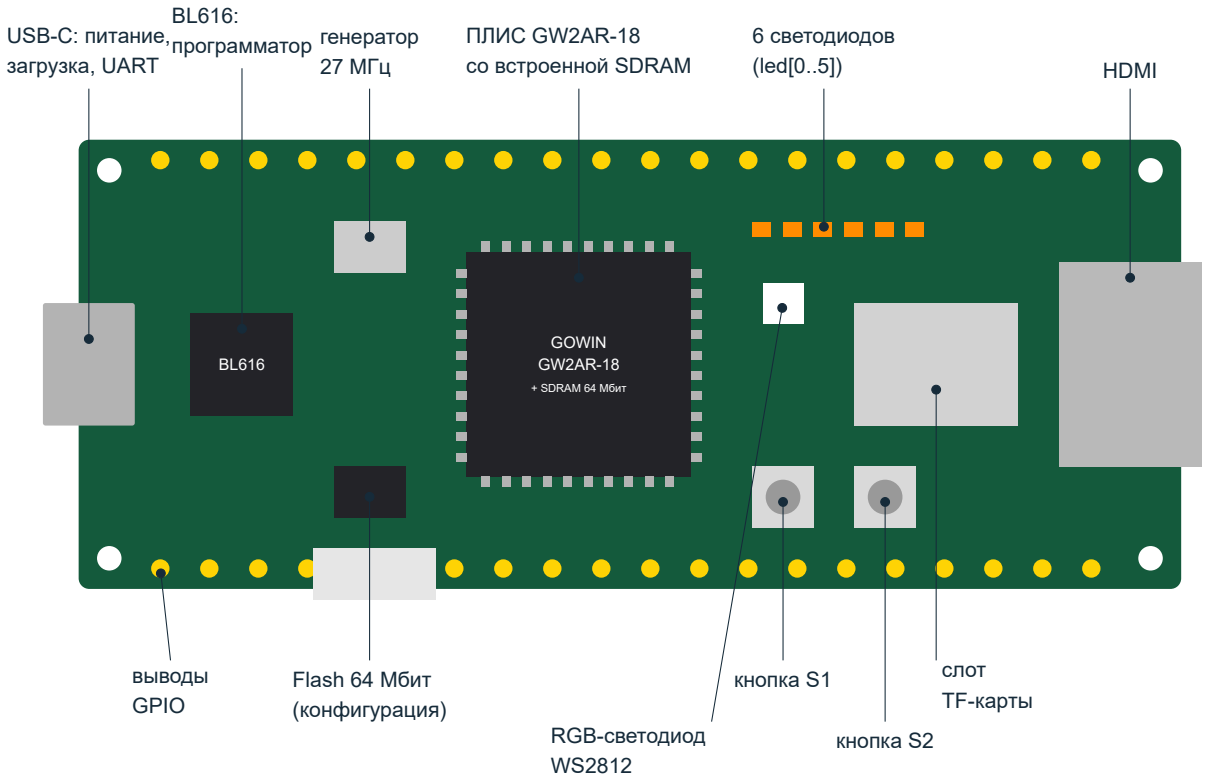


Рис. 12.1.: Расположение основных элементов на плате Tang Nano 20K (схематично)

12.2. Характеристики

Таблица 12.1.: Основные характеристики Tang Nano 20K

ПЛИС	Gowin GW2AR-LV18QN88C8/I7 (семейство GW2AR-18C, корпус QFN88)
Логика	20 736 LUT4, 15 552 триггеров
Распределённая память (SSRAM)	41 472 бит
Блочная память (BSRAM)	828 Кбит (46 блоков по 18 Кбит)
Встроенная SDRAM	64 Мбит (8 Мбайт), шина 32 бита — в том же корпусе, что и ПЛИС
Умножители	48 штук 18 × 18
PLL	2
Тактовый генератор	кварцевый, 27 МГц (плюс программируемый генератор MS5351)
Flash	64 Мбит, SPI — хранит конфигурацию ПЛИС
Программатор	BL616: USB-JTAG и USB-UART через один разъём USB-C
Периферия	HDMI, TF-карта, 6 светодиодов, 2 кнопки, RGB-светодиод WS2812, разъём для RGB-дисплея, усилитель звука
Напряжение на выводах	3,3 В (LVCMOS33)

Что значит «R» в названии GW2AR

Буква R означает, что в корпус микросхемы, помимо кристалла ПЛИС, установлен ещё один кристалл — память SDRAM. Такая сборка называется SiP (System in Package, «система в корпусе»). Провода между ПЛИС и SDRAM проложены внутри корпуса, поэтому на плате их не видно.

12.3. Структурная схема

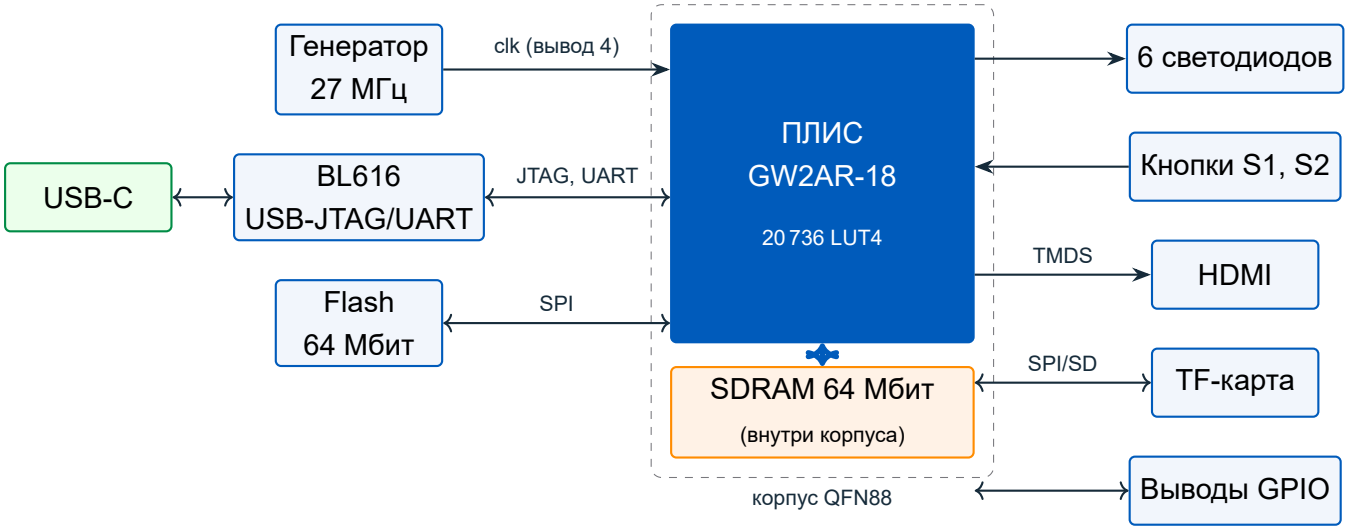


Рис. 12.2.: Структурная схема платы

12.4. Выводы, которые понадобятся в примерах

Каждая ножка ПЛИС имеет номер. Чтобы связать сигнал в коде (например, `led[0]`) с физической ножкой, используют файл ограничений `.cst` (Constraints). Номера выводов для Tang Nano 20K приведены в табл. 12.2.

Таблица 12.2.: Выводы ПЛИС, используемые в примерах

Сигнал	Вывод ПЛИС	Примечание
`clk`	4	тактовый генератор 27 МГц
`btn[0]`	88	кнопка S1, нажата → 1
`btn[1]`	87	кнопка S2, нажата → 1
`led[0]`	15	
`led[1]`	16	
`led[2]`	17	светодиоды, горят при **0**
`led[3]`	18	(активный уровень — низкий)
`led[4]`	19	
`led[5]`	20	
`uart_tx`	69	передача в компьютер через BL616
`uart_rx`	70	приём из компьютера
`ws2812`	79	RGB-светодиод

Светодиоды горят от нуля!

Светодиоды на плате подключены анодом к питанию $+3{,}3$ В, а катодом — через резистор к ножке ПЛИС. Ток течёт (и светодиод горит), когда на ножке **логический 0**. Поэтому во всех примерах перед выходом стоит инверсия: `assign led = ~value;`

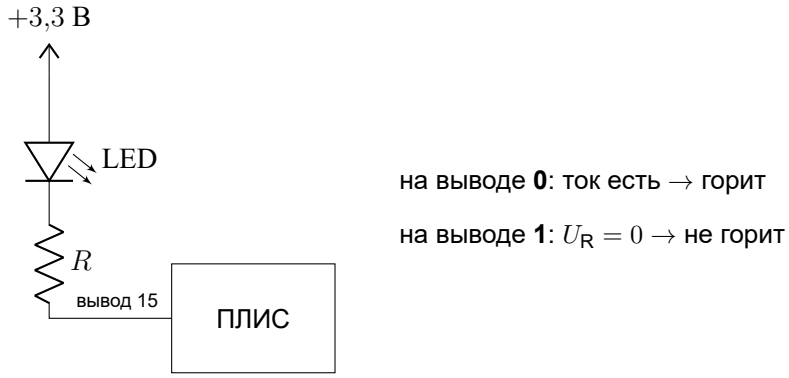


Рис. 12.3.: Подключение светодиода с активным низким уровнем

Кнопки

В примерах мы считаем, что нажатая кнопка даёт на выводе логическую 1. Если на вашей ревизии платы окажется наоборот, достаточно поставить инверсию на входе: `wire pressed = ~btn[0];`. Проверить это легко с помощью первого примера (глава 15).

13 От текста до работающей схемы

13.1. Маршрут проектирования

Для программиста путь от программы до работающей прошивки выглядит так: «исходный код → компилятор → машинные команды». У разработчика ПЛИС шагов больше, и каждый решает свою задачу.



Рис. 13.1.: Маршрут проектирования для ПЛИС

1. **Описание.** Вы пишете, *какой должна быть схема*, на языке описания аппаратуры (HDL, Hardware Description Language). Мы будем использовать самый распространённый из них — **Verilog**.
2. **Синтез.** Программа-синтезатор читает текст и превращает его в **список соединений** (netlist): «вот такие LUT, вот такие триггеры, вот так соединены». При этом логика упрощается, как при минимизации по картам Карно, только автоматически.
3. **Размещение и трассировка** (Place & Route). Каждый LUT и триггер из списка получает конкретное место на кристалле, а соединения прокладываются по трассировочным каналам. Здесь же используется файл `.cst`, чтобы порты схемы попали на нужные ножки.
4. **Битстрим.** Результат упаковывается в файл конфигурации `.fs`: те самые биты для каждого LUT и каждого ключа.
5. **Загрузка.** Файл по USB передаётся в ПЛИС (в SRAM) или во Flash-память на плате.

13.2. Что делает синтезатор

Рассмотрим, как синтезатор превращает текст в «железо». Пусть мы написали:

```
assign y = (a & b) | ~c;
```

Человек нарисовал бы схему из трёх элементов: И, НЕ и ИЛИ. Но у ПЛИС нет отдельных элементов — есть LUT. Функция зависит от трёх переменных, поэтому синтезатор вычисляет её таблицу истинности и записывает её в *один* LUT3 (или LUT4 с одним неиспользуемым входом).

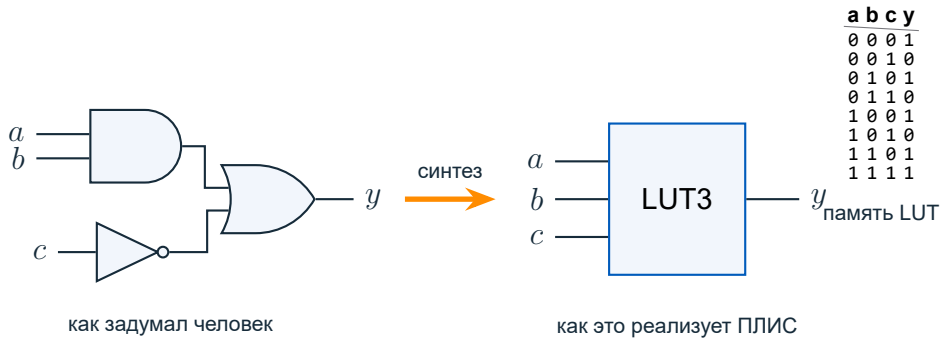


Рис. 13.2.: Синтез: три логических элемента превращаются в один LUT3 с таблицей истинности внутри

Главная мысль

Синтезатору всё равно, как вы нарисовали бы схему на бумаге. Важно лишь, *какую функцию* вы описали. Поэтому на Verilog описывают *поведение* схемы, а подбором элементов занимается программа.

13.3. Инструменты

Для Tang Nano 20K существуют два набора инструментов, и оба бесплатны.

13.3.1. Gowin EDA (официальная среда)

Среду разработки **Gowin EDA Education** можно скачать с сайта производителя (`https://www.gowinsemi.com`). Порядок работы:

1. **File** → **New** → **FPGA Design Project**, указать имя проекта.
2. Выбрать микросхему: серия **GW2AR**, корпус **QFN88**, скорость **C8/I7**, устройство **GW2AR-18C** (полное имя `GW2AR-LV18QN88C8/I7`).
3. **File** → **New** → **Verilog File** — создать файл модуля и написать код.
4. **File** → **New** → **Physical Constraints File** — создать файл `.cst` с номерами выводов.
5. Во вкладке **Process** запустить **Synthesize**, затем **Place & Route**.

6. Открыть **Programmer**, выбрать режим **SRAM Program** (для проверки) или **External Flash Mode** (чтобы схема сохранялась после выключения) и нажать **Program**.

13.3.2. Открытые инструменты

Существует полностью открытый набор программ, работающий в Linux, Windows и macOS (удобнее всего установить сборку **OSS CAD Suite**):

- **Yosys** — синтезатор;
- **nextpnr-himbaechel** — размещение и трассировка;
- **Apicula** (`gowin_pack`) — упаковка битстрима для Gowin;
- **openFPGALoader** — загрузка в плату;
- **Icarus Verilog** и **GTKWave** — симуляция и просмотр временных диаграмм.

Все команды собраны в файл `Makefile` в папке с примерами:

```
# 1. Синтез
yosys -p "read_verilog blink.v; synth_gowin -top blink -json blink.json"
# 2. Размещение и трассировка
nextpnr-himbaechel --json blink.json --write pnr.json \
    --device GW2AR-LV18QN88C8/I7 --vopt family=GW2A-18C \
    --vopt cst=blink.cst --freq 27
# 3. Битстрим
gowin_pack -d GW2A-18C -o blink.fs pnr.json
# 4. Загрузка: в SRAM ...
openFPGALoader -b tangnano20k blink.fs
# ... или во Flash
openFPGALoader -b tangnano20k -f blink.fs

# То же самое одной командой:
make PROJECT=02_blink TOP=blink load
```

13.4. Файл ограничений .cst

Код на Verilog ничего не знает о плате: в нём есть только имена портов (`clk`, `led`, `btn`…). Файл `.cst` (Physical Constraints, «физические ограничения») сообщает трассировщику, к какой ножке микросхемы подключить каждый порт и как эту ножку настроить.

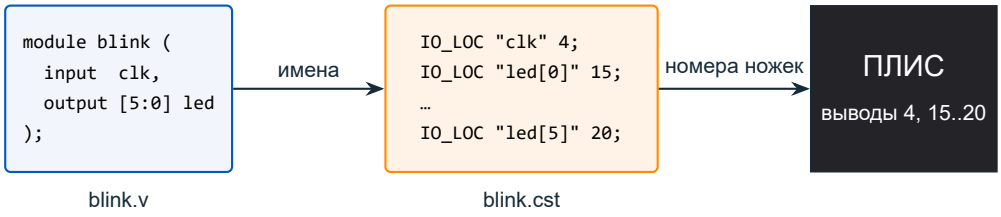


Рис. 13.3.: Файл `.cst` связывает имена портов модуля с ножками микросхемы

В файле встречаются всего две команды:

`IO_LOC "порт" номер;` — подключить порт к выводу с указанным номером. Номера выводов Tang Nano 20K приведены в табл. 12.2.

`IO_PORT "порт" параметр=значение ...;` — задать электрические параметры вывода.

Таблица 13.1.: Основные параметры команды `IO_PORT`

Параметр	Значения	Смысл
`IO_TYPE`	`LVCMOS33`, `LVCMOS18`, …	стандарт логических уровней; на Tang Nano 20K почти везде `LVCMOS33` (3,3 В), см. главу 1
`PULL_MODE`	`UP`, `DOWN`, `NONE`, `KEEPER`	встроенный резистор: подтяжка к питанию, к земле, без подтяжки или «удержание» последнего уровня
`DRIVE`	`4`, `8`, `12`, `16`, `24`	максимальный выходной ток в миллиамперах (только для выходов)

Правила записи:

- каждая команда заканчивается точкой с запятой;
- комментарии начинаются с `//`, как в Verilog;
- имена портов пишутся в кавычках и должны *в точности* совпадать с именами в модуле (регистр букв тоже важен);
- у шины каждый бит описывается отдельной строкой: `"led[0]"`, `"led[1]"`, …

Зачем нужна подтяжка? Вход, к которому ничего не подключено (например, вывод кнопки, пока она отпущена), «висит в воздухе» и ловит наводки: ПЛИС будет читать то 0, то 1. Резистор `PULL_MODE=DOWN` слабо притягивает такой вход к земле, и пока кнопка отпущена, на нём гарантированно 0.

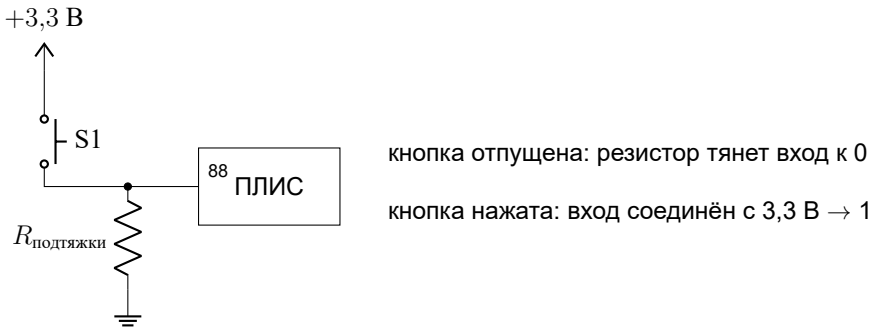


Рис. 13.4.: `PULL_MODE=DOWN`: резистор подтяжки включается внутри ПЛИС, паять его не нужно

13.4.1. Файлы .cst в примерах

У каждого проекта из главы 15 есть свой файл ограничений рядом с кодом, например `code/02_blink/blink.cst`. В нём описаны ровно те порты, которые есть в модуле, и каждая группа строк прокомментирована. Вот полный файл для мигалки:

code/02_blink/blink.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 02_blink
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля blink
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ============================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

Кроме того, в `code/common/tang_nano_20k.cst` собрана сводная распиновка всех выводов, которые используются в книге. Это справочник: не подключайте его к проекту целиком, а копируйте из него нужные строки.

Частые ошибки

Мы проверили поведение открытого трассировщика `nextpnr` на примерах:

- **Порт модуля не описан в `.cst`** — ошибка `Unconstrained IO`, прошивка не собирается. Так даже лучше: ошибка видна сразу.
- **В `.cst` описан порт, которого нет в модуле** — строка просто пропускается с со-

общением `Cell ... not found`, *без ошибки*. Обычно это опечатка в имени: тогда настоящий порт остаётся неописанным, и срабатывает предыдущая ошибка. Увидев `Unconstrained IO`, первым делом ищите опечатку в `.cst`.

- **Неверный номер вывода** — ошибка `Pin ... not found`.

13.4.2. Тактовая частота

Трассировщику нужно знать, на какой частоте будет работать схема, чтобы проверить, успевают ли сигналы пройти критический путь (глава 9). Для `nextpnr` частота задаётся ключом `--freq 27` (в мегагерцах), и в журнале появляется строка вида

```
Info: Max frequency for clock 'clk': 256.21 MHz (PASS at 27.00 MHz)
```

В Gowin EDA частоту задают в отдельном файле временных ограничений `.sdc`:

```
create_clock -name clk -period 37.037 [get_ports {clk}]
```

Здесь 37,037 нс — период сигнала 27 МГц.

14 Основы языка Verilog

14.1. Код описывает схему, а не последовательность действий

Главное, что нужно понять перед изучением Verilog: **это не язык программирования в привычном смысле**. Строчки кода на Verilog не выполняются одна за другой. Каждая строчка описывает кусок схемы, и все куски работают *одновременно*, как элементы на макетной плате.

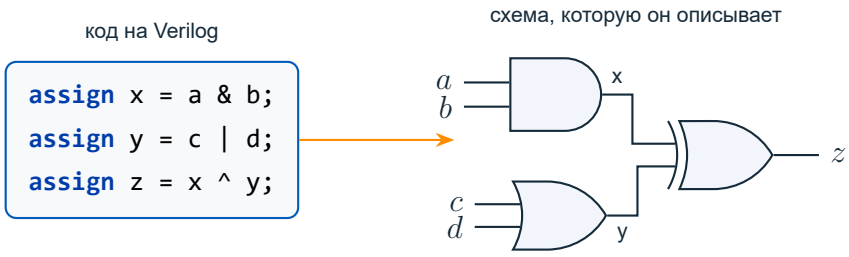


Рис. 14.1.: Три строчки кода — три логических элемента, работающих одновременно

Если поменять строчки местами, схема не изменится: провод x всё равно соединит выход элемента И со входом «Исключающего ИЛИ».

14.2. Модуль

Любая схема на Verilog оформляется как **модуль** (`module`). Модуль похож на микросхему: у него есть имя, входы, выходы и «начинка».

```
module half_adder (    // имя модуля
    input  wire a,     // входы
    input  wire b,
    output wire s,     // выходы
    output wire c
);
    assign s = a ^ b; // сумма
    assign c = a & b; // перенос
endmodule
```

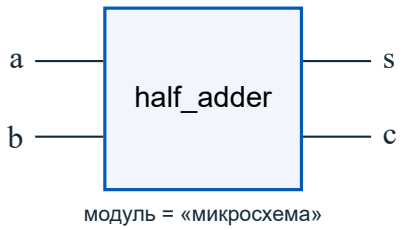


Рис. 14.2.: Модуль полусумматора и его «корпус»

Разберём ключевые слова:

- `module имя ( ... ); ... endmodule` — начало и конец описания;
- `input`, `output`, `inout` — направление порта;
- `wire` — провод (соединение);
- `assign` — «подключить к проводу слева выход схемы справа».

14.3. Сигналы: wire и reg

wire — провод. Не хранит значение, а лишь передаёт то, что на него подаёт какая-то схема. Присваивается через `assign`.

reg — переменная, которой присваивают значение внутри блока `always`. Несмотря на название, `reg` становится триггером *не всегда*: это зависит от того, в каком `always` он описан (см. раздел 14.5).

14.3.1. Шины (многоразрядные сигналы)

Несколько проводов можно объединить в **шину**:

```
wire [7:0] data;      // 8 проводов: data[7], data[6], ..., data[0]
reg  [5:0] counter;   // 6-разрядный регистр
wire       bit3 = data[3];      // один провод из шины
wire [3:0] low  = data[3:0];    // младшие 4 провода
```

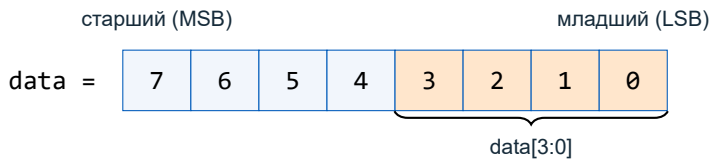


Рис. 14.3.: Шина `[7:0]`: нумерация разрядов справа налево

14.3.2. Запись чисел

Числа в Verilog записываются в формате `<разрядность>'<система><значение>`:

Таблица 14.1.: Запись чисел

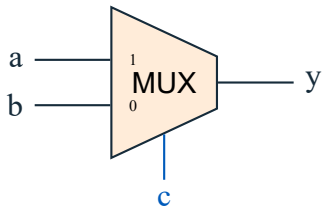
Запись	Значение	Пояснение
`4'b1010`	10	4 бита, двоичная система
`8'hFF`	255	8 бит, шестнадцатеричная
`6'd42`	42	6 бит, десятичная
`1'b0`, `1'b1`	0, 1	один бит
`27_000_000`	27000000	подчёркивания игнорируются, их ставят для удобства чтения
`4'bx01z`	—	`x` — неизвестно, `z` — отключено (высокий импеданс)

14.4. Операторы

Таблица 14.2.: Основные операторы Verilog

Группа	Операторы	Пример и смысл
Побитовые	`~ & | ^ ~& ~|`	`a & b` — И над каждой парой битов; `~a` — НЕ
Логические	`! && ||`	`(x > 3) && en` — результат 1 бит (истина/ложь)
Свёртка	`&a |a ^a`	`&a` — И всех битов шины (1, если все единицы)
Арифметика	`+ - * / %`	`cnt + 1'b1` — сумматор
Сравнение	`== != < > <= >=`	`cnt == 25` — компаратор
Сдвиги	`<< >>`	`x << 1` — сдвиг влево на 1
Склейка	`{a, b}`	`{4'b0, x}` — склеить шины в одну
Повтор	`{n{a}}`	`{6{1'b1}}` = `6'b111111`
Условие	`c ? a : b`	мультиплексор: если `c`, то `a`, иначе `b`

Каждый оператор — это кусок аппаратуры. `a + b` — это сумматор, `a == b` — компаратор, `c ? a : b` — мультиплексор.




```
assign y = c ? a : b;
```

Рис. 14.4.: Тернарный оператор `?:` — это мультиплексор

14.5. Блоки always

Описывать сложную логику одними `assign` неудобно. Для этого есть блок `always`. Он бывает двух видов, и важно их не путать.

14.5.1. Комбинационный: always @(*)

Звёздочка означает: «пересчитывай выход при любом изменении любого входа». Такой блок описывает **комбинационную схему без памяти**.

```
reg [1:0] y;
always @(*) begin
    case (sel)
        2'd0: y = a;
        2'd1: y = b;
        2'd2: y = c;
        default: y = 2'b00;
    endcase
end
```

14.5.2. Последовательностный: always @(posedge clk)

Этот блок срабатывает только по **переднему фронту тактового сигнала**. Все `reg`, которым присваивают значение внутри него, становятся **триггерами**.

```
reg [3:0] cnt = 0;
always @(posedge clk) begin
    cnt <= cnt + 1'b1;   // каждый такт прибавляем единицу
end
```

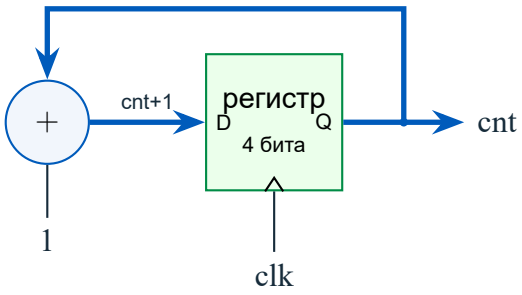


Рис. 14.5.: Счётчик: сумматор и регистр с обратной связью. Код `cnt <= cnt + 1` описывает именно эту схему

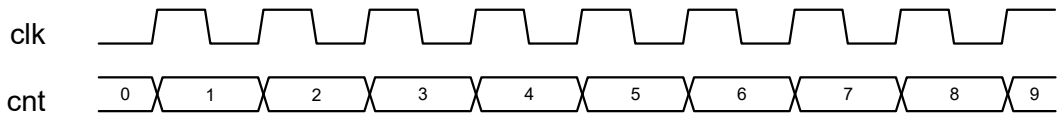


Рис. 14.6.: Временная диаграмма счётчика: значение увеличивается по каждому переднему фронту

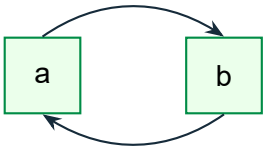
14.5.3. Блокирующее (=) и неблокирующее (<=) присваивание

Золотое правило

- В `always @(posedge clk)` используйте только `<=`.
- В `always @(*)` используйте только `=`.

Почему? Неблокирующее присваивание `<=` означает: «все правые части вычисляются по значениям *до* фронта, и все триггеры обновляются одновременно». Именно так ведут себя настоящие триггеры. Посмотрим на пример, в котором два регистра обмениваются значениями:

```
always @(posedge clk) begin
    a <= b;
    b <= a;    // обмен работает!
end
```




```
always @(posedge clk) begin
    a = b;
    b = a;     // ошибка: b = b
end
```

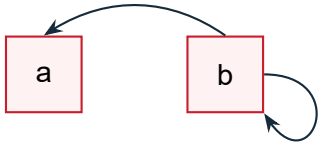


Рис. 14.7.: Слева — правильный обмен (`<=`), справа — ошибка (`=`): к моменту второй строки `a` уже равно `b`

14.6. Условия: if и case

Внутри `always` можно использовать условия. Они тоже превращаются в мультиплексоры.

```
always @(posedge clk) begin
    if (reset)
        cnt <= 0;                // синхронный сброс
    else if (enable)
        cnt <= cnt + 1'b1;       // счёт
    // иначе cnt сохраняет значение
end
```

Защёлки (latch)

Если в `always @(*)` при каком-то условии переменной ничего не присвоено, синтезатор вынужден «запомнить» старое значение и создаёт **защёлку** — нежелательный элемент памяти без такта. Всегда пишите `else` и `default`!

14.7. Параметры и константы

```
localparam integer CLK_HZ = 27_000_000;  // константа внутри модуля
localparam [1:0]   IDLE = 2'd0, RUN = 2'd1;

module blink #(parameter integer HZ = 1) (...);  // параметр можно
                                                  // менять снаружи
```

14.8. Иерархия: модуль внутри модуля

Большие схемы собирают из маленьких модулей, как устройство из микросхем. Подключение одного модуля внутри другого называется **созданием экземпляра** (instantiation).

```
module full_adder (
    input  wire a, b, cin,
    output wire s, cout
);
    wire s1, c1, c2;
    half_adder ha1 (.a(a),  .b(b),
                    .s(s1), .c(c1));
    half_adder ha2 (.a(s1), .b(cin),
                    .s(s),  .c(c2));
    assign cout = c1 | c2;
endmodule
```

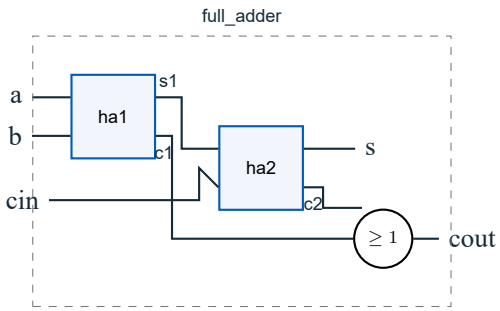


Рис. 14.8.: Полный сумматор из двух полусумматоров

Запись `.a(s1)` означает: «вход a экземпляра соединить с проводом `s1`».

Главная мысль

- `assign` и `always @(*)` описывают **комбинационную** логику (элементы И, ИЛИ, мультиплексоры, сумматоры);
- `always @(posedge clk)` описывает **триггеры** и всё, что перед ними;
- все блоки работают **одновременно**.

15 Практика: семь проектов

В этой главе мы шаг за шагом соберём семь схем, от простых логических элементов до конечного автомата и ШИМ. Каждая опирается на узлы из первой части книги. Каждый проект лежит в своей папке внутри `code/`: файл `.v` с описанием схемы и файл `.cst` с выводами (о том, как он устроен, — раздел 13.4).

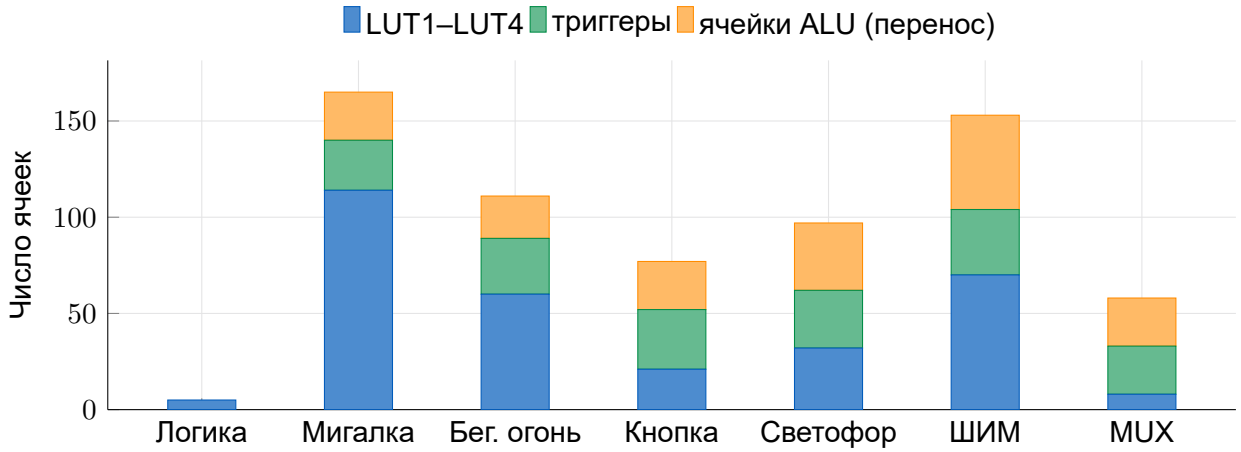


Рис. 15.1.: Сколько ресурсов занимает каждый проект (по отчёту синтезатора Yosys). Даже самый «тяжёлый» пример использует меньше 1 % ПЛИС

15.1. Проект 1. Логические элементы на кнопках

Цель: убедиться, что элементы И, ИЛИ, НЕ, И-НЕ, ИЛИ-НЕ и «Исключающее ИЛИ» действительно можно «собрать» кодом. Кнопки S1 и S2 будут входами a и b, шесть светодиодов — выходами шести функций.

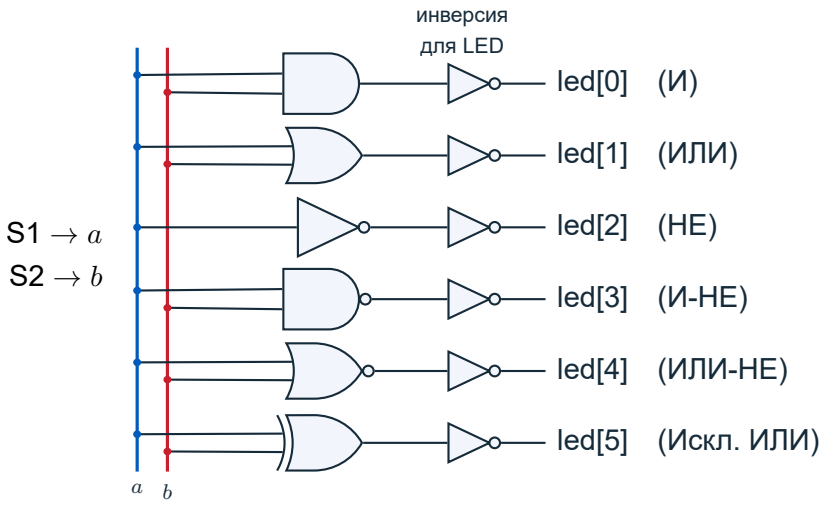


Рис. 15.2.: Схема проекта 1

code/01_gates/gates.v

```
// Пример 1. Логические элементы на кнопках.
// Две кнопки -- два входа, шесть светодиодов -- шесть функций.
// Светодиоды на плате загораются от нуля, поэтому на выходе стоит инверсия.
module gates (
    input  wire [1:0] btn,   // btn[0] = S1 (a), btn[1] = S2 (b)
    output wire [5:0] led    // активный уровень -- низкий
);
    wire a = btn[0];
    wire b = btn[1];

    wire [5:0] f;            // "логические" результаты: 1 = должен гореть

    assign f[0] = a & b;     // И
    assign f[1] = a | b;     // ИЛИ
    assign f[2] = ~a;        // НЕ
    assign f[3] = ~(a & b);  // И-НЕ
    assign f[4] = ~(a | b);  // ИЛИ-НЕ
    assign f[5] = a ^ b;     // Исключающее ИЛИ

    assign led = ~f;         // 1 -> 0 на ножке -> светодиод горит
endmodule
```

Файл выводов. Тактового сигнала здесь нет: схема чисто комбинационная, и в файле описаны только кнопки и светодиоды. Обратите внимание на `PULL_MODE=DOWN` у кнопок: без него отпущенная кнопка давала бы случайный уровень, и светодиоды мерцали бы.

code/01_gates/gates.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 01_gates
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля gates
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
```

```
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ===========================================================================

// В этом проекте нет тактового сигнала: схема чисто комбинационная.

// ==== Кнопки ====
// S1 -> btn[0] (вывод 88), S2 -> btn[1] (вывод 87).
// Нажатая кнопка подаёт на вывод логическую 1.
IO_LOC  "btn[0]" 88;
IO_LOC  "btn[1]" 87;
// PULL_MODE=DOWN включает внутренний резистор к земле: пока кнопка
// отпущена, на входе гарантированно 0, а не "висящий" неопределённый уровень.
IO_PORT "btn[0]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;
IO_PORT "btn[1]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

Таблица 15.1.: Какие светодиоды должны гореть (● — горит)

S1 (a)	S2 (b)	led0 И	led1 ИЛИ	led2 НЕ a	led3 И-НЕ	led4 ИЛИ-НЕ	led5 XOR
0	0			●	●	●	
0	1		●	●	●		●
1	0		●		●		●
1	1	●	●				

Что получилось после синтеза

Yosys сообщает, что схема заняла всего **5 LUT2**, а не 6 элементов и ещё 6 инверторов. Синтезатор объединил каждую функцию с инверсией на выходе в один LUT (например, $\overline{a \cdot b}$ стало одной таблицей). А для `led[2]` получилось $\overline{\overline{a}} = a$ — светодиод подключён к кнопке просто проводом, и LUT не понадобился вовсе!

Задание

1. Загрузите проект и по очереди проверьте все строки табл. 15.1.
2. Замените функцию на `led[5]` мажоритарной функцией от трёх сигналов: a, b и $a \oplus b$. Что получится и почему?

15.2. Проект 2. Мигающий светодиод

Цель: познакомиться с тактовым сигналом и счётчиком. Это «Hello, world!» мира ПЛИС.

Генератор выдаёт 27 МГц, то есть 27 000 000 импульсов в секунду. Человеческий глаз такого мигания не увидит. Нужно **поделить частоту**: считаем такты и каждые 13 500 000 тактов (0,5 с) переключаем светодиод. Какой разрядности нужен счётчик?

$2^{23} = 8\,388\,608 < 13\,500\,000 < 2^{24} = 16\,777\,216 \quad \Rightarrow$ достаточно 24 бит (мы взяли 25 с запасом).

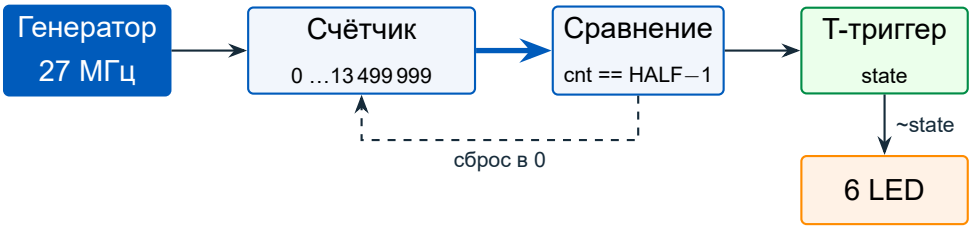


Рис. 15.3.: Структура делителя частоты

`code/02_blink/blink.v`

```
// Пример 2. Мигающий светодиод.
// Счётчик делит частоту 27 МГц до ~1 Гц.
module blink #(
    parameter integer CLK_HZ = 27_000_000     // частоту можно задать снаружи
) (
    input  wire       clk,       // 27 МГц
    output wire [5:0] led
);
    localparam integer HALF = CLK_HZ / 2;     // полпериода = 0,5 с

    reg [24:0] cnt   = 0;        // 2^25 > 13 500 000
    reg        state = 1'b0;
```

```
    always @(posedge clk) begin
        if (cnt == HALF - 1) begin
            cnt   <= 0;
            state <= ~state;     // переключаем раз в 0,5 с
        end else begin
            cnt <= cnt + 1'b1;
        end
    end

    assign led = {6{~state}};    // все шесть светодиодов мигают вместе
endmodule
```

Файл выводов: тактовый генератор на выводе 4 и шесть светодиодов. Кнопки в этом проекте не используются, поэтому их в файле нет — лишние строки трассировщик пропустил бы, но в аккуратном проекте их быть не должно.

code/02_blink/blink.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 02_blink
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля blink
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ============================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

```
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

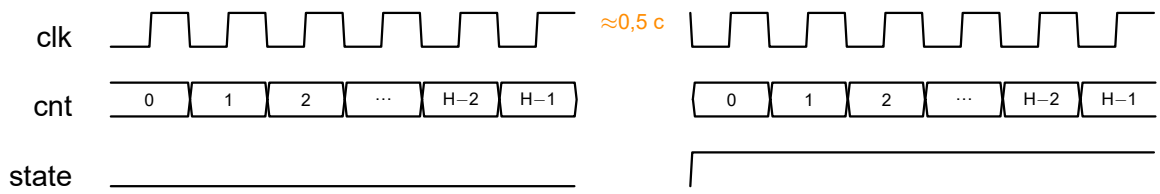


Рис. 15.4.: Временная диаграмма мигалки (H = HALF)

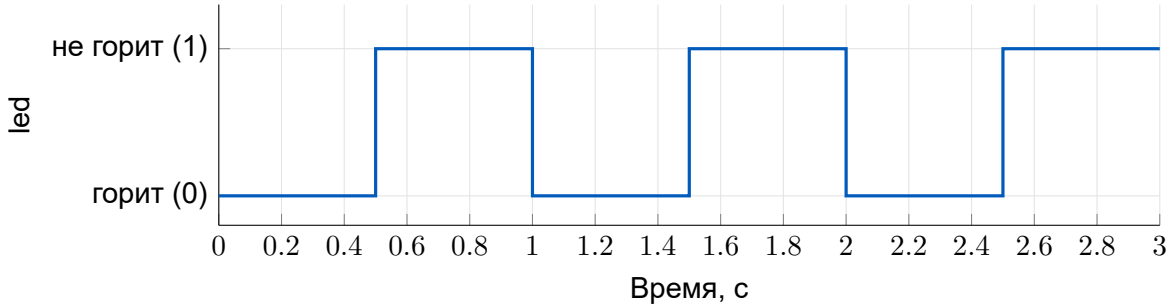


Рис. 15.5.: Сигнал на ножке светодиода: период 1 с, частота 1 Гц

Задание

1. Сделайте, чтобы светодиод мигал 5 раз в секунду.
2. Сделайте, чтобы `led[0]` мигал с частотой 1 Гц, `led[1]` — 2 Гц, `led[2]` — 4 Гц… Подсказка: используйте разные биты одного счётчика. Какой бит 25-разрядного счётчика меняется с частотой около 1 Гц?

15.3. Проект 3. Бегущий огонь

Цель: изучить сдвиговый регистр и сигнал разрешения (`tick`).

Шесть триггеров соединены в кольцо: по каждому импульсу `tick` значение каждого триггера переходит в соседний, а значение последнего — в первый. Если в начале в кольце одна единица, она будет «бегать» по кругу.

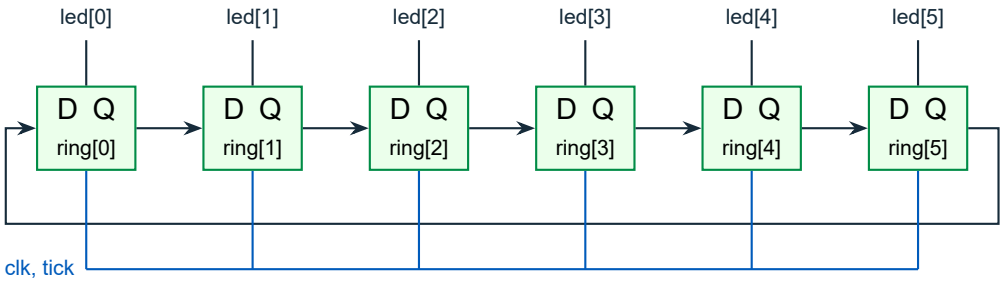


Рис. 15.6.: Кольцевой сдвиговый регистр из шести D-триггеров

code/03_running_lights/running_lights.v

```
// Пример 3. Бегущий огонь на кольцевом сдвиговом регистре.
module running_lights (
    input  wire       clk,       // 27 МГц
    output wire [5:0] led
);
    // Делитель: импульс tick длиной в один такт 8 раз в секунду
    localparam integer DIV = 27_000_000 / 8;

    reg [21:0] cnt  = 0;
    reg        tick = 1'b0;

    always @(posedge clk) begin
        if (cnt == DIV - 1) begin
            cnt  <= 0;
            tick <= 1'b1;
        end else begin
            cnt  <= cnt + 1'b1;
            tick <= 1'b0;
        end
    end

    // Кольцевой сдвиговый регистр: единица "бежит" по кругу
    reg [5:0] ring = 6'b000001;

    always @(posedge clk) begin
        if (tick)
            ring <= {ring[4:0], ring[5]};   // циклический сдвиг влево
    end

    assign led = ~ring;
endmodule
```

Файл выводов — такой же, как у мигалки: такт и светодиоды.

code/03_running_lights/running_lights.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 03_running_lights
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля running_lights
// с физическими ножками (выводами) микросхемы и задаёт их электрические
```

```
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ===========================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

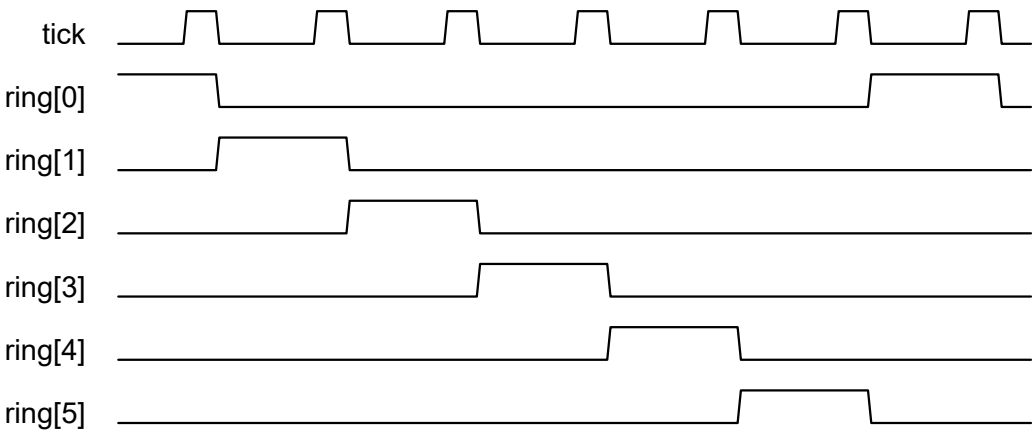


Рис. 15.7.: Единица «переезжает» в соседний разряд после каждого импульса `tick` (масштаб по времени условный)

Почему tick, а не отдельный медленный такт?

Можно было бы подать на триггеры самодельный медленный такт, взятый прямо с делителя. Но в ПЛИС так делать не принято: у такого сигнала нет доступа к глобальной тактовой сети, и схема может работать нестабильно. Правильный приём — **все триггеры тактируются одним `clk`**, а редкие события задаются **сигналом разрешения** длиной в один такт.

Задание

Сделайте огонь, который бегает туда и обратно («Кнайт Райдер»). Подсказка: нужен дополнительный регистр направления `dir`.

15.4. Проект 4. Кнопка: синхронизация, дребезг, фронт

Цель: научиться правильно работать с кнопками. Задача простая: считать нажатия S1 и показывать число в двоичном виде на светодиодах. Но на пути три препятствия.

15.4.1. Препятствие 1: дребезг контактов

Механические контакты кнопки при нажатии несколько миллисекунд «прыгают», и вместо одного перехода $0 \to 1$ получается целая серия. Схема, работающая на частоте 27 МГц, посчитает каждый «прыжок» как отдельное нажатие.

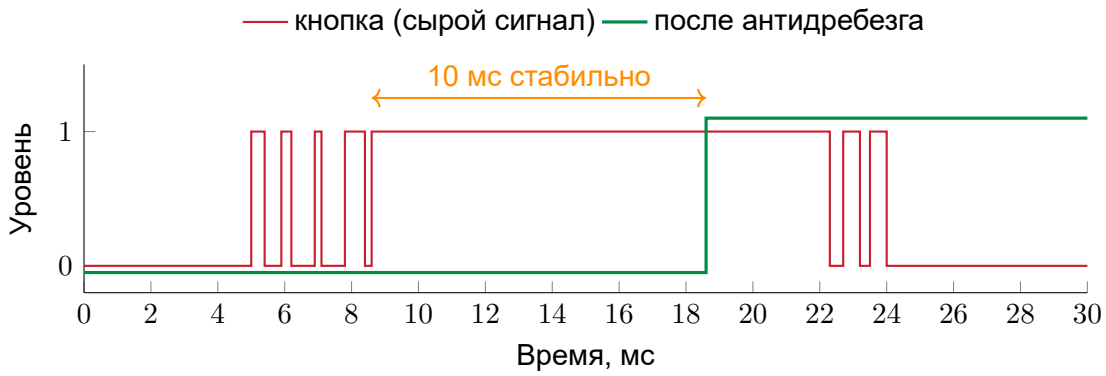


Рис. 15.8.: Дребезг контактов и его подавление: новое значение принимается, только если оно не меняется 10 мс

15.4.2. Препятствие 2: метастабильность

Кнопка нажимается в случайный момент, никак не связанный с тактом `clk`. Если вход триггера меняется точно в момент фронта, триггер может на какое-то время «зависнуть» между 0 и 1 (**метастабильное состояние**). Защита — **синхронизатор** из двух последовательно

включённых триггеров: даже если первый «задумается», ко второму фронту он почти наверняка успокоится.

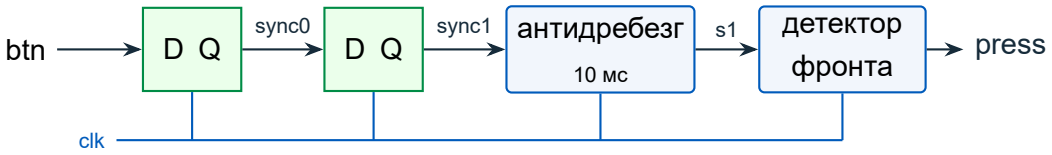


Рис. 15.9.: Цепочка обработки сигнала кнопки

15.4.3. Препятствие 3: нажатие длится миллионы тактов

Если писать `if (s1) count <= count + 1;`, счётчик будет прибавлять единицу *каждый такт*, пока кнопка нажата, — миллионы раз. Нужен **детектор переднего фронта**: сравниваем текущее значение с задержанным на такт. Импульс `press = s1 & ~s1_prev` длится ровно один такт.

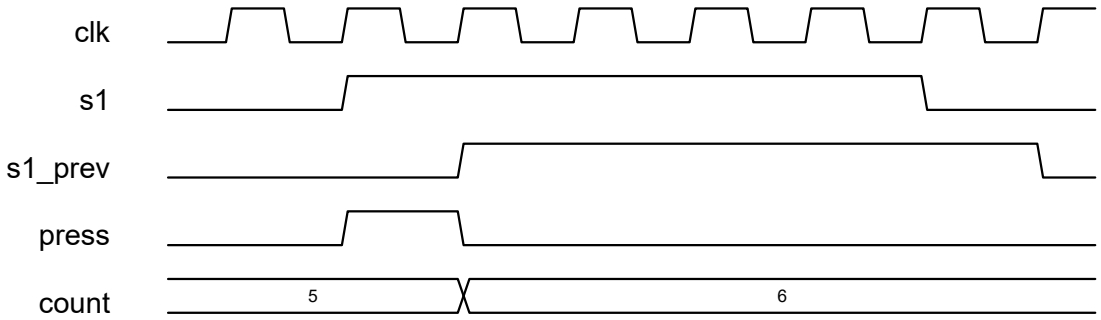


Рис. 15.10.: Детектор фронта: `press` равен 1 только один такт после нажатия

`code/04_button_edge/button_counter.v`

```
// Пример 4. Счётчик нажатий кнопки с подавлением дребезга.
// S1 -- прибавить единицу, S2 -- сбросить. Результат в двоичном виде на LED.
module button_counter (
    input  wire       clk,
    input  wire [1:0] btn,
    output wire [5:0] led
);
    // 1) Синхронизатор: два триггера подряд защищают от метастабильности
    reg [1:0] sync0 = 0, sync1 = 0;
    always @(posedge clk) begin
        sync0 <= btn;
        sync1 <= sync0;
    end

    // 2) Антидребезг: принимаем новое значение, только если оно
    //    держится 10 мс (270 000 тактов по 27 МГц)
    localparam integer STABLE = 270_000;
    reg [18:0] db_cnt = 0;
    reg        s1     = 1'b0;          // "чистое" состояние кнопки S1

    always @(posedge clk) begin
```

```
        if (sync1[0] == s1) begin
            db_cnt <= 0;              // ничего не поменялось
        end else if (db_cnt == STABLE - 1) begin
            s1     <= sync1[0];       // изменение подтверждено
            db_cnt <= 0;
        end else begin
            db_cnt <= db_cnt + 1'b1;
        end
    end

    // 3) Детектор переднего фронта: 1 ровно на один такт при нажатии
    reg  s1_prev = 1'b0;
    always @(posedge clk) s1_prev <= s1;
    wire press = s1 & ~s1_prev;

    // 4) Сам счётчик
    reg [5:0] count = 0;
    always @(posedge clk) begin
        if (sync1[1])                 // S2 -- сброс
            count <= 0;
        else if (press)
            count <= count + 1'b1;
    end

    assign led = ~count;
endmodule
```

Файл выводов: такт, обе кнопки и светодиоды. Подтяжка `PULL_MODE=DOWN` не спасает от дребезга — она лишь задаёт уровень отпущенной кнопки. С дребезгом справляется схема антидребезга в коде.

code/04_button_edge/button_counter.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 04_button_edge
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля button_counter
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ============================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;
```

```
// ==== Кнопки ====
// S1 -> btn[0] (вывод 88), S2 -> btn[1] (вывод 87).
// Нажатая кнопка подаёт на вывод логическую 1.
IO_LOC  "btn[0]" 88;
IO_LOC  "btn[1]" 87;
// PULL_MODE=DOWN включает внутренний резистор к земле: пока кнопка
// отпущена, на входе гарантированно 0, а не "висящий" неопределённый уровень.
IO_PORT "btn[0]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;
IO_PORT "btn[1]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

Задание

1. Сделайте, чтобы S2 не сбрасывал счётчик, а вычитал единицу. Для этого S2 тоже понадобятся антидребезг и детектор фронта. Удобно вынести их в отдельный модуль `debounce` и создать два экземпляра.
2. Что будет, если убрать антидребезг? Попробуйте и посчитайте, сколько лишних «нажатий» набегает.

15.5. Проект 5. Светофор: конечный автомат

Цель: освоить главный инструмент проектирования управляющей логики — **конечный автомат** (FSM, Finite State Machine).

Автомат находится в одном из конечного числа *состояний* и по определённым условиям *переходит* в другие. Светофор — классический пример:

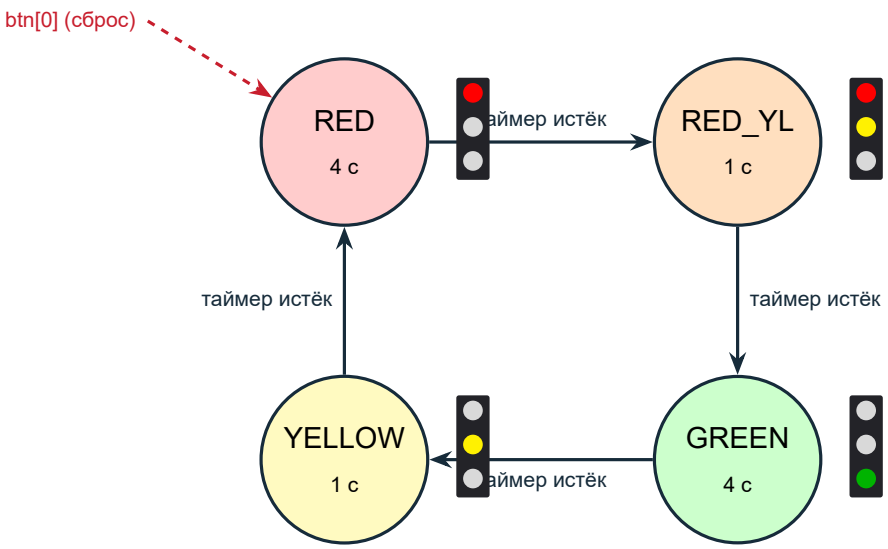


Рис. 15.11.: Граф переходов автомата «Светофор»

Автомат на Verilog описывается тремя частями:

1. **регистр состояния** (`state`) — триггеры, хранящие номер текущего состояния;
2. **логика переходов** — определяет следующее состояние;
3. **выходная логика** — что зажигать в каждом состоянии.

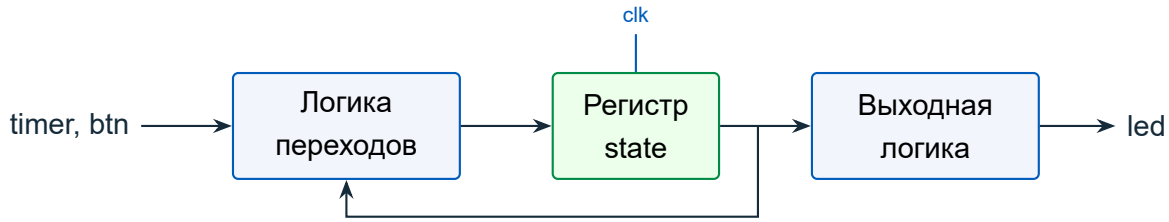


Рис. 15.12.: Структура автомата Мура: выход зависит только от состояния

`code/05_traffic_light/traffic_light.v`

```
// Пример 5. Светофор -- конечный автомат (FSM).
// led[0] -- красный, led[1] -- жёлтый, led[2] -- зелёный.
module traffic_light (
    input  wire       clk,
    input  wire [1:0] btn,        // btn[0] -- сброс
    output wire [5:0] led
);
    localparam integer SEC = 27_000_000;

    // Кодирование состояний
    localparam [1:0] RED    = 2'd0,
                     RED_YL = 2'd1,   // красный + жёлтый
                     GREEN  = 2'd2,
                     YELLOW = 2'd3;

    reg [1:0]  state = RED;
    reg [27:0] timer = 0;
```

```
    // Сколько тактов длится каждое состояние
    reg [27:0] duration;
    always @(*) begin
        case (state)
            RED:     duration = 4 * SEC;
            RED_YL:  duration = 1 * SEC;
            GREEN:   duration = 4 * SEC;
            YELLOW:  duration = 1 * SEC;
            default: duration = 1 * SEC;
        endcase
    end

    // Регистр состояния и логика переходов
    always @(posedge clk) begin
        if (btn[0]) begin
            state <= RED;
            timer <= 0;
        end else if (timer == duration - 1) begin
            timer <= 0;
            case (state)
                RED:     state <= RED_YL;
                RED_YL:  state <= GREEN;
                GREEN:   state <= YELLOW;
                YELLOW:  state <= RED;
            endcase
        end else begin
            timer <= timer + 1'b1;
        end
    end

    // Выходная логика (автомат Мура: выход зависит только от состояния)
    reg [2:0] lights;             // {зелёный, жёлтый, красный}
    always @(*) begin
        case (state)
            RED:     lights = 3'b001;
            RED_YL:  lights = 3'b011;
            GREEN:   lights = 3'b100;
            YELLOW:  lights = 3'b010;
            default: lights = 3'b001;
        endcase
    end

    assign led = ~{3'b000, lights};
endmodule
```

Файл выводов: такт, кнопки (используется S1 — сброс) и светодиоды. Порт `btn` в модуле двухбитный, поэтому описаны оба бита, хотя `btn[1]` не используется: иначе трассировщик сообщит, что порт не описан.

code/05_traffic_light/traffic_light.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 05_traffic_light
```

```
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля traffic_light
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ===========================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;

// ==== Кнопки ====
// S1 -> btn[0] (вывод 88), S2 -> btn[1] (вывод 87).
// Нажатая кнопка подаёт на вывод логическую 1.
IO_LOC  "btn[0]" 88;
IO_LOC  "btn[1]" 87;
// PULL_MODE=DOWN включает внутренний резистор к земле: пока кнопка
// отпущена, на входе гарантированно 0, а не "висящий" неопределённый уровень.
IO_PORT "btn[0]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;
IO_PORT "btn[1]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

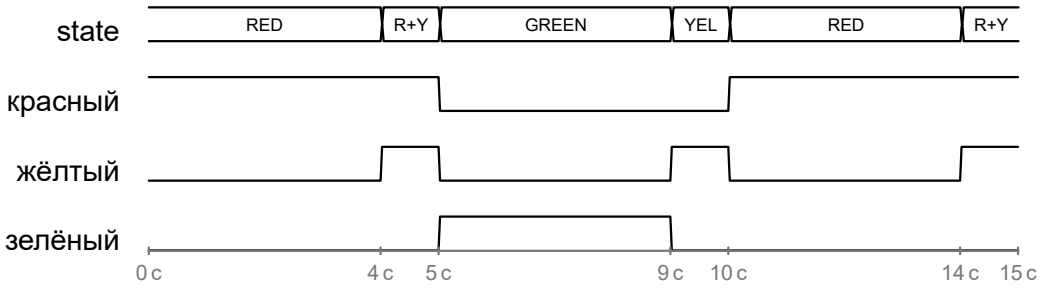


Рис. 15.13.: Работа светофора во времени (1 = лампа горит)

Задание

Добавьте состояние «мигающий зелёный» (3 раза по 0,5 с) между GREEN и YELLOW. Нарисуйте новый граф переходов перед тем, как писать код!

15.6. Проект 6. «Дышащий» светодиод: ШИМ

Цель: научиться управлять яркостью. У цифровой ножки только два состояния: 0 и 1. Но если очень быстро переключать светодиод, глаз увидит *среднюю* яркость. Это **широтно-импульсная модуляция** (ШИМ, англ. PWM): частота постоянна, а меняется доля времени, когда сигнал равен 1, — **коэффициент заполнения** D:

$$D = \frac{t_{\text{вкл}}}{T} \cdot 100\,\%, \qquad U_{\text{ср}} = D \cdot U_{\text{пит}}.$$

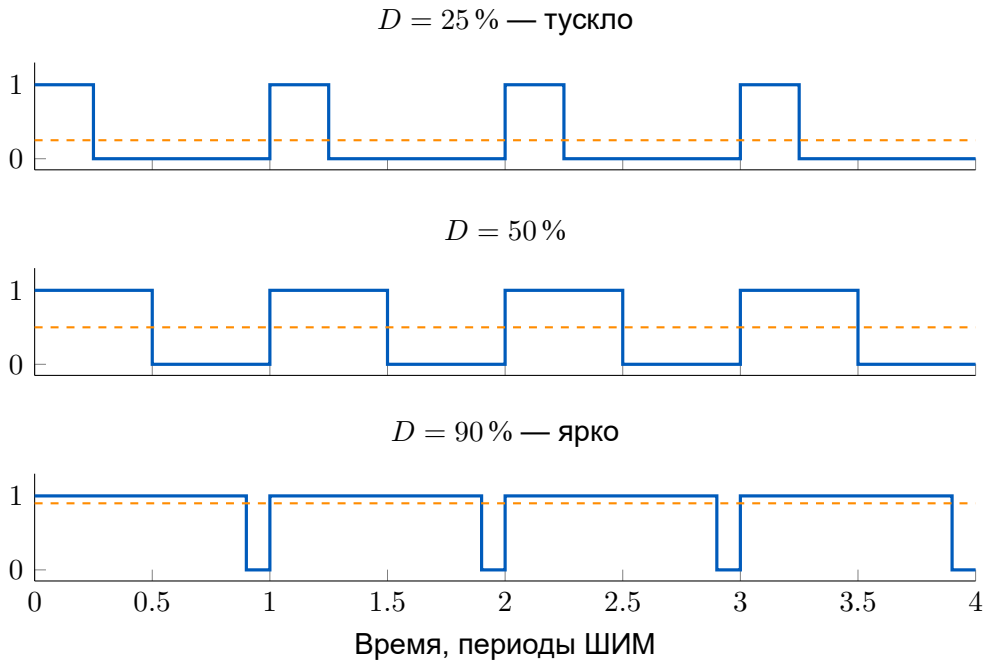


Рис. 15.14.: ШИМ с разным коэффициентом заполнения. Пунктир — среднее значение, которое «видит» глаз

Как получить ШИМ на ПЛИС? Нужен свободно бегущий счётчик и компаратор: пока счётчик меньше заданного числа `duty`, выход равен 1.

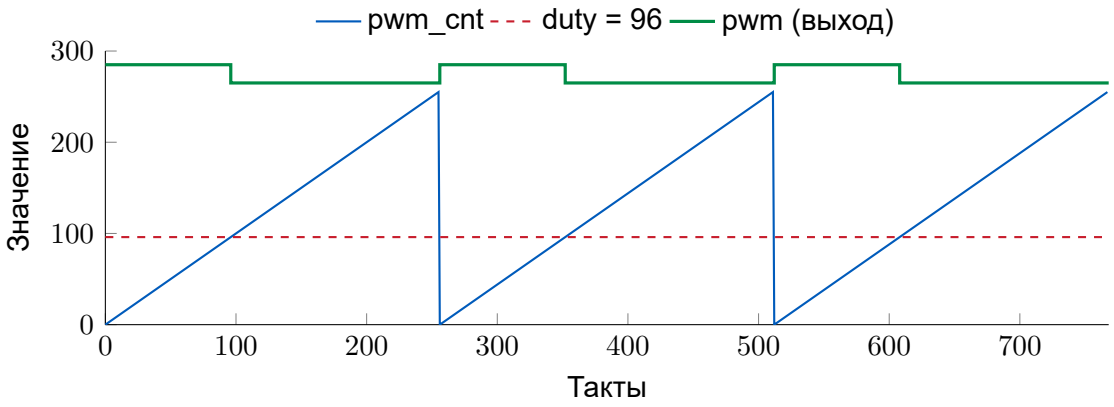


Рис. 15.15.: Счётчик-«пила» и компаратор: выход `pwm` равен 1, пока `pwm_cnt < duty`

Чтобы светодиод «дышал», будем медленно увеличивать `duty` от 0 до 255, а затем уменьшать обратно.

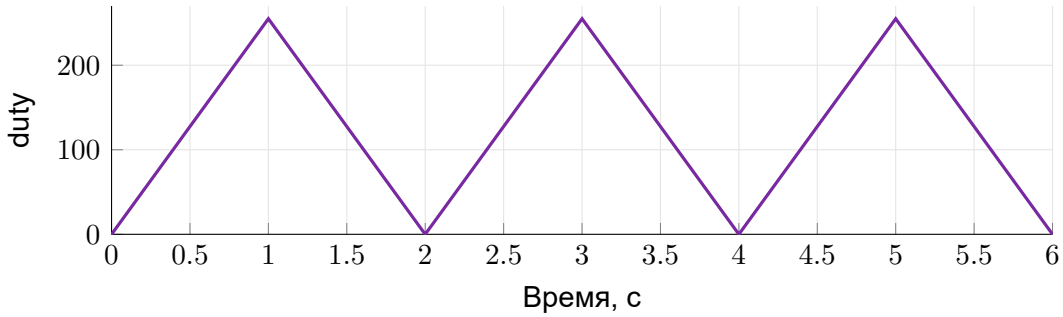


Рис. 15.16.: Изменение коэффициента заполнения: «вдох» за 1 с и «выдох» за 1 с

code/06_pwm_breath/pwm_breath.v

```
// Пример 6. "Дышащий" светодиод на ШИМ.
module pwm_breath (
    input  wire       clk,        // 27 МГц
    output wire [5:0] led
);
    // Генератор ШИМ: 8-битный счётчик -> частота 27 МГц / 256 ~ 105 кГц
    reg [7:0] pwm_cnt = 0;
    always @(posedge clk) pwm_cnt <= pwm_cnt + 1'b1;

    // Медленно меняем скважность: 0 -> 255 -> 0 примерно за 2 секунды
    localparam integer STEP = 27_000_000 / 512;
    reg [16:0] step_cnt = 0;
    reg [7:0]  duty     = 0;
    reg        up       = 1'b1;

    always @(posedge clk) begin
        if (step_cnt == STEP - 1) begin
            step_cnt <= 0;
            if (up) begin
```

```
                if (duty == 8'd255) up <= 1'b0; else duty <= duty + 1'b1;
            end else begin
                if (duty == 8'd0)   up <= 1'b1; else duty <= duty - 1'b1;
            end
        end else begin
            step_cnt <= step_cnt + 1'b1;
        end
    end

    // Компаратор: выход равен 1, пока счётчик меньше заданной скважности
    wire pwm = (pwm_cnt < duty);

    assign led = {6{~pwm}};
endmodule
```

Файл выводов: такт и светодиоды. `DRIVE=8` ограничивает ток выхода; яркость ШИМ от этого не зависит — она задаётся долей времени, когда светодиод включён.

code/06_pwm_breath/pwm_breath.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 06_pwm_breath
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля pwm_breath
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ============================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

```
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

Почему светодиод кажется «неравномерным»

Глаз воспринимает яркость нелинейно: разница между $D = 5\,\%$ и $10\,\%$ заметна гораздо сильнее, чем между $90\,\%$ и $95\,\%$. Чтобы «дыхание» выглядело плавным, в настоящих устройствах используют **гамма-коррекцию**: таблицу, которая переводит линейную яркость в нужное значение `duty`. Такую таблицу удобно хранить в блочной памяти BSRAM.

Задание

1. Сделайте, чтобы шесть светодиодов «дышали» со сдвигом по фазе («волна»).
2. Добавьте управление: кнопка S1 увеличивает яркость на 1/8, S2 — уменьшает (используйте модуль антидребезга из проекта 4).

15.7. Проект 7. Дешифратор, мультиплексор и демультиплексор

Цель: собрать в ПЛИС узлы из глав 4 и 5 и увидеть их работу на светодиодах.

Кнопки S1 и S2 образуют двухбитный адрес s (от 0 до 3). Счётчик-делитель выдаёт четыре сигнала разной частоты — это его старшие разряды (рис. 8.8).

- **Мультиплексор 4:1** выбирает по адресу один из четырёх сигналов и подаёт его на `led[4]`: чем больше номер, тем быстрее мигание.
- **Дешифратор 2→4** превращает адрес в унитарный код.
- **Демультиплексор 1:4** — это тот же дешифратор, у которого выходы «разрешаются» информационным сигналом. Мигание попадает только на светодиод `led[s]`.

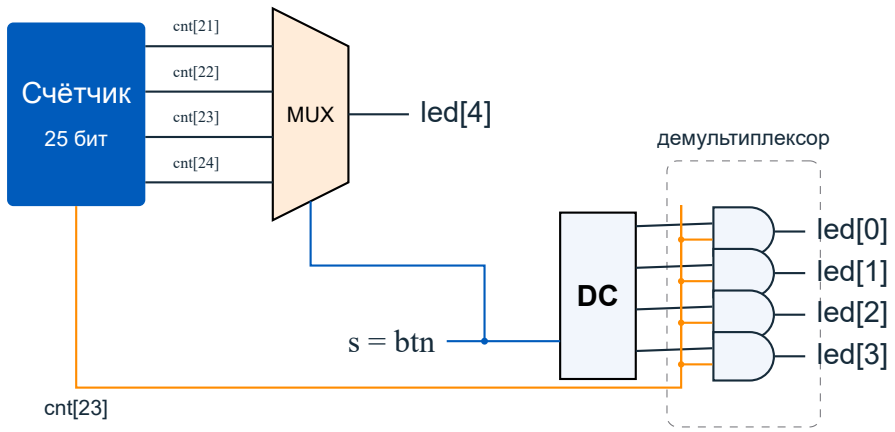


Рис. 15.17.: Схема проекта 7. Демультиплексор = дешифратор + элементы И

code/07_mux_demux/mux_demux.v

```
// Пример 7. Дешифратор, мультиплексор и демультиплексор на плате.
// Кнопки S1, S2 задают двухбитный адрес s.
//   led[4]   -- выход мультиплексора 4:1: мигает с частотой, выбранной адресом;
//   led[3:0] -- выходы демультиплексора 1:4: мигающий сигнал попадает
//               только на светодиод с номером s.
module mux_demux (
    input  wire       clk,       // 27 МГц
    input  wire [1:0] btn,       // адрес s = {S2, S1}
    output wire [5:0] led
);
    // Счётчик-делитель: его старшие разряды -- четыре сигнала разной частоты
    reg [24:0] cnt = 0;
    always @(posedge clk) cnt <= cnt + 1'b1;

    wire [3:0] x = cnt[24:21];   // ~0,8 Гц, ~1,6 Гц, ~3,2 Гц, ~6,4 Гц
    wire [1:0] s = btn;

    // Мультиплексор 4:1 -- два уровня мультиплексоров 2:1 (дерево)
    wire m0 = s[0] ? x[1] : x[0];
    wire m1 = s[0] ? x[3] : x[2];
    wire y  = s[1] ? m1   : m0;

    // Дешифратор 2 -> 4: единица на выходе с номером s
    reg [3:0] dc;
    always @(*) begin
        case (s)
            2'd0:    dc = 4'b0001;
            2'd1:    dc = 4'b0010;
            2'd2:    dc = 4'b0100;
            default: dc = 4'b1000;
        endcase
    end

    // Демультиплексор 1:4 = дешифратор, на вход разрешения которого
    // подан информационный сигнал (здесь это мигание ~1,6 Гц)
    wire       data  = cnt[23];
    wire [3:0] demux = dc & {4{data}};
```

```
    // led[5] не используется; инверсия -- светодиоды горят от нуля
    assign led = ~{1'b0, y, demux};
endmodule
```

Файл выводов: такт, обе кнопки (они образуют адрес s) и светодиоды.

code/07_mux_demux/mux_demux.cst

```
// ============================================================================
// Файл ограничений (Physical Constraints) для проекта 07_mux_demux
// Плата: Sipeed Tang Nano 20K, ПЛИС GW2AR-LV18QN88C8/I7
//
// Что делает этот файл: связывает имена портов модуля mux_demux
// с физическими ножками (выводами) микросхемы и задаёт их электрические
// параметры. В файле перечислены ровно те порты, что есть в модуле.
//
// Синтаксис:
//   IO_LOC  "имя_порта" номер_вывода;          -- к какой ножке подключить
//   IO_PORT "имя_порта" параметр=значение ...; -- как её настроить
// Каждая строка заканчивается точкой с запятой, комментарии начинаются с //.
// Для шины каждый бит описывается отдельно: "led[0]", "led[1]", ...
// ============================================================================

// ==== Тактовый сигнал ====
// Кварцевый генератор 27 МГц на плате подключён к выводу 4.
IO_LOC  "clk" 4;
// LVCMOS33 -- обычная логика с питанием 3,3 В.
IO_PORT "clk" IO_TYPE=LVCMOS33 PULL_MODE=UP;

// ==== Кнопки ====
// S1 -> btn[0] (вывод 88), S2 -> btn[1] (вывод 87).
// Нажатая кнопка подаёт на вывод логическую 1.
IO_LOC  "btn[0]" 88;
IO_LOC  "btn[1]" 87;
// PULL_MODE=DOWN включает внутренний резистор к земле: пока кнопка
// отпущена, на входе гарантированно 0, а не "висящий" неопределённый уровень.
IO_PORT "btn[0]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;
IO_PORT "btn[1]" IO_TYPE=LVCMOS33 PULL_MODE=DOWN;

// ==== Светодиоды ====
// Шесть светодиодов на выводах 15..20. Они подключены катодом к ножке,
// поэтому горят, когда на выходе 0 (активный низкий уровень).
IO_LOC  "led[0]" 15;
IO_LOC  "led[1]" 16;
IO_LOC  "led[2]" 17;
IO_LOC  "led[3]" 18;
IO_LOC  "led[4]" 19;
IO_LOC  "led[5]" 20;
// DRIVE=8 -- выход может отдать ток до 8 мА, этого хватает светодиоду.
// PULL_MODE=UP -- пока ПЛИС не загружена, выводы подтянуты к 3,3 В и
// светодиоды не горят.
IO_PORT "led[0]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[1]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

```
IO_PORT "led[2]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[3]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[4]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
IO_PORT "led[5]" IO_TYPE=LVCMOS33 PULL_MODE=UP DRIVE=8;
```

Обратите внимание, что один и тот же узел можно описать по-разному: мультиплексор здесь записан как дерево операторов `?:` (как на рис. 5.5), а дешифратор — как `case`. Короче всего мультиплексор записывается как `assign y = x[s];` — синтезатор получит ту же схему.

Что получилось после синтеза

Yosys разместил мультиплексор не в обычных LUT, а в специальных мультиплексорах `MUX2_LUT5` и `MUX2_LUT6`, которые стоят в каждой логической ячейке рядом с LUT. Весь проект занял 8 LUT и 25 триггеров (триггеры — это счётчик).

Задание

1. Замените мультиплексор строкой `assign y = x[s];` и сравните отчёты синтеза.
2. Добавьте приоритетный шифратор (глава 4): выведите на `led[5]` признак V «нажата хотя бы одна кнопка».
3. Выведите номер s на семисегментный индикатор, подключённый к выводам GPIO, используя дешифратор из табл. 4.1.

16 Симуляция: проверяем схему до загрузки

16.1. Зачем нужна симуляция

Отладка на плате напоминает поиск неисправности с завязанными глазами: видно только шесть светодиодов, а внутри тысячи сигналов, и меняются они 27 миллионов раз в секунду. Поэтому профессионалы **сначала проверяют схему в симуляторе** — программе, которая вычисляет значения всех сигналов во времени и рисует временные диаграммы.

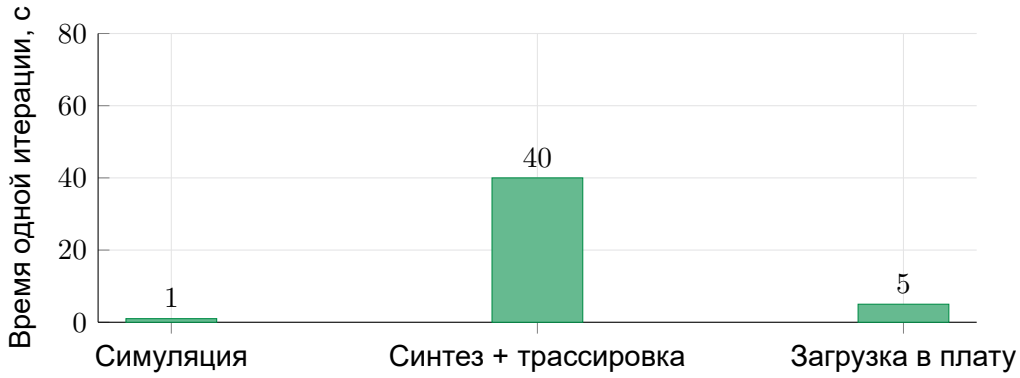


Рис. 16.1.: Типичное время одного цикла «исправил — проверил» для небольшого проекта. В симуляторе видны все внутренние сигналы, а на плате — только выходы

16.2. Тестбенч

Для симуляции пишут ещё один модуль на Verilog — **тестбенч** (testbench, «испытательный стенд»). У него нет портов: он сам создаёт входные сигналы, подключает к ним проверяемый модуль (DUT — Device Under Test) и следит за выходами.

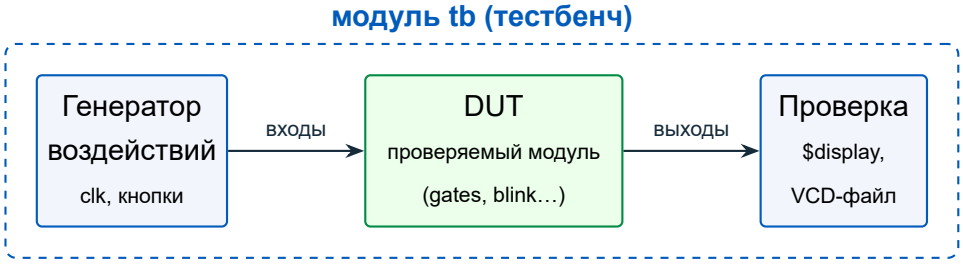


Рис. 16.2.: Устройство тестбенча

В тестбенче разрешены конструкции, которые *нельзя* синтезировать в «железо», но удобно использовать для моделирования:

- `#10;` — подождать 10 единиц времени;
- `initial begin ... end` — выполнить действия один раз с начала моделирования;
- `$display(...)`, `$monitor(...)` — печать в консоль;
- `$dumpfile`, `$dumpvars` — запись всех сигналов в файл `.vcd` для просмотра в GTKWave;
- `$finish` — завершить моделирование.

16.3. Пример 1: таблица истинности проекта 1

code/01_gates/tb_gates.v

```
// Тестбенч для проекта 1: перебираем все комбинации кнопок
// и печатаем таблицу истинности.
`timescale 1ns/1ps
module tb_gates;
    reg  [1:0] btn;          // сигналы, которые "нажимает" тестбенч
    wire [5:0] led;          // выходы проверяемого модуля

    // Экземпляр проверяемого модуля (DUT -- Device Under Test)
    gates dut (.btn(btn), .led(led));

    integer i;
    initial begin
        $dumpfile("tb_gates.vcd");
        $dumpvars(0, tb_gates);
        $display(" a b | AND OR NOT NAND NOR XOR");
        for (i = 0; i < 4; i = i + 1) begin
            btn = i[1:0];
            #10;             // ждём 10 нс, пока сигналы установятся
            // ~led -- потому что светодиоды инверсные
            $display(" %b %b |  %b   %b   %b    %b    %b   %b",
                     btn[0], btn[1], ~led[0], ~led[1], ~led[2],
                     ~led[3], ~led[4], ~led[5]);
        end
        $finish;
    end
endmodule
```

Запуск в терминале:

```
iverilog -o tb_gates tb_gates.v gates.v   # компиляция
vvp tb_gates                              # моделирование
```

Результат (это настоящий вывод симулятора):

```
 a b | AND OR NOT NAND NOR XOR
 0 0 |  0   0   1    1    1   0
 1 0 |  0   1   0    1    0   1
 0 1 |  0   1   1    1    0   1
```

```
 1 1 |  1   1   0   0   0   0
```

Сравните с таблицей 2.1 из главы 2 — всё совпадает.

16.4. Пример 2: мигалка с уменьшенной частотой

Проверять мигалку на реальной частоте долго: чтобы дождаться одного переключения, нужно промоделировать 13,5 млн тактов. Поэтому в модуле `blink` частота вынесена в `parameter`, и тестбенч задаёт `CLK_HZ = 20`: светодиод переключается каждые 10 тактов.

code/02_blink/tb_blink.v

```
// Тестбенч для проекта 2 (Icarus Verilog).
// Чтобы не моделировать 13,5 млн тактов, задаём "частоту" 20 Гц:
// тогда светодиод переключается каждые 10 тактов.
`timescale 1ns/1ps
module tb_blink;
    reg clk = 0;
    always #18.5 clk = ~clk;          // период 37 нс ~ 27 МГц

    wire [5:0] led;

    blink #(.CLK_HZ(20)) dut (.clk(clk), .led(led));

    initial begin
        $dumpfile("tb_blink.vcd");    // файл для GTKWave
        $dumpvars(0, tb_blink);
        $monitor("t = %6.1f нс  cnt = %2d  led = %b", $realtime, dut.cnt, led);
        #1000;                        // моделируем 1000 нс
        $finish;
    end
endmodule
```

Фрагмент вывода:

```
t =    0.0 нс  cnt =  0  led = 111111
t =   18.5 нс  cnt =  1  led = 111111
...
t =  314.5 нс  cnt =  9  led = 111111
t =  351.5 нс  cnt =  0  led = 000000
t =  388.5 нс  cnt =  1  led = 000000
```

После `cnt = 9` (это `HALF - 1`) счётчик сбрасывается, и светодиоды загораются (`000000`). Схема работает как задумано.

16.5. Временные диаграммы в GTKWave

Команда `gtkwave tb_blink.vcd` откроет окно, похожее на рис. 16.3. Слева выбирают сигналы, справа видны их диаграммы.

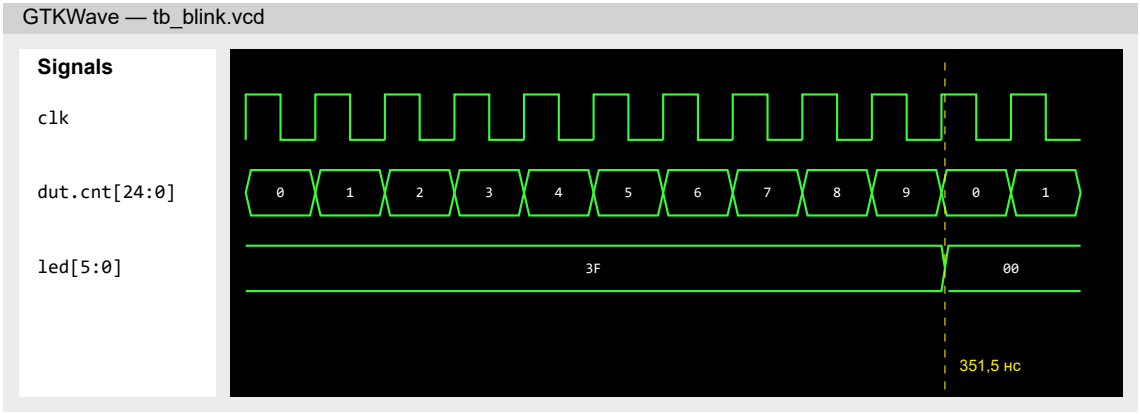


Рис. 16.3.: Так выглядит результат симуляции в GTKWave (схематично)

Симуляция не заменяет плату

Симулятор проверяет *логику*, но не знает о дребезге кнопок, задержках в проводах и ошибках в файле `.cst`. Поэтому последний шаг всегда проверка на реальной плате.

17 Память SDRAM на 64 Мбит

17.1. Зачем ПЛИС внешняя память

Триггеры и блочная память BSRAM очень быстрые, но их мало. Для изображения, звука или большого массива данных нужна память большего объёма. Именно для этого в корпус GW2AR-18 встроена микросхема SDRAM на **64 Мбит = 8 Мбайт**.

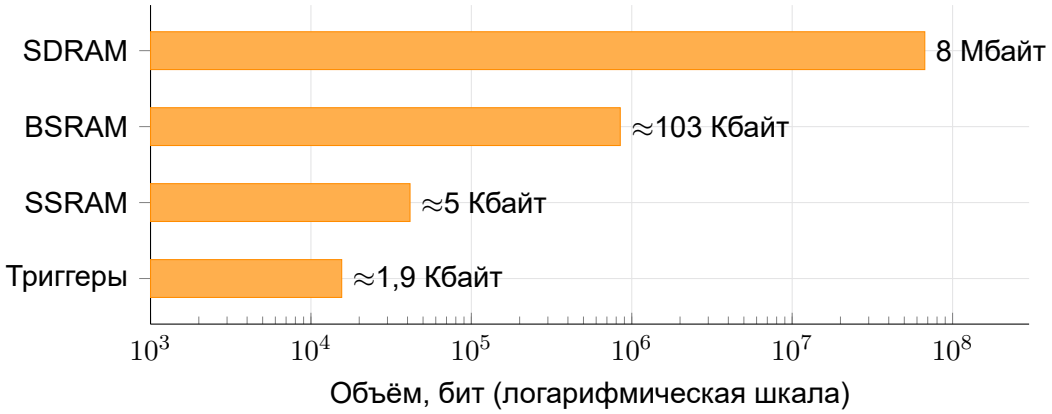


Рис. 17.1.: Сколько данных можно хранить в разных видах памяти Tang Nano 20K

Например, один кадр изображения 640×480 точек по 16 бит на точку занимает

$$640 \cdot 480 \cdot 16 = 4\,915\,200 \text{ бит} \approx 600 \text{ Кбайт},$$

что в шесть раз больше всей блочной памяти ПЛИС, но легко помещается в SDRAM: туда влезает больше десятка таких кадров.

17.2. Как устроена DRAM

Ячейка динамической памяти (DRAM) состоит всего из **одного транзистора и одного конденсатора**. Заряженный конденсатор хранит 1, разряженный — 0. Поэтому DRAM такая плотная и дешёвая. Но у неё есть два неудобства:

- конденсатор постепенно **разряжается**, поэтому каждую строку нужно периодически перезаписывать (**регенерация**, refresh) — все строки за 64 мс;
- чтение разрушает заряд, поэтому строку сначала копируют в быстрый буфер (**открывают**), а по окончании работы записывают обратно (**закрывают**).

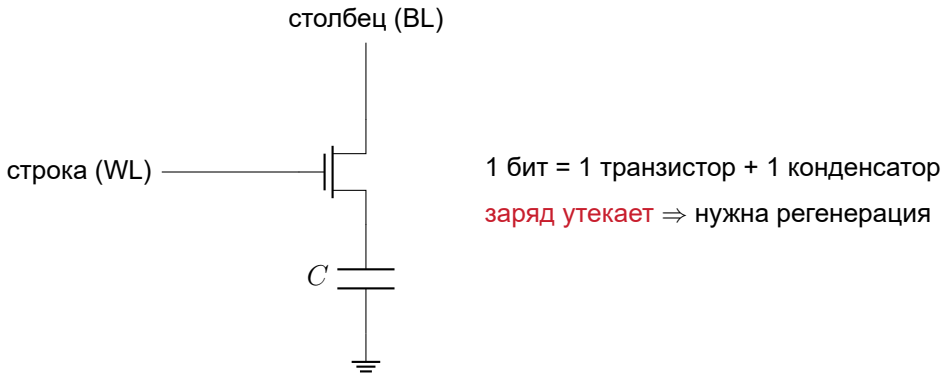


Рис. 17.2.: Ячейка DRAM (упрощённо)

17.3. Организация памяти

SDRAM в GW2AR-18 имеет 32-разрядную шину данных и организована как 4 банка × 2048 строк × 256 столбцов × 32 бита:

$$4 \cdot 2048 \cdot 256 \cdot 32 = 67\,108\,864 \text{ бит} = 64 \text{ Мбит}.$$

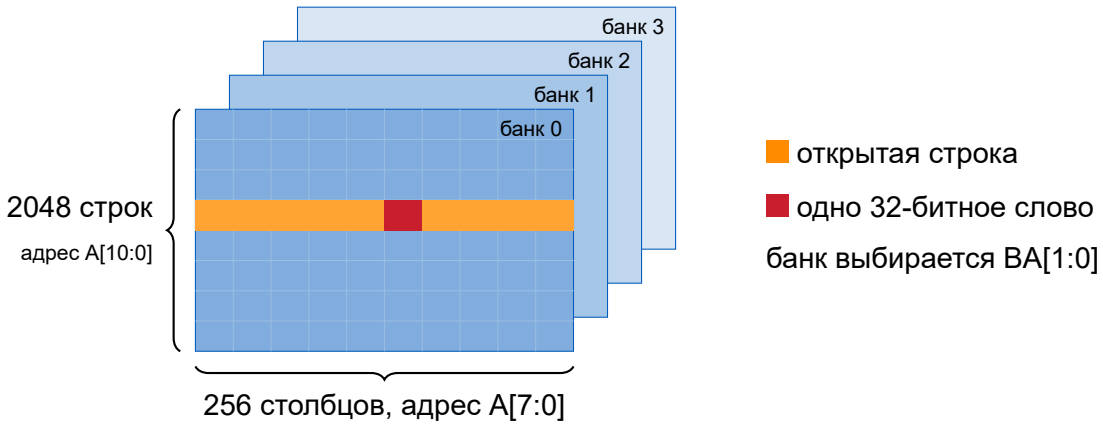


Рис. 17.3.: Организация SDRAM: банк, строка, столбец

17.4. Команды SDRAM

В отличие от BSRAM, в SDRAM нельзя просто подать адрес и получить данные. Микросхема управляется **командами**, которые кодируются комбинацией сигналов `CS#`, `RAS#`, `CAS#`, `WE#` (решётка означает активный низкий уровень) по фронту тактового сигнала.

Таблица 17.1.: Основные команды SDRAM

Команда	CS#	RAS#	CAS#	WE#	Действие
NOP	0	1	1	1	ничего не делать
ACTIVE	0	0	1	1	открыть строку в банке
READ	0	1	0	1	прочитать из открытой строки
WRITE	0	1	0	0	записать в открытую строку
PRECHARGE	0	0	1	0	закрыть строку
AUTO REFRESH	0	0	0	1	регенерация
LOAD MODE	0	0	0	0	настройка (CAS latency, длина пакета)

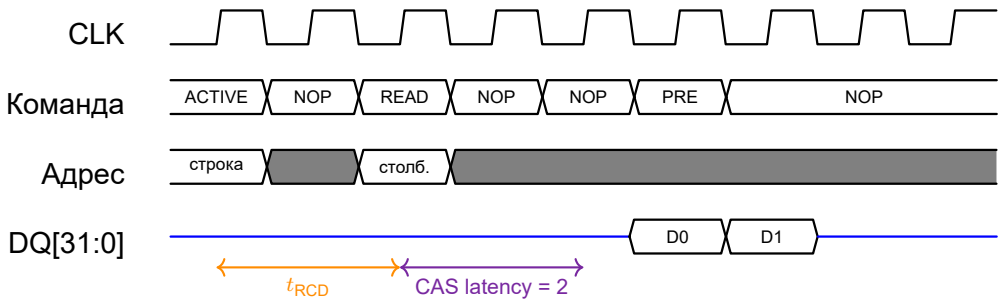


Рис. 17.4.: Чтение из SDRAM: открыть строку (ACTIVE), подождать t_{RCD}, выдать команду READ, через CAS latency тактов получить данные

17.5. Контроллер SDRAM

Помнить обо всех командах и задержках при каждом обращении к памяти неудобно. Поэтому между логикой пользователя и SDRAM ставят **контроллер** — модуль (обычно конечный автомат, как в проекте 5), который:

- после включения питания инициализирует микросхему;
- превращает простые запросы «прочитай слово по адресу X» в последовательность команд ACTIVE → READ → PRECHARGE;
- сам вовремя выполняет регенерацию.

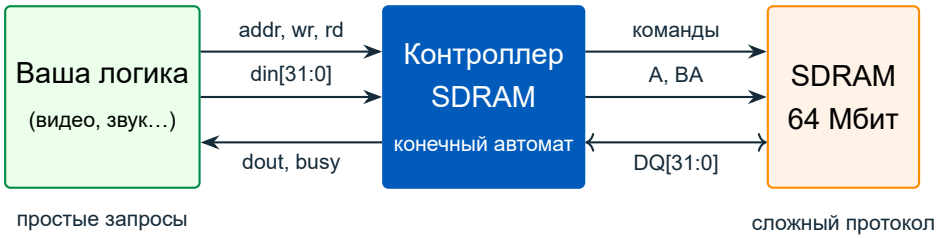


Рис. 17.5.: Контроллер скрывает сложность протокола SDRAM

17.5.1. Порты встроенной памяти

В проекте Gowin встроенная SDRAM подключается к модулю верхнего уровня через порты со специальными (зарезервированными) именами. Номера выводов в `.cst` для них указывать не нужно: соединение находится внутри корпуса.

```
module top (
    input  wire        clk,
    // --- Встроенная SDRAM (имена изменять нельзя!) ---
    output wire        O_sdram_clk,
    output wire        O_sdram_cke,
    output wire        O_sdram_cs_n,    // CS#
    output wire        O_sdram_ras_n,   // RAS#
    output wire        O_sdram_cas_n,   // CAS#
    output wire        O_sdram_wen_n,   // WE#
    output wire [3:0]  O_sdram_dqm,     // маска байтов
    output wire [10:0] O_sdram_addr,    // адрес строки/столбца
    output wire [1:0]  O_sdram_ba,      // номер банка
    inout  wire [31:0] IO_sdram_dq      // данные
);
```

Где взять контроллер

- В Gowin EDA есть готовый IP-блок: **Tools** → **IP Core Generator** → **Memory Interface** → **SDRAM Controller**. Мастер сам сгенерирует модуль с простым интерфейсом.
- Существуют открытые контроллеры, написанные для Tang Nano 20K энтузиастами; их используют, например, проекты ретро-приставок NESTang и SNESTang.
- Хорошее упражнение повышенной сложности — написать свой контроллер, следуя рис. 17.4 и документации на микросхему.

17.6. Пропускная способность

При тактовой частоте SDRAM, например, 100 МГц и шине 32 бита пиковая скорость передачи равна

$$B = 100 \cdot 10^6 \text{ Гц} \cdot 32 \text{ бит} = 3{,}2 \text{ Гбит/с} = 400 \text{ Мбайт/с}.$$

Реальная скорость ниже из-за открытия и закрытия строк и регенерации. Чем длиннее пакеты последовательных адресов, тем ближе скорость к пиковой.

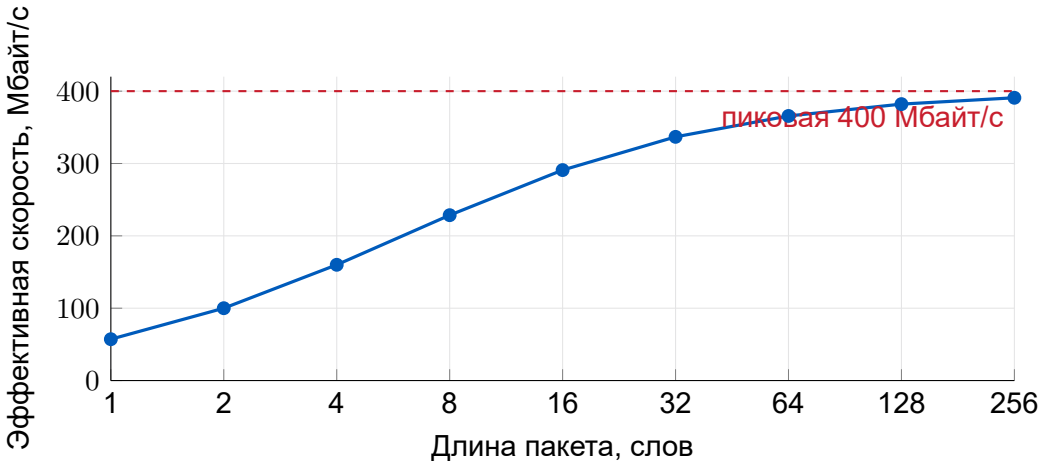


Рис. 17.6.: Оценка скорости обмена при 100 МГц: на каждый пакет приходится около 6 «служебных» тактов (ACTIVE, t_{RCD}, CAS latency, PRECHARGE)

Главная мысль

SDRAM даёт много памяти (8 Мбайт), но требует контроллера: строки нужно открывать и закрывать, а память регулярно регенерировать. Самая высокая скорость получается, когда данные читаются и пишутся длинными последовательными пакетами.

18 Задания для самостоятельной работы

Задания расположены по возрастанию сложности. Звёздочками отмечена сложность: ⋆ — разминка, ⋆⋆ — нужно подумать, ⋆ ⋆ ⋆ — мини-проект.

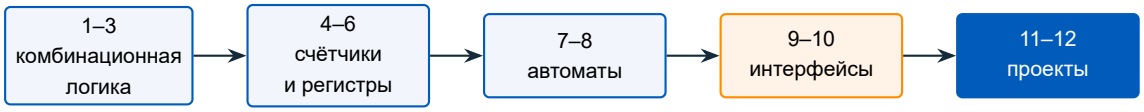


Рис. 18.1.: Карта заданий

Задание 1 ⋆

Реализуйте на двух кнопках **полусумматор**: `led[0]` — сумма, `led[1]` — перенос. Составьте таблицу истинности и проверьте её на плате.

Задание 2 ⋆

Выразите функции И, ИЛИ и НЕ **только через И-НЕ** и опишите получившуюся схему операторами `~(... & ...)`. Сколько LUT займёт результат по отчёту синтезатора? Объясните, почему.

Задание 3 ⋆⋆

Сделайте **шифратор**: при нажатии одной кнопки на светодиодах отображается двоичный код 1, другой — 2, обеих — 3. Используйте `always @(*)` и `case`. Не забудьте `default`!

Задание 4 ⋆

Двоичный счётчик на 6 светодиодах, прибавляющий единицу 4 раза в секунду. Через сколько секунд он переполнится?

Задание 5 ⋆⋆

Счётчик в **коде Грея**: соседние значения отличаются ровно одним битом. Подсказка: `gray = bin ^ (bin >> 1)`. Сравните мигание со счётчиком из задания 4.

Задание 6 ★★

Бегущий огонь, у которого кнопка S1 меняет направление, а S2 — скорость (3 скорости по кругу).

Задание 7 ★★

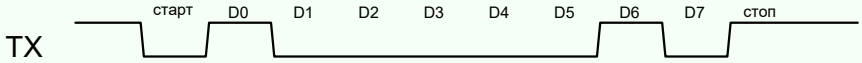
Кодовый замок. Правильная последовательность нажатий: S1, S1, S2, S1. После верного ввода загораются все светодиоды, после ошибки — мигает `led[5]`, и ввод начинается заново. Нарисуйте граф переходов автомата.

Задание 8 ★★

Секундомер: S1 — старт/стоп, S2 — сброс. На светодиодах — число секунд в двоичном виде (до 63).

Задание 9 ★★★

Передатчик UART. Раз в секунду отправляйте в компьютер символ `'A'` на скорости 115 200 бод (вывод 69). Кадр UART: стартовый бит (0), 8 бит данных младшим битом вперёд, стоповый бит (1). Длительность бита — $27\,000\,000/115\,200 \approx 234$ такта. Смотрите результат в любом терминале (PuTTY, `screen`, Arduino IDE).



Передача символа ’A’ = `0x41` = $\texttt{0100 0001}_2$

Задание 10 ★★★

Приёмник UART: принятый из компьютера байт показывайте на светодиодах (младшие 6 бит). Вход — вывод 70. Не забудьте синхронизатор!

Задание 11 ★★★

Игра «Реакция». Через случайное время (используйте сдвиговый регистр с линейной обратной связью, LFSR) загорается светодиод. Игрок должен как можно быстрее нажать S1. Время реакции в десятках миллисекунд выводится на светодиоды или отправляется по UART.

Задание 12 ★★★

Генератор ШИМ для сервопривода. Сервоприводы и регуляторы моторов дронов управляются импульсами длительностью 1–2 мс с периодом 20 мс. Сделайте генератор, в котором кнопки S1 и S2 меняют длительность импульса с шагом 0,1 мс, и выведите сигнал на любой свободный вывод GPIO. Проверьте осциллографом или с настоящим сервоприводом.

A Шпаргалка

A.1. Verilog на одной странице

```
module name #(parameter integer N = 8) (   // параметр
    input  wire         clk,
    input  wire [N-1:0] a,
    output wire         y,
    output reg  [N-1:0] q
);
    localparam [1:0] S0 = 2'd0;            // константа

    wire t = &a;                           // свёртка И
    assign y = t ? 1'b1 : 1'b0;            // мультиплексор

    always @(*) begin                      // комбинационная логика: =
        case (a[1:0])
            2'd0:    ...;
            default: ...;
        endcase
    end

    always @(posedge clk) begin            // триггеры: <=
        if (a == 0) q <= 0;
        else        q <= q + 1'b1;
    end

    other_module u1 (.in(a), .out(...));   // экземпляр модуля
endmodule
```

A.2. Типичные ошибки

Симптом	Вероятная причина
Светодиоды горят «наоборот»	Забыли, что LED горят от 0: нужна инверсия
Ничего не работает, схема «пустая»	Выход модуля ни к чему не подключён, и синтезатор удалил всю логику; неверные имена в `.cst`
Счётчик считает одно нажатие много раз	Нет антидребезга и/или детектора фронта
Предупреждение *latch inferred*	В `always @(*)` не хватает `else` или `default`

Симптом	Вероятная причина
Два регистра не обмениваются значениями	В `always @(posedge clk)` использовано `=` вместо `<=`
Число не помещается, счётчик не доходит до нужного значения	Мала разрядность регистра: проверьте $2^n > N$
Multiple drivers	Одному сигналу присваивают значение в двух разных `always` или `assign`
После выключения питания схема пропадает	Загрузили в SRAM; для постоянной работы загрузите во Flash

A.3. Выводы Tang Nano 20K

Сигнал	Вывод	Назначение
`clk`	4	27 МГц
`btn[0]`, `btn[1]`	88, 87	кнопки S1, S2
`led[0..5]`	15–20	светодиоды (активный 0)
`uart_tx`, `uart_rx`	69, 70	UART через USB
`ws2812`	79	RGB-светодиод

A.4. Полезные ссылки

- Документация Sipeed: https://wiki.sipeed.com/hardware/en/tang/tang-nano-20k/nano-20k.html
- Примеры Sipeed: https://github.com/sipeed/TangNano-20K-example
- Среда Gowin EDA: https://www.gowinsemi.com
- Открытые инструменты (OSS CAD Suite): https://github.com/YosysHQ/oss-cad-suite-build
- Проект Apicula: https://github.com/YosysHQ/apicula
- Интерактивный учебник по Verilog HDLBits: https://hdlbits.01xz.net